

有界示向误差下无线电干扰源的 覆盖认证搜索与快速清除

摘要

无线电干扰源数量、接收半径以及定向源的发射方向均未知，因而快速定位不仅是多条示向线求交，还必须解决误差约束、全区域发现与正确停止三个问题。本文采用“确定性覆盖保证正确性、观测驱动调度降低耗时”的统一框架。

针对问题一，将每次示向度转为正向楔形半平面约束，给出空集、有界和无界情形的判别及顶点对直径算法。构造由合法测向观测实现的边长 36 米等边三角形反例，证明直径为区域直径的圆不一定覆盖区域；以最小包围圆半径代替区域直径作为一次清除判据。

针对问题二，利用首次接收到信号隐含的接收半径下界，推导无需已知目标距离的三圆交候选区域。以交会夹角与距离共同刻画测向条件，并以候选点的后验可行域包围圆半径进行采样极小极大评分，区分可证明的接收安全性与启发式的最优性。

针对问题三，建立七点全向覆盖及其精确半径公式，以 1200 米环保留 31.10 米接收余量；针对问题四，以原点和错相双十二点环构成 25 点骨架，用 36 个闭三角形证明任意发射方向不漏检。每频道分别维护清除或全覆盖无源证书，局部保留有界测向与完整光学后备。

在 Q3、Q4 各 10 轮官方模拟器演练（非正式测试）中，原策略分别清除 139/139 与 138/138 个源；移动占虚拟时间约 74.38% 和 73.20%。据此提出 `field`：保留原路线作为控制，加入多初解、2-opt 与单点搬移；Q3 另用负观测的保守凸外包、原地预检测及认证清除点接近减少移动。冻结后，在每间 1000 个新合成世界上，相对原 `refined` 的单源耗时由 258.99、542.41 秒降至 250.17、508.60 秒，降低 3.40% 和 6.23%，两题均全清。新确认、压力和误差替代试验共 9400 次，全部逐动作复算计时；聚集源部分组的区间仍含零，不主张普遍占优。这些提升是本地配对证据，不冒充新策略的官方成绩。补充的非配对官方模拟器演练显示 Q3 的 `field` 为 263.550883 秒/源而原 `refined` 为 237.998263 秒/源，Q4 的 `field` 为 484.966300 秒/源而原 `refined` 为 531.177617 秒/源；由于案例不同且新日志未归档，本文不作因果推断。综合这一诊断与本地配对证据，程序默认在 Q3 使用 `refined`、在 Q4 使用 `field`，并允许显式覆盖。

关键词：有界误差；交会定位；最小包围圆；覆盖证书；主动测向；定向源搜索

1 问题重述

1.1 问题背景

无线电干扰源排查需要在有限时间内完成搜索、定位和清除。人工携带测向设备作业时，通常先根据示向度判断目标的大致方向，再通过多点交会缩小位置范围，最后近距离确认并清除。机器狗能够代替人员进入危险区域，但它只能根据当前位置的检测结果行动，无法预先获知全部目标的位置和数量。

本题的目标区域为半径 1800 米的圆域，机器狗从圆心出发，移动速度为 5 米/秒，测向机初始频道为 1。每个频道至多有一个干扰源，有效接收半径在 1000 至 1500 米之间，示向误差不超过 1° 。全向源在接收范围内均可检测，定向源还要求检测点位于其发射前方。进入目标 20 米范围后，机器狗可完成光学定位与清除。搜索过程因此同时受到几何位置、发射方向和操作耗时的影响。

1.2 问题要求

根据题目条件，四个问题可整理为：

1. 已知检测点和示向度，建立交会定位区域，计算其直径，并判断直径圆能否覆盖该区域。
2. 已知一个全向源的首次示向度，给出第二检测点的选择方法和候选区域，使两次观测获得较好的定位效果。
3. 对数量未知的 10 至 16 个全向源，设计一只机器狗的搜索、定位与清除策略，在全清条件下缩短任务时间。
4. 将策略扩展到含定向源的情形，处理发射背面无信号、接收中断和区域边界等问题，并通过模拟器演练评价结果。

2 问题分析

2.1 问题一与问题二的分析

示向度存在误差，单次检测给出的不是一条准确射线，而是一段角度范围。多次检测形成若干扇形楔区，它们的交集表示可能的源位置。问题一的重点是把角度条件转为可计算的平面约束；计算区域直径后，还需检查整个区域能否被一个清除圆盘覆盖。

第二检测点既影响两次测向的交会角，也影响移动距离和接收条件。沿首次示向方向前进虽然省路，但两条示向线接近平行时，距离方向的误差难以压缩。因此问题二先确定能够继续接收到信号的区域，再比较该区域内不同横向测点的定位效果。

2.2 问题三与问题四的分析

问题三中，机器狗不能以“暂时没有发现目标”结束任务。需要先构造覆盖整个目标区域的检测点集，再把未知频道的扫描任务与已知源的清除任务合并安排。局部定位

使用问题一的可行区域，测向收缩不足时改用光学覆盖，完成从全域搜索到近距清除的衔接。

问题四增加了发射方向限制。即使距离很近，机器狗也可能位于发射背面；尤其是圆周上朝外发射的源，需要在目标区域外安排检测点。为此，本文用双环点集覆盖各个方向，并在信号中断后向最近一次有信号的位置回退，继续定位。

四问：从几何覆盖到定向定位

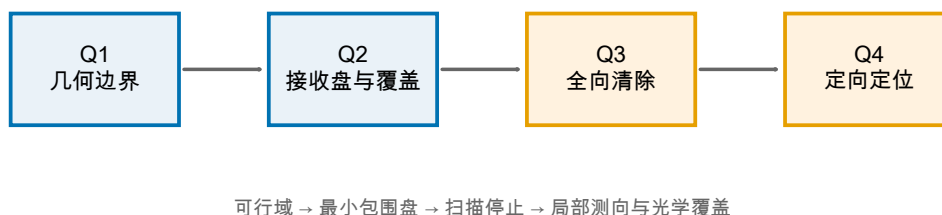


图 1: 四个问题的建模关系：交会区域支撑局部定位，覆盖点集支撑全域搜索。

2.3 评价指标

记总行程为 L ，检测次数为 M ，换频次数为 S ，失败清除次数为 F ，成功清除源数为 K 。按照题目计时规则，任务的虚拟总时间为

$$T = \frac{L}{5} + 5M + S + 3F + 5K. \quad (1)$$

成功清除的 5 秒已经包含光学定位与激光清除；清除指令不改变测向机频道。设实际源数为 N ，两项评价指标分别为清除率 K/N 和平均清除时间 T/K 。模型首先满足全清要求，再降低总时间。实验同时报告逐局等权平均 $\frac{1}{n} \sum_j T_j / K_j$ 与汇总指标 $\sum_j T_j / \sum_j K_j$ ，主策略比较采用前者。程序现实运行时间另行记录。

3 模型假设

在题目给出的源静止、频道唯一、发射方向固定等条件下，采用以下计算约定：

1. 同一坐标、同一频道的测向误差保持固定。不同地点的误差均在题目规定的范围内，不通过原地重复测量平均降噪。
2. 示向度保留两位小数，计算容差取 1.005° ，包含 1° 测向误差与舍入影响。
3. 用外切正 180 边形表示目标圆域，边界向外最大偏差小于 0.275 米；光学规划半径取 19.9 米，预留 0.1 米计算余量。

4. 在线策略只使用已返回的检测和清除结果。源总数与类型只在测试结束后用于计算评价指标。

定向源的接收条件写为

$$\|s - g_c\| \leq R_c, \quad n_c^\top (s - g_c) \geq 0. \quad (2)$$

全向源只需满足距离条件。可检测且距离不超过 5 米时返回近源信息，可在原地清除；光学清除不受发射方向限制。

4 符号说明

表 1: 主要符号及含义

符号	含义	单位或范围
$\mathcal{D}$	干扰源所在圆域	半径 1800 米
c, g_c	频道编号、该频道源坐标	$c = 1, \dots, 20$
R_c, n_c	接收半径、发射前向单位向量	$R_c \in [1000, 1500]$ 米
s_i, θ_i	检测点及示向度	米、度
α	示向度容差	计算取 1.005°
$\mathcal{P}_c$	频道 c 的可能位置多边形	在线更新
$D(P), r_*(P), z_*(P)$	区域直径、最小包围圆半径及圆心	米
V, V_c	全部骨架索引、频道 c 已检测索引	有限集合
L, M, S, F, K	行程、检测、换频、失败清除及成功清除数	米、次
T, N	虚拟总时间、实际源数	秒、个

5 问题一：交会区域直径与清除判据

5.1 从示向度到正向半平面交

定义 $u(\theta) = (\cos \theta, \sin \theta)$, $u_i^\pm = u(\theta_i \pm \alpha)$, 二维叉积为 $\text{cross}(a, b) = a_x b_y - a_y b_x$ 。一次观测对应

$$W_i = \left\{ g : \begin{array}{l} \text{cross}(u_i^-, g - s_i) \geq 0, \\ \text{cross}(u_i^+, g - s_i) \leq 0, \\ u(\theta_i)^\top (g - s_i) \geq 0 \end{array} \right\}. \quad (3)$$

前两个半平面在 $\alpha > 0$ 时已经给出正向楔形，第三个条件显式排除精确零误差情形下的背向射线。使用正弦、余弦表达使 359.9° 和 0.1° 之间的跨零问题无需分支处理。

原始交会区域为 $P = \bigcap_i W_i$ 。若题意需要同时利用已知目标区域或距离上限，可再取交集；但添加先验后的区域不是原始无限楔形交，不能混淆二者的直径。

5.2 空集、无界集和有界多边形的算法

将全部约束写成 $Ag \leq b$ 。一个透明、易核实的实现如下。

1. 解二维可行性线性规划。无可行解时报告观测或条件矛盾，不返回虚假的零直径。
2. 分别最大化及最小化 x, y 。任一坐标目标无界，则非空区域无界，直径为 $+\infty$ ；四个坐标上下界都有限则区域有界。
3. 对每对非平行边界直线求交，保留满足全部半平面约束的交点；去除数值重复点，按凸边界排序。
4. 求所有顶点对距离的最大值。单点直径为 0，线段直径为两端点距离。

配套 `intersection.py` 直接完成上述分类，不用任意大包围盒冒充原区域的有界性。若约束数为 m ，直线交点枚举并逐个检验的朴素复杂度为 $O(m^3)$ ；通常只需少数测向，足够迅速。大规模情形可改为按极角排序的半平面交和旋转卡壳，几何阶段降至 $O(m \log m)$ ，但这不是本题在线计算的瓶颈。

结论 1 (直径的顶点刻画). 若 $P = \text{conv}\{v_1, \dots, v_n\}$ 非空、有界，则

$$D(P) = \max_{i,j} \|v_i - v_j\|. \quad (4)$$

证明. 任取 $p = \sum_i a_i v_i$ 与 $q = \sum_j b_j v_j$ ，其中权重非负且各自和为 1，有

$$\|p - q\| = \left\| \sum_{i,j} a_i b_j (v_i - v_j) \right\| \leq \sum_{i,j} a_i b_j \|v_i - v_j\| \leq \max_{i,j} \|v_i - v_j\|.$$

反向不等式由各顶点本身属于 P 得到。 □

5.3 直径圆不能总是覆盖：一个真实可实现的反例

边长为 D 的等边三角形直径就是 D ，最小包围圆半径却是 $D/\sqrt{3} > D/2$ 。这不仅是任意多边形反例，还能由本题 $\pm 1^\circ$ 测向直接生成。

取 $D = 36$ ， $A = (0, 0)$ 、 $B = (36, 0)$ 、 $C = (18, 18\sqrt{3})$ 。依次对三条逆时针有向边取单位方向 e_j ，令

$$S_j = V_j - L e_j, \quad L = 18\sqrt{3} \cot 2^\circ - 18. \quad (5)$$

测向读数取边方向加 1° ，即 $1^\circ, 121^\circ, 241^\circ$ 。每个楔形下边界沿对应边、上边界经过对角顶点；三个楔形都包含该三角形，而三条下边界的内半平面交恰为该三角形。所有三角形点距对应检测站小于 1000 米，故可取合法接收半径 1000 米。

实际计算得到直径约 36.000000 米，最小包围圆半径约 20.784610 米。于是即使 $D < 40$ ，仍不一定能在某一点以 20 米光学半径保证一次清除。

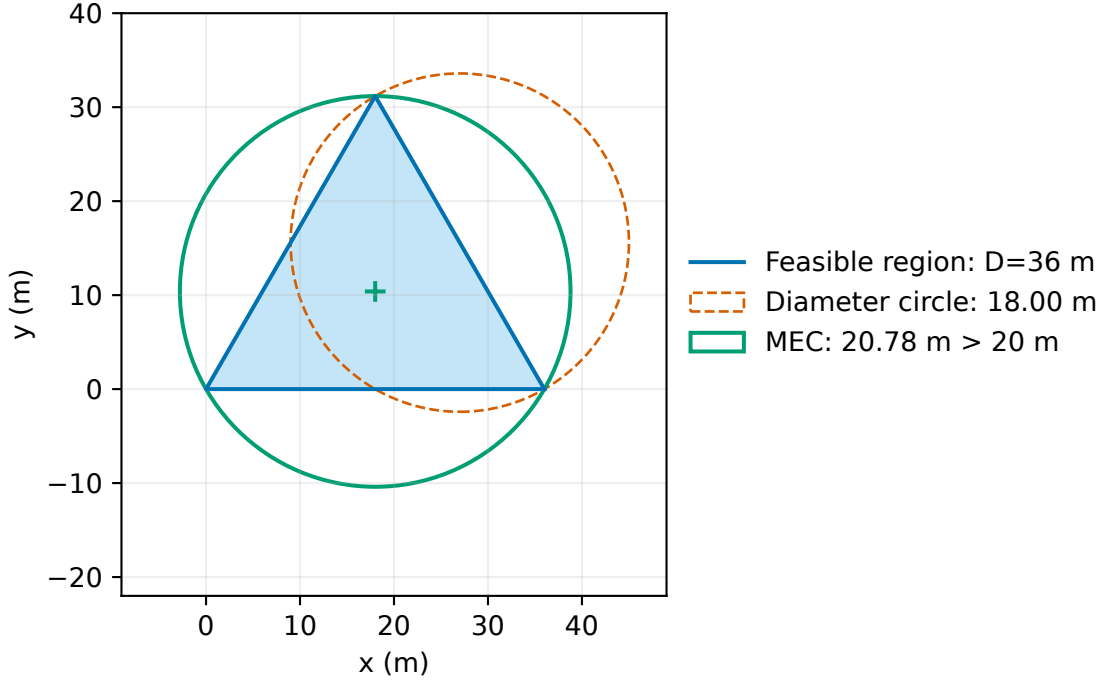


图 2: 直径为 36 米的可实现三角形反例。任何半径 18 米的圆都无法覆盖三顶点；正确包围半径约 20.785 米。

5.4 用最小包围圆直接连接定位与清除

定义

$$r_*(P) = \min_z \max_{p \in P} \|p - z\|, \quad z_*(P) = \arg \min_z \max_{p \in P} \|p - z\|. \quad (6)$$

圆盘是凸集，包含多边形所有顶点即包含整个多边形。最小包围圆由一个、两个或三个边界支持点确定，可用随机增量支持点算法求解 [2]。本文实现对近共线三点重算高精度行列式，并在最终半径上保留向外余量。

平面 Jung 界为

$$\frac{D(P)}{2} \leq r_*(P) \leq \frac{D(P)}{\sqrt{3}}. \quad (7)$$

两点支持时左端取等；三点支持时圆心在支持三角形内，三段圆心角中最大的一段至少 120° ，对应弦长至少 $\sqrt{3}r_*$ ，得到右端。等边三角形取右端等号。因此：

- $r_*(P) \leq 20$ 是存在一次保证清除点的充要几何条件；执行点取 $z_*(P)$ 。
- 仅使用直径时， $D(P) \leq 20\sqrt{3}$ 是充分条件， $D(P) > 40$ 则必不可能一次保证覆盖。
- $20\sqrt{3} < D(P) \leq 40$ 时必须进一步算包围圆，不能凭直径直接清除。

5.5 在线使用的保守多边形

在问题三、四中，每次有效示向度还说明 $\|g - s_i\| \leq 1500$ 。为保持纯线性裁剪，可使用安全松弛

$$u(\theta_i)^\top (g - s_i) \leq 1500. \quad (8)$$

这不是把接收半径错误设为 1500，而是利用其已知上界。若初始外切多边形为 A_{180} ，则维护

$$\mathcal{P}_c = A_{180} \cap \bigcap_{i \in I_c} (W_i \cap \{g : u(\theta_i)^\top (g - s_i) \leq 1500\}). \quad (9)$$

真实源初始在 A_{180} 内，每条新增约束也包含它，故归纳得到 $g_c \in \mathcal{P}_c$ 。一次观测裁剪四个半平面， m 次观测最多形成 $180 + 4m$ 条边，复杂度 $O(180m + m^2)$ 。空交集是需要定位原因的异常，而不是允许重置状态的信号。

6 问题二：第二检测点的安全候选区域与选点策略

6.1 单次测向为什么没有唯一最优第二点

只有角度而没有距离时，目标位于长而窄的扇形中。若第二点沿首次示向方向移动，两条示向线可能几乎平行或反平行，径向误差难以压缩。若仅为了接近 90° 交会而绕行很远，则会增加移动时间，未必有利于题目总目标。

令 $r_i = \|g - s_i\|$ 、 $u_i = (g - s_i)/r_i$ 、 $n_i = (-u_{iy}, u_{ix})$ 。对小误差线性化，有

$$|n_i^\top \delta g| \leq \alpha r_i. \quad (10)$$

两条误差带交形成的平行四边形面积近似为

$$A \approx \frac{4\alpha^2 r_1 r_2}{|\sin \beta|}, \quad (11)$$

其中 β 为两条真实视线夹角， α 以弧度计。固定距离时， 90° 交会最有利；距离也变化时，应同时考虑缩短 r_2 、避免共线及移动代价。该式是局部解释，不能代替精确楔形裁剪。

若额外采用独立高斯角误差的辅助似然，才可写出 Fisher 信息 $J = \sigma^{-2} \sum_i n_i n_i^\top / r_i^2$ 及 $\det J = \sin^2 \beta / (\sigma^4 r_1^2 r_2^2)$ 。没有这种似然假设时，这至多是加权最小二乘的局部信息代理，不能当成题目提供的真实误差统计模型。最终安全性仍由式(9)保证。

6.2 利用首次已接收事实推导三圆交

取第一次检测点 S_1 为局部原点，首次读数方向为 u ，法向为 n ，第二点写为

$$S_2 = S_1 + tu + bn. \quad (12)$$

若可能真实距离为 r ，首次已经接收说明

$$R \geq \max(1000, r), \quad (13)$$

而不仅仅是 $R \geq 1000$ 。设 F_1 为当前全部可能源位置，则保证全向源在第二点仍有信号或返回近源的精确鲁棒条件为

$$S_2 \in \bigcap_{g \in F_1} B(g, \max(1000, \|g - S_1\|)). \quad (14)$$

用完整扇形 $r \in [0, 1500]$ 、角差 $\varphi \in [-\alpha, \alpha]$ 覆盖真实 F_1 ，可得闭式充分候选域。

结论 2 (三圆交接收保证). 令 $u_{\pm} = u(\theta_1 \pm \alpha)$ 。以下候选域内的任一点, 对整个放宽扇形都能保证全向接收:

$$\mathcal{C} = S_1 + [B(0, 1000) \cap B(1000u_-, 1000) \cap B(1000u_+, 1000)]. \quad (15)$$

对这一完整扇形, 式(15)也恰好是式(14)的交集; 若再利用圆域等更强先验, 它可能不是最大的安全候选域。

证明. 写 $q = S_2 - S_1$ 。必要性由 $r = 0$ 和 $r = 1000, \varphi = \pm\alpha$ 给出。反之, 两端圆约束意味着 $q^T u_{\pm} \geq \|q\|^2 / 2000 \geq 0$ 。小于 180° 的中间短弧上, $q^T u_{\varphi}$ 的最小值在端点, 因此对所有允许角度都满足同样下界。若 $r \leq 1000$, 距离 $\|q - ru_{\varphi}\|$ 关于 r 是凸函数, 两端 $r = 0, 1000$ 均不超过 1000, 内部也不超过 1000; 若 $r \geq 1000$,

$$\|q - ru_{\varphi}\|^2 - r^2 = \|q\|^2 - 2rq^T u_{\varphi} \leq 0.$$

故距离不超过 $\max(1000, r)$, 由首次接收所隐含的半径下界即可保证可检测。 $\square$

6.3 显式候选区域与可执行搜索

在 (t, b) 局部坐标中, 三圆交等价于

$$\begin{cases} t^2 + b^2 \leq 1000^2, \\ (t - 1000 \cos \alpha)^2 + (b - 1000 \sin \alpha)^2 \leq 1000^2, \\ (t - 1000 \cos \alpha)^2 + (b + 1000 \sin \alpha)^2 \leq 1000^2. \end{cases} \quad (16)$$

令

$$b_{\max}(t) = \min \left\{ \sqrt{1000^2 - t^2}, \sqrt{1000^2 - (t - 1000 \cos \alpha)^2} - 1000 \sin \alpha \right\}. \quad (17)$$

根号有定义且 $b_{\max} \geq 0$ 时, $0 < t < 1000$ 、 $0 < |b| \leq b_{\max}(t)$ 形成首次方位两侧的横向候选区。排除 $b = 0$ 用于避免把共线点作为唯一方案, 而非禁止为了接近目标沿轴行进。

程序按下式构造 42 个候选:

$$\begin{aligned} t &\in \{125, 250, 375, 500, 625, 750, 875\} \text{ 米}, \\ b &= \pm \kappa b_{\max}(t), \quad \kappa \in \{1/4, 1/2, 3/4\}. \end{aligned} \quad (18)$$

对每个候选模拟允许的第二读数, 计算新多边形最小包围圆; 优先选择采样最坏半径较小的点, 可再以本步移动和检测代价打破并列。所有这些点的接收安全性来自定理2, 不是来自采样。

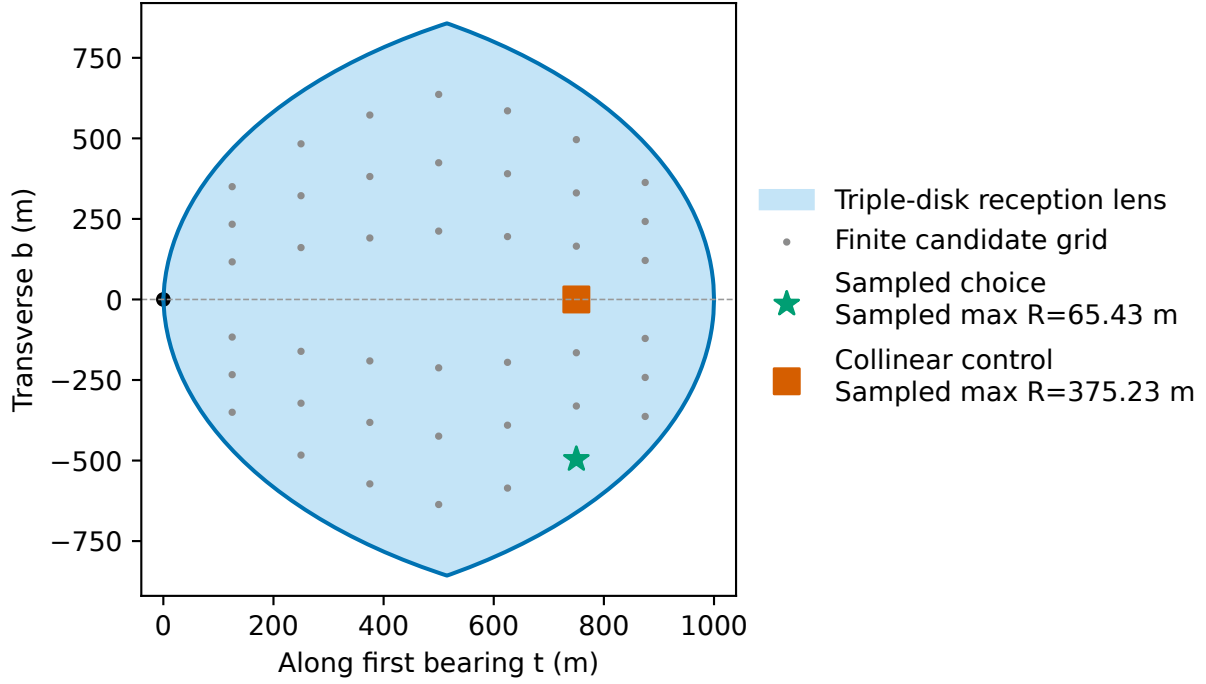


图 3: 第二点的三圆交候选区域。横向基线改善交会条件，但候选排序仍需兼顾移动耗时和未知径向距离。

在 $S_1 = (0, 0), \theta_1 = 0$ 的归一化示例中，取 60 个径向位置、3 个首次真实角差和 3 个第二次测向误差进行比较。程序得到采样最佳几何候选约为 (750, -496.078) 米，采样最坏包围半径约 65.426 米；共线对照 (750, 0) 的对应值约 375.231 米。两者本步耗时分别约 184.844 秒和 155 秒。这说明增加约 29.844 秒移动检测成本可显著改善此例的径向辨识，但第二次测向后仍未必能一次清除。

该示例是候选域与评分算法的演示，不是题目给定正式案例，也不是连续最坏情形或总时间的全局最优解。二维方位误差的连续极值、未知距离分布及后续代价若未严格处理，不能将网格最小值写成“最优定位策略”。

7 问题三：全向干扰源的搜索、定位与清除

7.1 任务特点与总体思路

问题三中的干扰源均为全向发射源。机器人在检测点收到信号，便可据此把该频道的源位置限制在一个保守可行域内；对尚未发现的频道，若在一个覆盖保证为 1000 米的检测点未收到信号，则真源必满足 $\|g - s\| > 1000$ 米。因而本问在扫描与定位之间采用边发现、边调度：一旦某频道出现正观测，立即插入该频道的定位任务，同时继续保留尚未履行的七点扫描义务。搜索、定位和清除三阶段使用同一频道状态，避免把“已经发现”误当成“已经完成”。

设源区域为

$$\mathcal{D} = \{g \in \mathbb{R}^2 : \|g\| \leq R\}, \quad R = 1800 \text{ m},$$

题给最小接收半径为 1000 米，最大接收半径为 1500 米。本文把七点环作为问题三的基

准发现骨架，采用环半径 $d = 1200$ 米；随后按照当前位置与任务代理点的距离统一安排检测任务，问题三取代理目标中的 $\gamma = 0$ 。这一构造是完整覆盖下的可执行方案，不把七点下界误读为允许半径 1000 米时任意布局的全局最优性结论。

7.2 七点环构造与覆盖半径

取原点及正六边形环上的六个检测点：

$$p_0 = (0, 0), \quad p_{k+1} = d u(k\pi/3), \quad k = 0, \dots, 5, \quad u(\theta) = (\cos \theta, \sin \theta). \quad (19)$$

七个位置同时承担每个频道的发现义务。令 $g = ru(\theta)$ 。总有一个环点与 g 的极角差不超过 $\pi/6$ ，因此在固定半径 r 的圆周上，到最近检测点的最坏距离为

$$f_d(r) = \min \left\{ r, \sqrt{r^2 + d^2 - \sqrt{3}dr} \right\}. \quad (20)$$

第一项对应原点，第二项对应相邻环点的角平分线。两项在 $r = d/\sqrt{3}$ 处相等；在此之前原点更近，在此之后环点更近。后一项的平方是 r 的凸二次函数，所以在区间端点达到最大值。在参数范围 $1200 \leq d \leq 900\sqrt{3}$ 内，七点构型的精确覆盖半径为

$$\rho(d) = \max_{g \in \mathcal{D}} \min_{0 \leq j \leq 5} \|g - p_j\| = \max \left\{ \frac{d}{\sqrt{3}}, \sqrt{R^2 + d^2 - \sqrt{3}Rd} \right\}. \quad (21)$$

这个证明覆盖了圆盘内部、外边界以及六个扇区的接缝，不依赖网格采样。 $d = 900\sqrt{3}$ 时两项均为 900 米，给出传统的 900 米覆盖基线。

当 $d = 1200$ 米时，边界项控制最坏距离：

$$\begin{aligned} \rho(1200) &= \sqrt{1800^2 + 1200^2 - \sqrt{3} \cdot 1800 \cdot 1200} \\ &\approx 968.901572 \text{ m}, \\ 1000 - \rho(1200) &\approx 31.098428 \text{ m}. \end{aligned} \quad (22)$$

因此在理想点位下，七点环距离题给最小接收半径仍有约 31.10 米余量。若理想点与实际停靠点的总欧氏偏差不超过 η ，由三角不等式，实际最坏距离不超过 $\rho(1200) + \eta$ ；例如 $\eta \leq 1$ 米时仍严格小于 1000 米。这里的余量只说明全向发现骨架的距离鲁棒性，并不等同于测向角误差或局部清除半径。

900 米半径下七点的必要性也可作简洁说明。距原点为 t 的一个半径 900 米圆盘，在源圆周上能够覆盖的交弧半宽 β 满足

$$\cos \beta = \frac{R^2 + t^2 - 900^2}{2Rt} \geq \frac{\sqrt{R^2 - 900^2}}{R} = \cos(\pi/6).$$

故每个非中心圆盘的弧宽至多为 $\pi/3$ ，覆盖整条边界至少需要六个，并且等号迫使六个圆心位于半径 $900\sqrt{3}$ 的正六边形环上；这些圆盘又不能覆盖原点，所以还必须增加中心点。这个下界仅针对 900 米覆盖要求。允许半径改为 1000 米后，弧宽约束改变，本文不声称七点是所有布局的全局最优。

7.3 骨架路径与联合调度

从原点出发依次访问六个环点，环上相邻点弦长为 d ，任意两个不同环点的距离不小于 d 。因此开放骨架路径的最短长度为

$$L_{\text{ring}}^{\min}(d) = 6d. \quad (23)$$

沿环依次访问即可达到该下界；插入中间停靠点不能缩短折线。最终 $d = 1200$ 米时，纯骨架长度为 7200 米。这个数值不包含定位支线、清除动作和频道切换，不能直接当作每局实际节省时间。

执行时，不把七个检测点孤立地排成一条预先固定的路线。设当前位置为 q ，每一个未完成骨架点或已发现未清除频道都形成一个任务代理。骨架任务的代理半径为 0；已知频道以其当前保守域最小包围圆中心 z_c 及半径 r_c 表示。对任务排列 π ，采用开放路线代理目标

$$J_\gamma(\pi) = \|q - z_{\pi_1}\| + \sum_{j=1}^{n-1} \|z_{\pi_{j+1}} - z_{\pi_j}\| + \gamma r_{\pi_1}. \quad (24)$$

问题三的基准 refined 联合调度取 $\gamma = 0$ ，即只按几何行程安排首个任务，不对首个不确定区域另加半径惩罚。该目标只是排序代理，不替代真实的测量、定位和清除耗时。

算法先以最近插入法形成完整任务排列，再进行有界的连续区间反转改进。每次只执行当前排列首项；动作完成后，根据新的位置、频道状态和观测结果重建任务集。骨架任务与频道任务即使坐标重合也不合并，以确保所有七个覆盖义务均被实际履行。这样，联合调度只改变访问次序，不删去任何用于认证未知频道的检测位置。

7.4 局部测向与清除

在七点扫描得到正观测后，记录该频道的测向角，并以初始位置圆盘与各次测向约束的交集作为保守可行域 P_c 。每次测向的角度误差按题设和实现的保守边界扩张半平面；因此真实源始终满足

$$g_c \in P_c. \quad (25)$$

问题三的局部定位最多使用 6 次固定测向。每次测量均从当前可行域和已拒绝位置出发选择新的接入点；基准 refined 收到 `direction` 后加入相应的角度约束，收到 `no_signal` 则保留原可行域并记录失败探测，不能把失败位置当作新的正约束。`field` 在问题三对有效无信号进一步使用下述保守外包。收到 `near` 时进入原有的清除流程。

记当前最小包围圆为 $B(z, r)$ ，机器人位置为 q 。每一轮先检查清除条件：若 q 到 P_c 全部顶点的距离不超过 19.9 米，则原地清除；若 $r \leq 19.9$ 米，则到 z 清除。若 $19.9 < r \leq 38$ 米，先在中心作一次预测清除；这种尝试不具有全域保证，失败后立即转入完整光学覆盖，不反复测向。阈值 38 米用于在有限观测与光学搜索之间切换，而 19.9 米才是保守清除条件。

对更大的区域，取最近一个收到正示向的位置 s^+ ，令 $e = (z - s^+)/\|z - s^+\|$ ， $n = (-e_2, e_1)$ ，沿示向法向构造左右两个候选点

$$p_{\pm} = z \pm \delta n, \quad \delta = \min\{120, \max\{25, 0.22r\}\} \text{ m}.$$

从尚未失败的候选点中选择较近者测向；该横向偏移使新的示向线与原示向线形成交角，有助于压缩长条形可行域。最多 6 次测向用完，或发生无法继续选点的退化情形时，转入完整光学覆盖。每次调用局部过程均以该频道成功清除为正常返回条件；若观测不一致或完整覆盖仍未清除，则报告异常并中止，不能以未清除状态返回调度后重新获得六次预算。

清除判定使用半径 19.9 米的保守覆盖，而不是把直径小于 40 米简单当作半径 20 米证书。若当前凸多边形的全部顶点都落入以计划清除点 a 为中心、半径 19.9 米的圆盘，则由凸性有 $P_c \subseteq B(a, 19.9)$ ，结合式(25)可知真源距 a 不超过设备的 20 米清除半径，并留有 0.1 米余量。默认 `refined` 不启用沿射线缩短接入的 `approach`；该动作只属于下述 `field` 对照增强。几何上满足条件不等于已经清除，只有接口返回该频道的 `success` 才可更新为完成。作为对照增强，`field` 版本还可在满足条件时做一次原地预检测，并在问题三使用负观测的保守凸外包：将严格位于 1000 米圆盘内部的正十六边形各边外侧保留，再取保留片的凸包。对全向源，`no_signal` 排除的是该 1000 米接收圆盘内部的真源位置；但该外包只是效率层的保守收缩，不能改变七点扫描义务。安全近侧清除同样只缩短已经由当前可行域保证的接入段。上述两项属于对照增强，不改变本文默认的 `refined` 基准。

当局部测向达到上限仍有 $r > 19.9$ 米时，采用矩形网格覆盖整个 P_c ，而非只覆盖其顶点。取当前可行域直径方向为正交坐标轴，令包围矩形为 $[x_-, x_+] \times [y_-, y_+]$ ，并取安全半径 $\bar{r} = 19.9 - \varepsilon$ 。令

$$h = \sqrt{2} \bar{r}, \quad N_x = \max\left(1, \left\lceil \frac{x_+ - x_-}{h} \right\rceil\right), \quad N_y = \max\left(1, \left\lceil \frac{y_+ - y_-}{h} \right\rceil\right),$$

将矩形分成 $N_x N_y$ 个小矩形，在第 (i, j) 格中心

$$a_{ij} = \left(x_- + \frac{i + 1/2}{N_x} (x_+ - x_-), y_- + \frac{j + 1/2}{N_y} (y_+ - y_-) \right)$$

执行光学检测。每个小矩形的两边均不超过 h ，故任意 w 落在该格内时

$$\|w - a_{ij}\| \leq \frac{1}{2} \sqrt{h^2 + h^2} = \bar{r} < 19.9.$$

$$P_c \subseteq \bigcup_{i=0}^{N_x-1} \bigcup_{j=0}^{N_y-1} B(a_{ij}, 19.9). \quad (26)$$

包围矩形覆盖 P_c ，所以这些圆盘覆盖 P_c 的每一个点；真源属于 P_c ，必被至少一个光学点覆盖。最后仍须以接口返回的 `success` 作为清除证书。 ε 只吸收坐标舍入和边界浮点余量，不改变 19.9 米阈值。

7.5 扫描规则与逐频道停止条件

未知频道必须在七个骨架位置逐一检测。已建立正测向轨迹但尚未清除的频道，不因骨架点再次无信号而被判定为无源，而是保留为独立的定位清除任务；这一区分避免把已知源从扫描记录中错误删除。已知频道收到的负观测不填入未知频道的完成集合，也不替代清除反馈。

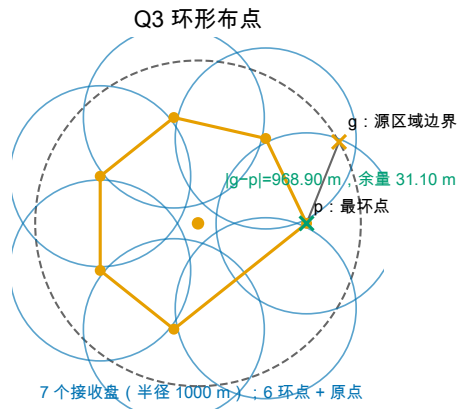


图 4: 问题三七点环发现骨架。圆盘为半径 1800 米的源域，环半径取 1200 米；图中几何位置为示意，实际机器人停靠误差由覆盖余量吸收。

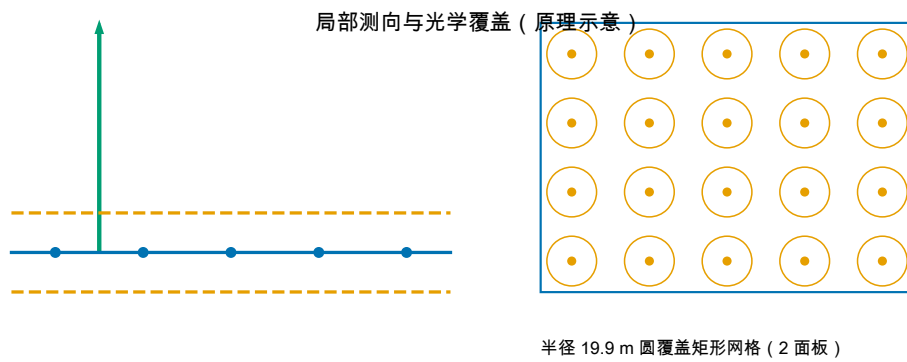


图 5: 局部测向与光学覆盖示意。正观测逐步收缩保守可行域，最终以矩形覆盖并逐顶点验证 19.9 米清除条件；示意线段不表示观测轨迹。

对每个频道 c 定义完成证书

$$\text{Cert}_c = \begin{cases} C, & \text{已收到频道 } c \text{ 的 success 清除反馈,} \\ N, & \text{从未发现该频道源, 且七个位置均得到有效无信号,} \\ U \text{ 或 } D, & \text{尚未确定, 或已发现但未清除.} \end{cases} \quad (27)$$

只有在全部 20 个频道均取得 C 或 N 证书, 或者已经收到 16 个不同频道的 C 证书时, 任务才可停止:

$$[\forall c \in \{1, \dots, 20\}, \text{Cert}_c \in \{C, N\}] \quad \text{或} \quad [\#\{c : \text{Cert}_c = C\} = 16]. \quad (28)$$

式(28)中的第二种情形使用题给源数上界, 统计的是不同频道的成功清除数, 而不是清除尝试数。队列暂时为空、连续无信号、或动作异常都不能替代证书。由于全向覆盖定理保证未知频道的真源若存在必落入至少一个有效接收圆盘, 七点扫描全部无信号时才可形成 N 证书; 一旦频道进入 D 状态, 则必须以 C 完成, 不能再转为 N 。

7.6 算法流程与结果说明

问题三的实施步骤为:

1. 初始化 20 个频道状态与七点骨架任务, 当前位置设为原点;
2. 按 J_0 对未完成骨架任务和已知未清除频道统一排序, 执行最近插入及有界 2-opt 得到开放路线;
3. 对当前首项执行真实测量或清除动作。未知频道不跳过骨架位置, 已知频道仅作为定位清除任务调度;
4. 对正观测维护保守凸可行域, 最多进行 6 次局部测向, 随后进行矩形光学覆盖和清除尝试;
5. 仅在收到 success 后记为 C ; 未发现频道在全部七点有效无信号后记为 N ;
6. 重建任务集并重复, 直至满足式(28)。

结果评价统一按实际执行的移动距离、测量次数、换频次数、清除尝试及动作时间计算。七点构造提供全域发现保证, 联合调度减少的是任务间的无效绕行, 而不是检测义务; 局部测向和矩形光学覆盖保证的是已发现频道的清除路径。Q3 官方演练的全清结果和局均时间在模型检验章中统一列示, 局部测向、光学覆盖与路线机制的分项结果也在该章进行比较。基准 refined 与 field 对照增强分开统计, 以区分几何覆盖构造与效率调度的作用。

8 问题四：定向干扰源的搜索、定位与清除

8.1 问题特征与建模思路

问题四与问题三的根本差别不在于接收距离, 而在于信号具有方向性。设源位置为 g , 单位发射法向为 n , 机器人在 p 处检测; 除距离条件 $\|p - g\| \leq 1000$ 外, 还需满足

$$n^\top(p - g) \geq 0. \quad (29)$$

因此，距离覆盖并不能推出可探测性：位于源附近但处于其发射背面的点，可能完全没有信号。特别是源在圆域边界且朝外发射时，圆内的近邻点都可能落在背向半平面。问题三的七点环只证明每个源附近存在一个距离足够小的点，不能保证式 (29)，所以问题四必须同时覆盖位置和方向。

我们采用“先建立方向鲁棒发现骨架，再对已发现频道局部定位”的两层结构。第一层不依赖在线知道 g 或 n ，只要求所有可能源位置和所有发射方向均有一个安全检测点；第二层使用实际测向反馈逐步收缩可行域，并在接收成功后完成清除。这样，搜索保证与局部效率分开，任何扫描排序都不会改变覆盖证书。

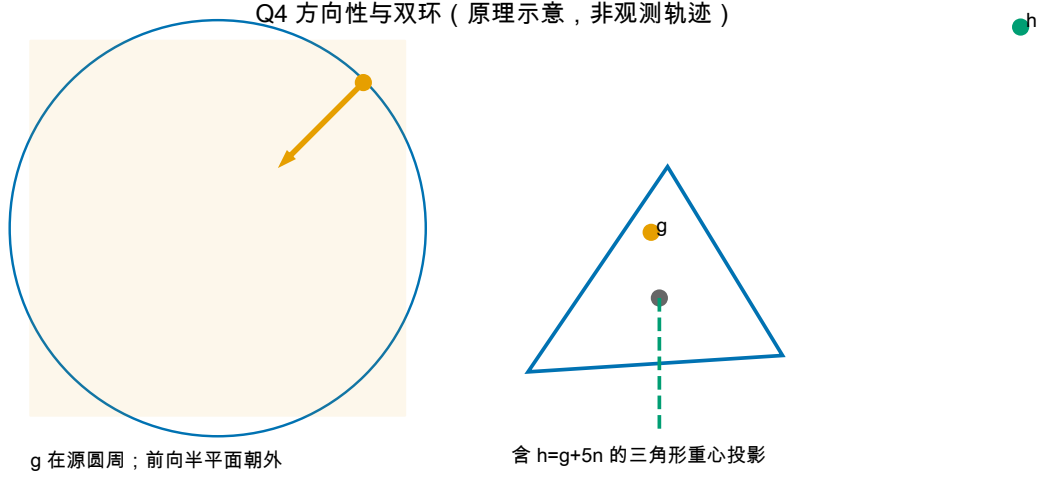


图 6: 25 点错相双环的方向覆盖示意。外环点位于源域外，辅助点 $h = g + 5n$ 仅用于证明，不表示观测轨迹；图中几何关系为示意图。

8.2 25 点错相双环发现骨架

取 $a = 940$ m、 $b = 1870$ m，令 $u(\theta) = (\cos \theta, \sin \theta)$ ，布置原点、内环 12 点和外环 12 点：

$$\begin{aligned} p_0 &= O = (0, 0), \\ I_k &= au(k\pi/6), \quad k = 0, \dots, 11, \\ E_k &= bu(k\pi/6 + \pi/12), \quad k = 0, \dots, 11. \end{aligned} \quad (30)$$

内环相邻点相差 30° ，外环相对内环错开 15° 。13 个点位于源圆域内，12 个点位于圆域外；外部点不是多余的行程点，而是为方向覆盖提供前方候选。点的编号只表示扫描义务，不规定实际访问顺序。

将下标按模 12 计算，外正十二边形由 E_k 的凸包给出。其内部由 36 个闭三角形剖分：

$$\begin{aligned} T_k^{(1)} &= \text{conv}\{O, I_k, I_{k+1}\}, \\ T_k^{(2)} &= \text{conv}\{I_k, E_k, I_{k+1}\}, \quad k = 0, \dots, 11. \\ T_k^{(3)} &= \text{conv}\{I_{k+1}, E_k, E_{k+1}\}, \end{aligned} \quad (31)$$

第一类填充内十二边形，后两类填充内外两环之间的带状区域。由于外环相对内环错开半个扇区，连接边在各个 15° 扇区内不交叉；且 $a < b \cos(\pi/12)$ ，内环点严格位于相应

外边的内侧。因此这些闭三角形没有孔洞，覆盖整个外正十二边形。

记 $c = \cos(\pi/12)$ 、 $s = \sin(\pi/12)$ 。三角形边长只有四类：

$$\ell_0 = a, \quad \ell_I = 2as, \quad \ell_E = 2bs, \quad \ell_{IE} = \sqrt{a^2 + b^2 - 2abc}. \quad (32)$$

由 $c > 0.9659$ 、 $0.25 < s < 0.259$ 得

$$bc > 1805, \quad \ell_0 < 993, \quad \ell_I < 486.92, \quad \ell_E < 968.66, \quad \ell_{IE} < 993. \quad (33)$$

故每个三角形的直径 $D_T < 993$ m，同时外十二边形包含半径 1805 m 的圆盘 $B(O, 1805)$ 。前一个界控制候选点到三角形内辅助位置的距离，后一个界保证辅助位置仍在剖分覆盖范围内。

8.3 方向覆盖的关键证明

结论 3 (25 点的方向鲁棒覆盖). 对任意 $g \in \mathcal{D}$ 和任意单位向量 n ，式 (30) 至少存在一个顶点 v ，使得理想点和总位置误差不超过 1 m 的实际点 v' 满足

$$n^\top(v' - g) \geq 4 \text{ m}, \quad \|v' - g\| < 999 \text{ m}. \quad (34)$$

证明. 引入仅用于证明的辅助点

$$h = g + 5n. \quad (35)$$

由于 $\|g\| \leq 1800$ 且 $\|n\| = 1$ ，有 $\|h\| \leq 1805$ ，从而 h 落在外十二边形内，必属于至少一个闭三角形 T 。将其写成该三角形三个顶点的重心组合 $h = \sum_{i=1}^3 \lambda_i v_i$ ，其中 $\lambda_i \geq 0$ 且 $\sum_i \lambda_i = 1$ 。线性投影给出

$$\sum_{i=1}^3 \lambda_i n^\top(v_i - g) = n^\top(h - g) = 5. \quad (36)$$

因此至少有一个正权重顶点 v 满足 $n^\top(v - g) \geq 5$ 。同时 v, h 同在 T 中，故

$$\|v - g\| \leq \|v - h\| + \|h - g\| \leq D_T + 5 < 998. \quad (37)$$

若实际检测点满足 $\|v' - v\| \leq 1$ ，柯西不等式和三角不等式分别给出

$$n^\top(v' - g) \geq 5 - 1 = 4, \quad \|v' - g\| < 998 + 1 = 999.$$

这同时保证严格位于发射前方和最小接收圆盘内。□

证明中的 h 是方向覆盖的等价转换：任意方向只需把源沿发射方向平移 5 m，再由有限三角形剖分找到一个顶点。它不代表机器人实际访问的位置，也不需要算法估计方向。源在圆周、辅助点落在边界、重心系数为零等退化情形均由闭三角形包含。1 m 是坐标计算、舍入和实际停靠误差的总预算，不可在不同误差来源间重复使用。

8.4 有限可行路线与扫描执行

25 点骨架只要求每个点完成一次必要检测，访问顺序则交给有限路线组合。作为明确可行的开放路线，可按

$$O, I_0, I_1, \dots, I_{11}, E_{11}, E_{10}, \dots, E_0$$

访问全部点，其长度为

$$L_{25} = a + 22(a + b) \sin(\pi/12) + \sqrt{a^2 + b^2 - 2ab \cos(\pi/12)}. \quad (38)$$

这是一条有限的开放可行路线；实际访问顺序还需结合频道清除、换频和局部定位反馈动态确定。

实际执行采用有限路线组合。对每个未完成骨架任务和已发现未清除频道建立代理任务：骨架点半径取 0，已知频道取其当前可行域最小包围圆中心 z_c 及半径 r_c 。从当前位置 q 出发，用

$$J_\gamma(\pi) = \|q - z_{\pi_1}\| + \sum_{j=1}^{n-1} \|z_{\pi_{j+1}} - z_{\pi_j}\| + \gamma r_{\pi_1}, \quad \gamma = 1 \quad (39)$$

选择有限候选排列，保留最近插入、受限 2-opt、Or-opt1 以及两种不同首任务的近邻初解；最多进行 $\min(n, 12)$ 轮最佳改善。每次只执行排列首项，执行后依据真实反馈重建任务集。所有操作均保持完整任务排列，联合调度只改变顺序，不删除方向覆盖所需的 25 个检测义务。

扫描频道采用 `useful` 规则：未知频道在每一个尚未完成的骨架点都必须检测；已有轨迹的频道仅当 $r_c > 19.9$ 且 $\|p - z_c\| - r_c \leq 1500$ m 时补测，否则跳过。边界等号属于“有用”一侧。这个判据只表达保守区域可能与接收范围相交，不保证定向源正面可见，因此不能把跳过检测当作负证据。已发现频道最终仍需收到清除成功反馈。

8.5 局部定位、失信号恢复与光学兜底

骨架检测或共享观测得到正信号后，沿用有限局部测向过程。对每个频道，只有在当前包围圆半径 $r_c > 38$ m 且可能进入接收范围、并且此前未在完全相同坐标执行过预检测时，才至多执行一次原地 `pmeasure`；该动作位于六次固定测向预算之外。预检测支付检测和换频费用，但不增加移动，也不保证定向源响应。

每次有效测向按返回的方向和距离约束收缩可行域。若下一次测向暂时失去信号，则：(1) 保留最近一次有效正测点 s 作为恢复基准；(2) 在其方向轴上按既有反射/收缩规则生成下一个候选点；(3) 重新检测，并在获得新正信号后用新的 s 更新轴。恢复点不把失败点与 1500 m 作平均。固定测向至多六次；若仍未获得成功清除，依次执行矩形光学兜底：以正测点和方向约束形成的可行域构造保守矩形，以半径 19.9 m 的蛇形覆盖逐点访问候选中心，并在每个候选处复核全部可行域顶点；收到 `success` 立即停止该频道。

对问题四，正面规则是：定向源的无信号反馈只表示当前姿态下未收到信号，不能删去其 1000 m 位置圆盘；因此禁用负圆盘推断，并关闭 `approach_clear`。可行域始终保留正观测约束及题设先验。矩形覆盖是对仍可能包含真源的区域进行有限穷举，不依赖对失信号原因的额外判断。

8.6 停止条件、成本与有限终止

令 $V = \{0, \dots, 24\}$ 为完整骨架索引集, V_c 为频道 c 已接受的实际骨架检测位置。频道证书定义为

$$\text{Cert}_c = \begin{cases} C, & \text{收到频道 } c \text{ 的成功清除反馈,} \\ N, & \text{从未发现该频道且全部 } V \text{ 上有效检测均无信号,} \\ U \text{ 或 } D, & \text{尚未确定或已发现但未清除.} \end{cases} \quad (40)$$

当未知频道完成全部 25 点且全无信号时, 定理 3 排除了其存在, 故可记为 N ; 一旦频道进入 D , 只能通过成功清除转为 C , 不能再改判无源。合法停止为

$$[\forall c \in \{1, \dots, 20\}, \text{Cert}_c \in \{C, N\}] \quad \text{或} \quad [\#\{c : \text{Cert}_c = C\} = 16]. \quad (41)$$

第二项统计 16 个不同频道的成功清除; 队列为空或长时间未发现新源都不是停止理由。

成本由实际移动、检测、清除、换频和光学尝试共同计入式 (1); 25 点骨架相对 31 点方案少 120 次潜在频道检测, 但提前清除、`useful` 跳过、预检测和局部定位会改变实际次数。路线组合的距离矩阵为 $O(n^2)$, 有限插入与局部交换搜索为 $O(n^3)$; 它们只用于产生可控的候选顺序, 不改变覆盖保证。每个正常局部任务至多一次预检测、六次测向, 随后成功清除或进入有限矩形覆盖; 完整骨架只有 25 个点, 最多处理 16 个真实源。因此在接口正常、误差预算成立且不被现实期限中断时, 搜索、定位、清除和停止均在有限动作内完成。Q4 的运行结果与成本由后续“模型检验”章节统一报告, 本章不单列独立实验数字。

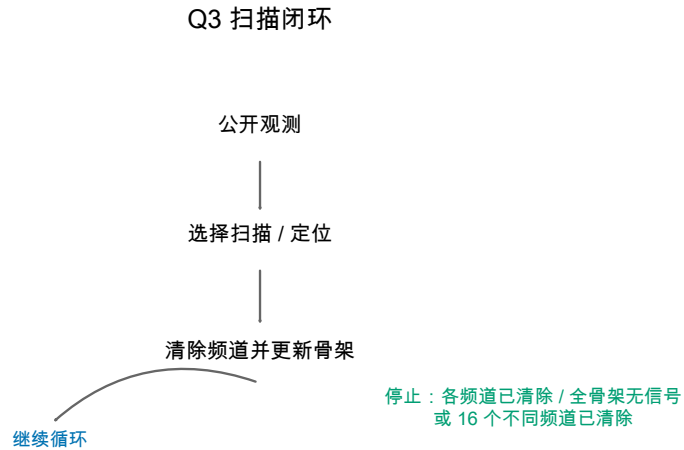


图 7: 机器狗根据公开反馈循环执行扫描、定位和停止判断。

9 模型求解与结果分析

9.1 合成环境与实验设置

本地模拟器按照题目公开的动作和反馈规则生成测试场景, 实验中源数取 10 至 16, 位置位于半径 1800 米圆域内, 接收半径取 1000 至 1500 米。问题三使用全向源, 问题

四包含定向源。策略只读取接口返回的信息，源坐标、类型和总数仅用于实验结束后的统计。

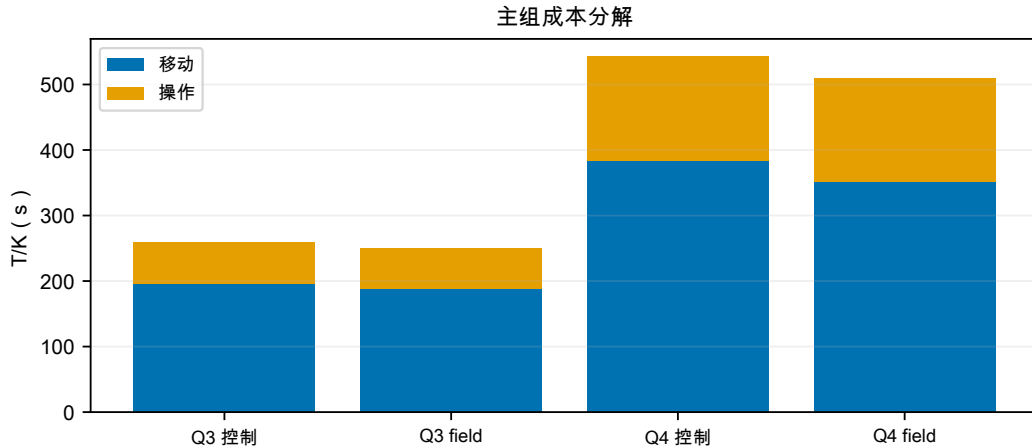


图 8: 主测试组的移动与操作成本分解。

10 官方演练观察：结果与可识别的瓶颈

用户提供了 2026 年 9 月 12 日在官方模拟器完成的 Q3、Q4 各 10 轮演练：每局包含脱敏请求日志、控制器摘要及模拟器退出后结果文件，案例关联由 `manifest.csv` 固定。执行时采用原 `refined`，不是新增 `field`。事后公开的源数和类型仅用于评价，绝不传入在线规划器。`official_analysis.py` 逐请求与响应独立复算移动距离、检测、换频及清除费用，并按微秒累计与每步虚拟时刻核对；客户端派生的距离和换频字段只用于交叉检查。

表 2: 20 轮官方演练的汇总；每题 10 轮均全清，时间单位秒/源

问题	局数	全清源数	局等权 T/K	汇总 T/K	移动占比/%	检测/局
Q3	10	139	238.00	235.82	74.38	128.90
Q4	10	138	531.18	519.38	73.20	311.80

其中“局等权”为 $\frac{1}{10} \sum_j T_j / K_j$ ，“汇总”为 $\sum_j T_j / \sum_j K_j$ ，后者给予源数更多的局更高权重。问题三 139 个、问题四 138 个源均清除；局等权均值分别为 238.00、531.18 秒/源。问题四第 8 局只有 10 个源，仍需完成 25 点搜索，因此 $T/K = 768.87$ 秒/源，说明小源数案例的固定发现成本不能忽略。

表 3: 20 轮官方演练逐局结果； N 为退出后标签，不是正式测试分母

题号	轮次	案例编码	N	K	T/s	$T/K/(s/源)$
Q3	1	BUWJ-NM9Q-E3FP-KBNQ	16	16	3501.15	218.82
Q3	2	XSTE-PSFN-9F8U-8P2D	12	12	2929.34	244.11
Q3	3	WQXS-VYHK-4B2N-4NWU	13	13	3230.83	248.53
Q3	4	3YB4-CC66-WNAD-XVQ9	15	15	3301.25	220.08
Q3	5	AKGC-XRSQ-KQ42-DE4U	14	14	2867.24	204.80
Q3	6	F22D-5NT7-9XRW-7DY6	13	13	3119.85	239.99
Q3	7	FDS3-ZJ7Q-V3E9-2QDN	11	11	3334.99	303.18
Q3	8	XMZP-GEJC-W9KA-UTWY	14	14	3452.13	246.58
Q3	9	8V3W-N88T-ED46-PXCE	15	15	3294.33	219.62
Q3	10	HGVD-7ZTC-XNGP-2JRT	16	16	3748.24	234.26
Q4	1	ZGME-Z9ZY-UQDJ-UZRF	15	15	7091.90	472.79
Q4	2	AQVF-W7A2-YAFY-F6JT	16	16	7385.70	461.61
Q4	3	HS4Z-3F9D-BE4W-FUBW	13	13	7233.66	556.44
Q4	4	BFSU-HJRZ-2WQX-63MP	14	14	7461.23	532.95
Q4	5	RFFB-U23B-VMS7-ASRE	15	15	6420.23	428.02
Q4	6	AJFX-AY9F-VETM-UBJ8	15	15	7089.98	472.67
Q4	7	CR6D-5DV5-WCJQ-6N3F	16	16	7677.93	479.87
Q4	8	TY94-4B9M-J2RU-EDHA	10	10	7688.73	768.87
Q4	9	5J2D-BATC-W2P2-BNQ2	13	13	7151.24	550.10
Q4	10	CENN-WQ49-E4D5-9JJ7	11	11	6473.24	588.48

移动约占 Q3 的 74.38%、Q4 的 73.20%，失败光学清除仅占 0.11%、0.12%。因此优先改进移动与观测价值，而不是以减少失败尝试次数这一小分项代替总时间目标。20 轮未出现 near、HTTP 拒绝或传输重试；它们支持正常协议联通，不支持“官方已经验证故障恢复”的结论。Q3、Q4 在已发现频道后分别仍有 223 和 530 次无信号反馈，尤其不能对 Q4 据此删去 1000 米位置圆盘。

严格回放与反事实边界。 历史日志没有真实源位置、接收半径或发射方向；成功清除点距源至多 20 米，不能作为源坐标真值。严格回放只在新请求与记录路径、频道、坐标一致时返回原反馈，一旦分岔立即停止，不为未测位置猜反馈。原算法在本机可完整匹配 8/20 局，其余在任务顺序或对称测点选择处分岔；这与跨平台浮点近似及近等价几何决策相容，但仅凭日志不能唯一确定原因。20 局的实际费用仍全部由原始坐标和响应核验，不依赖策略回放成功。故以下新策略收益来自独立合成配对，不是这 20 局的官方反事实成绩；新 field 仍需另做官方演练。

11 新策略的独立配对验证与收益来源

11.1 冻结规则、样本分工和实验分母

实测日志用于识别移动占比高、重复无信号和清除接入等瓶颈，不作为可随意查询的世界。开发阶段仅使用 6100000 至 6100149 的 150 个合成世界/问，Q3 七个、Q4 八

个有限配置共 2250 次运行；比较原地补测、Q3 负约束、路线组合、首项半径权重及安全清除接近。只增加补测、硬限制补扫次数或轻微缩环均不足以解释主要收益；最终保留原 compact25 和两题 0.22 固定探测比例，不将此前临时参数试探当作正式改进。

冻结后使用 7100000 至 7100999 的每间 1000 个新世界。Q3 对照、仅路线、路线与信息、完整 field 四配置，Q4 对照和 field 两配置，共 6000 次；三种压力位置、三种误差场、两策略、两题各 50 对 (7200000 至 7200049)，共 1800 次；随机位置的另两种误差各 200 对 (7210000 至 7210199)，共 1600 次。合计 9400 次本地运行，全部完成全清并逐动作独立核对微秒计时。它们与 20 轮官方演练、早期 10700 次研究和 2250 次开发分别计数，不能相加称为独立世界数。

统计以每世界的 T_j/K_j 为等权单位，并按同一世界作差 $d_j = T_j^{\text{refined}}/K_j - T_j^{\text{field}}/K_j$ 。配对均值差的近似 95% 区间为 $\bar{d} \pm 1.96s_d/\sqrt{n}$ 。这说明合成分布下的均值不确定性，不是未来官方成绩的置信区间。参数、骨架与策略在本次确认前冻结，确认结果不再用于回调参数。

11.2 相对已演练 refined 的直接比较

表 4: 每间 1000 个新合成世界的配对结果，单位秒/源；不是官方反事实成绩

问题	策略	均值	p_{90}	最坏	全清
Q3	原 refined	258.99	305.38	370.28	1000/1000
Q3	仅路线改进	254.07	301.86	352.51	1000/1000
Q3	路线与信息	251.13	299.49	340.73	1000/1000
Q3	新 field	250.17	298.55	338.79	1000/1000
Q4	原 refined	542.41	682.86	825.03	1000/1000
Q4	新 field	508.60	642.48	760.05	1000/1000

问题三从 258.99 降至 250.17 秒/源，降低 3.40%；问题四从 542.41 降至 508.60 秒/源，降低 6.23%。节省均值分别为 8.82 和 33.81 秒/源，近似 95% 区间分别为 [8.01, 9.62] 与 [31.61, 36.01]；分别 827/1000 和 843/1000 局更快。两题均 1000/1000 全清，不以降低清除率换时间。问题四 p_{90} 由 682.86 降至 642.48 秒/源，样本最坏值由 825.03 降至 760.05 秒/源。补充的用户汇总记录 (results/user_supplied_field_practice_summary.json) 报告了新的非配对官方模拟器演练：field 在 Q3 为 11 局、134/134 个源，逐局等权平均 263.550883 秒/源；原 refined 为 10 局、139/139 个源，237.998263 秒/源。Q4 中 field 为 12 局、166/166 个源，484.966300 秒/源；原 refined 为 10 局、138/138 个源，531.177617 秒/源。案例不同且原始新日志未纳入本匿名包，因此这只是策略选择的诊断依据，不是因果比较。综合该非配对官方信号与本地配对证据，最终可执行默认取 Q3refined、Q4field；两题均保留显式--strategy 覆盖选项。

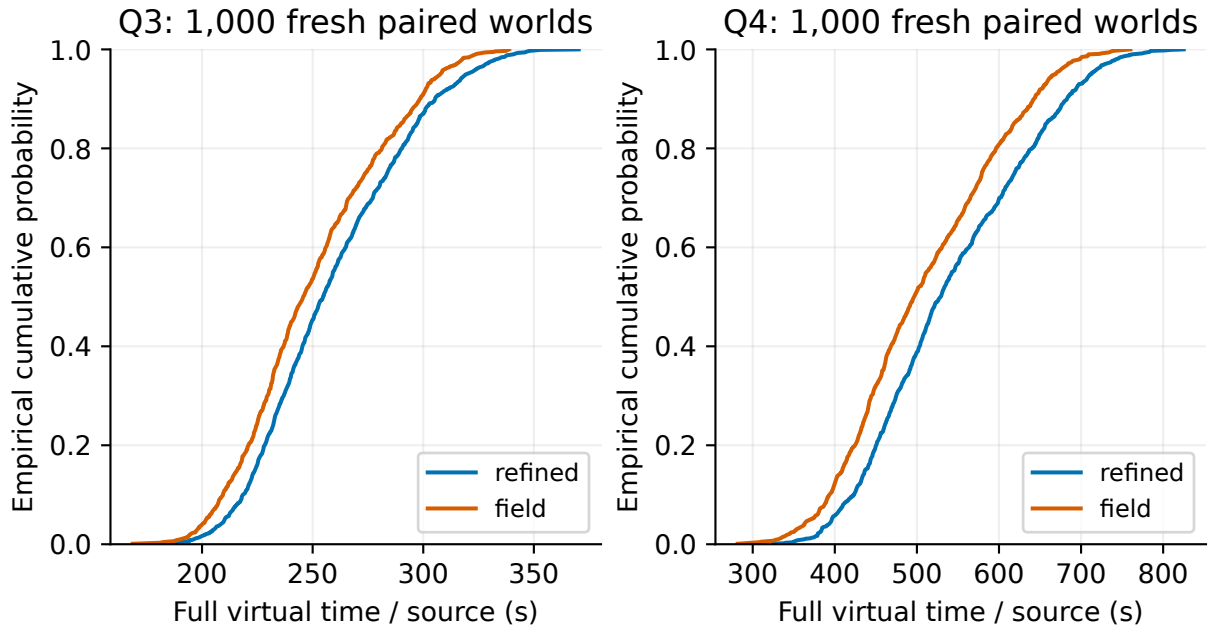


图 9: 已冻结 field 与原 refined 的同世界耗时分布；两题均保留原覆盖骨架与完整停止证书。

11.3 消融和完整成本分解

表 5: 相邻配置的配对节省，单位秒/源；信息项合并原地补测与 Q3 负约束

问题	相邻配置	节省均值	近似 95% 区间
Q3	原 refined 至仅路线改进	4.92	[4.13, 5.70]
Q3	仅路线改进至路线与信息	2.94	[2.59, 3.28]
Q3	路线与信息至新 field	0.96	[0.87, 1.05]
Q4	原 refined 至新 field	33.81	[31.61, 36.01]

问题三的路线改进节省 4.92 秒/源，加入原地信息与负约束再节省 2.94 秒/源，安全接近再节省 0.96 秒/源。三者沿同一配置链比较，增量有条件性，不声称各机制脱离其他条件仍有相同贡献。问题四的主要收益来自路线组合；安全清除点移动在开发中没有体现稳定增益，因此对 Q4 关闭，不为堆叠机制而采用。

表 6: 按每世界每源的完整计费分解；移动含逐步微秒舍入，展示保留两位小数

问题	策略	移动	检测	换频	失败清除	成功清除	合计
Q3	原 refined	195.50	48.98	9.32	0.19	5.00	258.99
Q3	新 field	188.36	47.58	9.06	0.17	5.00	250.17
Q4	原 refined	383.47	129.62	23.91	0.41	5.00	542.41
Q4	新 field	350.93	128.53	23.66	0.48	5.00	508.60

Q3 平均行程由 12607.63 降至 12143.48 米，Q4 由 24581.18 降至 22499.71 米。Q4 失败清除尝试均值却由 1.782 增至 2.089 次/局，说明不能把失败次数本身当作任务目

标；它的增加被移动节省抵消。这里的失败统计是尝试次数/局，不是失败案例比例，所有案例最终均全清。

11.4 压力、数值与现实运行边界

22 个本地压力/替代误差组全部全清，完整逐组均值、区间和更快案例数保存在 `results/field_confirmation.json` 及 `report/generated/field_stress.tex`。Q4 最小接收半径三组平均改善约 5.90% 至 7.06%，边界向外三组约 5.49% 至 5.86%；聚集团部分组的节省区间为 $[-9.37, 44.84]$ 秒/源，包含零，不能宣称在该组证实改善。Q3 边界源收益较小，聚集团约 0.53% 至 0.63%，不主张逐场景等比例下降，更不主张每局占优。

独立检查包含 3000 次负约束真实源包含性、1000 米边界与退化域、双精度溢出拒绝、开放路线任务守恒、两任务可解析最优例、200 组清除圆交接近及现实截止。保守内接多边形和顶点复验提供几何依据，随机检查只补充实现证据。两题新策略在主检验中本地单局最大运算耗时分别约 0.018 秒、0.072 秒；这些数值不包含官方 HTTP 延迟，不能替代 20 分钟现实期限的实测。现有 20 轮官方日志验证的是原 `refined`，新 `field` 必须新的官方演练局确认，正式测试仍需各题三次原始结果及加密日志。

11.5 默认配置的有限动作与虚拟时间上界

本节覆盖最终 `field`、官方演练所用原 `refined` 及历史 `enhanced` 策略：问题三均用原点及半径 1200 的六点环，问题四前两者用 25 点双环、后者用 990 米 31 点三角格。联合调度只重排任务，已知频道筛选不增加补测义务；均保留每局部任务至多 6 次固定测向和原矩形光学后备。`field` 每任务另允许至多一次原地预检测，问题三负观测的保守凸外包不扩大原域、保证清除的安全接近不增加当次接入距离。假定接口正常、误差模型成立且无现实时间中断。以下沿用较大的 31 点范数与次数界，仅为新预检测增加检测预算；不涵盖条带光学或自适应候选。此界不声称紧或最优，距离单位均为米。

可行域尺寸与 216 单元证书。 记真源为 g ，最近正示向点为 s ， $\alpha = 1.005^\circ$ 。示向轴坐标满足 $0 \leq x \leq 1500$ 、 $|y| \leq x \tan \alpha$ 。计入显式外扩，令 $\varepsilon = 10^{-7}$ ，取

$$X = 1500 + 2\varepsilon, \quad W = 2((1500 + \varepsilon) \tan \alpha + \varepsilon / \cos \alpha).$$

于是纵向长度至多 X ，横向宽度至多 $W < 53$ ， $\sqrt{X^2 + W^2} < 1501$ 。最小包围圆圆心 $z \in P$ ，故 $\text{diam}(P) < 1501$ 且 $\|z - s\| < 1501$ 。正测点范数至多 3300、初始外切多边形范数小于 1801，使相关 `_padding` 为 $10^{-9} < \varepsilon$ ；此为几何与显式外扩的解析界，不是全部浮点运算的形式化认证。

单点域直接用一圆覆盖。否则设直径端点差为 (a, b) 、 $d = \sqrt{a^2 + b^2} > 0$ 。任意点差满足 $|\Delta x| \leq d$ 、 $|\Delta y| \leq W$ 、 $|b| \leq W$ ，故直径法向投影差 $|-b\Delta x + a\Delta y|/d \leq 2W$ 。计入投影外扩，直径对齐矩形仍长小于 1501、宽小于 106，对角线小于 1505。令 $h_- = \sqrt{2}(19.9 - 8\varepsilon)$ ，不大于代码单元边长上限，则

$$J \leq \left\lceil \frac{1501}{h_-} \right\rceil \left\lceil \frac{106}{h_-} \right\rceil = 54 \times 4 = 216 \quad (42)$$

个半径 19.9 的圆盘覆盖整个矩形。蛇形相邻中心距至多 $\sqrt{2} 19.9$ ，循环移位至多增加一条矩形内跳跃，故

$$L_{\text{snake}} \leq (216 - 1)\sqrt{2} 19.9 + \sqrt{1501^2 + 106^2} < 7570. \quad (43)$$

每个覆盖中心 q_{opt} 与 g 均在同一矩形中，故 $\|q_{\text{opt}} - g\| < 1505$ 。

恢复点必须以最近正测点为中心估计。 固定候选 $q_0 = z \pm \delta n$ 满足 $n \perp (z - s)$ 、 $\delta \leq 120$ ，因此 $\|q_0 - s\| \leq A := \sqrt{1501^2 + 120^2} < 1506$ 。在获得新正示向前，所有恢复仍使用同一个 s 。设 R_k 为相应轴向反射，则

$$q_{k+1} - s = \frac{1}{2}R_k(q_k - s), \quad \|q_{k+1} - s\| = \frac{1}{2}\|q_k - s\|.$$

反射只保持到 s 的距离，不能把失败点的源距与 1500 取平均。故连续恢复有 $\|q_k - s\| \leq 2^{-k}A$ ；新正示向更新 s 后重用首点界。由 $\|s - g\| \leq 1500$ ，全部局部测向点满足

$$\|q - g\| \leq B := 1500 + A < 3006, \quad \|q\| \leq 1800 + B < 4806. \quad (44)$$

圆心源距小于 1501，覆盖中心源距小于 1505，两者范数亦小于 4806。测向间连线至多 $2B < 6020$ ；额外圆心清除接入至多 $B + 1501 < 4520$ ；光学接入至多 $B + 1505 < 4520$ ，若紧接圆心试清则至多 $1501 + 1505 < 4520$ 。

任务进出、动作计数与总界。 每任务至多 6 次固定测向，`field` 在进入该过程前另可执行一次原地预检测；各提前返回分支均成功清除或转入有限光学覆盖。真源在覆盖域内，故每个正常完成任务必清除一新源，任务数至多 16。成功终点距该源至多 20、范数至多 1820；原地预检测、共享补测及其 `near` 清除不移动。这处理了从另一源清除点进入下一任务的情形。

骨架点范数至多 $990\sqrt{7} < 2620$ 。集成策略整次骨架扫描不插入局部移动，故至多 31 次骨架抵达，出发于原点、骨架或成功清除点，每段小于 5240。任务首个需移动的动作由范数至多 2620 的任务外位置进入范数小于 4806 的局部点，入口小于 7500；原地预检测不改变入口位置。此后至多 5 条测向连线、一次额外圆心清除移动（保证清除或小区域试清）、一次光学接入及一条蛇形，故

$$L \leq 31 \times 5240 + 16(7500 + 5 \times 6020 + 4520 + 4520 + 7570). \quad (45)$$

若首动作即圆心清除或光学首点，入口已支付该段，多余预算仅为放宽。任务至下一骨架的退出段已计入 31 次骨架抵达；无需返回原锚点。

骨架至多 31×20 次测向；共享只在任务结束后调用，至多 16 次、每次至多 16 源；固定局部测向至多 16×6 次。`field` 原地预检测在六次之外，每任务至多一次，另加 16 次且不新增行程。放宽共享预算得到

$$M \leq 31 \times 20 + 16 \times 16 + 16 \times 6 + 16 \leq 31 \times 20 + 31 \times 16 + 16 \times 6 + 16 = 1228.$$

每源无信号后试清至多 6 次、小区域试清至多一次、覆盖至多 216 次，另留一次给非覆盖成功清除（骨架、共享、局部 `near` 或保证清除）；成功后不再处理，故 $C \leq$

$16(216 + 6 + 1 + 1) = 3584$ 。移动速度为 5 米/秒，测向含换频至多 6 秒，清除至多 5 秒，因而

$$T \leq \frac{31 \times 5240 + 16(7500 + 5 \times 6020 + 4520 + 4520 + 7570)}{5} + 1228 \times 6 + 3584 \times 5 < 100 \times 3600 \text{ s.} \quad (46)$$

这是虚拟时间保守界，不是预期成绩或最优界；refined/enhanced 没有新增预检测，仍可使用原 $M \leq 1212$ 。field 的任务重排及域缩小不改变其余范数、行程和光学次数预算，上界也不表示每个实例更快。现实 20 分钟限制须由客户端期限机制另行遵守。

12 模型检验

12.1 几何与算法检验

为检验模型的可行性，分别对交会区域、七点环和双环骨架进行数值核对。测试包含边界点、角度跨越 0° 、重合检测点和近共线观测。程序重新计算每一步的移动、检测、换频和清除时间，并将结果与式(1)核对。对于已经发现的频道，只有收到清除成功反馈才改变频道状态；对于尚未发现的频道，必须完成全部骨架位置检测后才可判断无源。

表 7: 模型与算法检验结果

检验项目	检验规模	结果
最小包围圆交叉计算	504 组点集	与支持点枚举结果一致
七点环与真实源包含	2000 组、每组 4 次测向	全部满足可行域包含条件
双环方向覆盖	36 个闭三角形	边界、顶点和方向扰动均满足距离及投影条件
频道状态与终止动作计时	两种题型完整流程 主测试逐世界记录	无任务遗漏，终止条件一致 $T = L/5 + 5M + S + 3F + 5K$ 成立

12.2 策略结果与成本分析

图8给出了主测试组的移动成本和操作成本。移动仍是总时间的主要组成部分，因此联合任务排序优先减少跨任务绕行。Q3 使用 refined 作为默认策略；Q4 使用 field 的路线组合。两题策略均保留完整覆盖骨架和逐频道终止条件，清除率不以牺牲全清为代价。

12.3 结果讨论

在 1000 个主测试世界中，field 相对 refined 的 Q3 平均单源时间节省 8.8158 秒，Q4 节省 33.8128 秒；两题均完成 1000/1000 个世界的全清。Q3 的官方演练结果显示 field 在另一组案例中耗时较高，因此最终程序采用 Q3 refined、Q4 field 的分题配置。该选择同时考虑了实测时间、覆盖结构和算法复杂度。

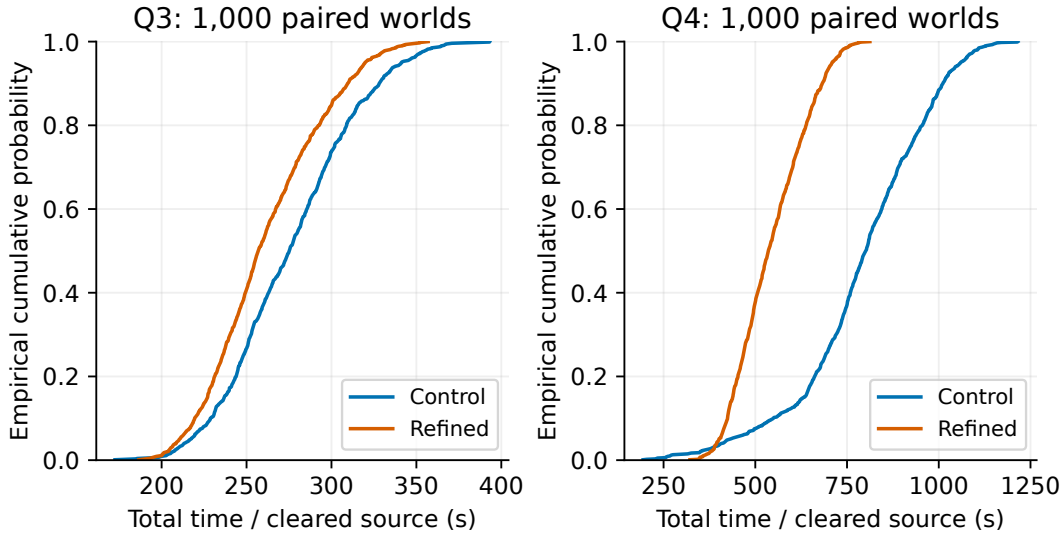


图 10: 两类问题的逐世界单源耗时分布。

13 模型评价与结论

本文完成了四个问题的建模与求解。问题一由正向半平面交求区域直径，最小包围圆而非直径圆是一次保证清除的判据；问题二给出有全向接收保证的三圆交候选域，并据交会条件选择第二检测点。问题三以七点收缩环完成全向源搜索，问题四以 25 点双环完成定向源搜索，逐频道证书负责判断清除和停止。

13.1 模型优缺点与适用范围

模型将覆盖保证与任务调度分开处理，既能证明搜索不漏检，又能用统一时间模型比较路线效率。Q3 默认使用原 `refined` 策略，Q4 默认使用 `field` 策略；`field` 仍可通过命令行显式调用。局限在于任务路线没有全局最优证明，实验误差场属于合成设置，官方隐藏参数尚未公开。模型可推广到不同区域半径、接收下界和检测骨架，但每次推广都需重新验证覆盖条件与清除证书。

13.2 最终策略与结果

用户提供的官方模拟器演练中，原 `refined` 在 Q3 完成 139/139 个源、Q4 完成 138/138 个源。独立本地配对试验中，`field` 相对于 `refined` 的 Q3 平均单源时间由 258.9861 降至 250.1703 秒，Q4 由 542.4134 降至 508.6006 秒；两题主组均 1000/1000 全清。结合 Q3 官方非配对演练中 `field` 的较慢表现，最终采用 Q3`refined`、Q4`field`。

AI 工具使用声明

本研究过程中使用了 AI 工具，参与核心模型推导、代码实现、实验设计、结果整理、独立自动审查及论文写作，详细过程见支撑材料中的《AI 工具使用详情》。本稿尚

无真实参赛队完成逐项人工核验的记录，不将自动审查写作人工审查；该声明用于如实披露研究过程，不能替代正式提交前应完成的参赛队责任。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 B 题：无线电干扰源的快速自动定位与清除，以及附件 1、附件 2[Z]. 2026. https://www.mcm.edu.cn/html_cn/node/27b6e148f8113f09b0269f64a02629fb.html.
- [2] Welzl E. Smallest enclosing disks (balls and ellipsoids)[C]//New Results and New Trends in Computer Science. Lecture Notes in Computer Science, 555. Springer, 1991: 359–370. DOI: 10.1007/BFb0038202.
- [3] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）[EB/OL]. https://www.mcm.edu.cn/html_cn/node/4cd596519c9eb9fbd866398f6df0caa3.html.
- [4] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛人工智能工具使用规定（2026 年试行）[EB/OL]. https://www.mcm.edu.cn/html_cn/node/fef94648f2836ab6cc81586f4c38512b.html.

A 匿名复现附录

本附录只列出匿名包中实际存在、可供复现使用的源文件。代码路径均相对于本文件所在的 `report/` 目录；结果路径均相对于匿名包项目根目录；不含个人绝对路径、队伍身份或私有凭据。主文件应在导言区加载 `listings` 并设置合适的 Python 样式。

A.1 核心运行与验证代码

Listing 1: 规划器与统一运行入口

```
1 """Coverage-certified search; consumes observations only, never simulator truth."""
2
3 from dataclasses import dataclass, field
4 from math import dist, hypot
5 from time import monotonic
6
7 from active_policy_candidate import choose_probe
8 from client import AmbiguousActionError, DeadlineExceeded
9 from coverage import directional_waypoints, omni_waypoints
10 from geometry import clip_bearing, enclosing_circle, initial_polygon, optical_cover
11 from route_policy_candidate import optimize_waypoints
12 from joint_dispatch_candidate import choose_task
13 from optical_policy_candidate import certified_optical_cover
14 from polar_coverage_candidate import polar_waypoints
15
16
17 class SessionTerminated(RuntimeError):
18     """The simulator closed after an accepted action reached its limit."""
19
20
21 class InconsistentObservations(RuntimeError):
22     """The specified bounded-error model no longer contains a possible source."""
23
24
25 @dataclass
26 class Track:
27     polygon: list = field(default_factory=initial_polygon)
28     readings: list = field(default_factory=list)
29     observations: list = field(default_factory=list)
30     failures: list = field(default_factory=list)
31
32
33 class Planner:
34     """Baseline clears detections immediately; integrated batches and shares stops.
35
36     Sampling and route heuristics affect time only. Completeness comes from the
37     waypoint certificate and the conservative, finite optical-cover fallback.
38     """
39
40     def __init__(
41         self,
42         environment,
43         problem=3,
44         strategy="refined",
45         spacing=990.0,
46         probe_scale=0.22,
47         max_probes=6,
48         loss_recovery=True,
49         anchor_policy=None,
50         probe_policy="fixed",
51         ring_radius=None,
52         coverage_policy=None,
53         optical_policy="rectangle",
```

```

54     dispatch_policy=None,
55     scan_policy=None,
56 ):
57     if problem not in (3, 4) or strategy not in (
58         "baseline",
59         "batched",
60         "integrated",
61         "enhanced",
62         "refined",
63     ):
64         raise ValueError("Unsupported problem or strategy")
65     if anchor_policy is None:
66         anchor_policy = "rolling" if strategy in ("enhanced", "refined") and problem == 4 else "nearest"
67     if ring_radius is None and strategy in ("enhanced", "refined") and problem == 3:
68         ring_radius = 1200.0
69     if coverage_policy is None:
70         coverage_policy = "compact25" if strategy == "refined" and problem == 4 else "lattice"
71     if dispatch_policy is None:
72         dispatch_policy = ("center" if problem == 3 else "guarded") if strategy == "refined" else "separate"
73
74     if scan_policy is None:
75         scan_policy = ("unknown" if problem == 3 else "useful") if strategy == "refined" else "all"
76     if anchor_policy not in ("nearest", "two_opt", "rolling"):
77         raise ValueError("Unsupported anchor policy")
78     if probe_policy not in ("fixed", "adaptive"):
79         raise ValueError("Unsupported probe policy")
80     if not 900 <= spacing <= 999:
81         raise ValueError("Directional lattice spacing must be in [900, 999] m")
82     if not 0 < probe_scale <= 1 or max_probes < 1:
83         raise ValueError("Invalid localization policy parameters")
84     if coverage_policy not in ("lattice", "seed25", "compact25", "fast25"):
85         raise ValueError("Unsupported discovery coverage policy")
86     if optical_policy not in ("rectangle", "slabs"):
87         raise ValueError("Unsupported optical covering policy")
88     if dispatch_policy not in ("separate", "center", "guarded"):
89         raise ValueError("Unsupported task dispatch policy")
90     if scan_policy not in ("all", "unknown", "useful"):
91         raise ValueError("Unsupported known-channel scan policy")
92     self.scan_policy = scan_policy
93     self.coverage_policy = coverage_policy if problem == 4 else "ring"
94     self.optical_policy = optical_policy
95     self.dispatch_policy = dispatch_policy
96     self.environment = environment
97     self.problem = problem
98     self.strategy = strategy
99     self.waypoints = (
100         omni_waypoints(ring_radius) if problem == 3 else
101         directional_waypoints(spacing) if coverage_policy == "lattice" else
102         polar_waypoints(coverage_policy)
103     )
104     self.anchor_policy = anchor_policy
105     self.anchor_order = optimize_waypoints(self.waypoints) if anchor_policy == "two_opt" else []
106     self.anchor_rank = {index: rank for rank, index in enumerate(self.anchor_order)}
107     self.pending = set(range(len(self.waypoints)))
108     self.visited = []
109     self.tracks = {}
110     self.cleared = set()
111     self.scan_records = {ch: set() for ch in range(1, 21)}
112     self.position = (0.0, 0.0)
113     self.radio_channel = 1
114     self.virtual_time = 0.0
115     self.deadline = float("inf")
116     self.max_virtual_duration = float("inf")
117     self.terminal_reason = None
118     self.observation_points = {ch: [] for ch in range(1, 21)}
119     self.channel_state = {ch: "unresolved" for ch in range(1, 21)}
120     self.probe_scale = probe_scale

```

```

120     self.max_probes = max_probes
121     self.loss_recovery = bool(loss_recovery) and problem == 4
122     self.fallbacks = 0
123     self.probe_policy = probe_policy
124     self.adaptive_probes = 0
125     self.termination = None
126
127     def action(self, path, point=None, channel=None):
128         if self.terminal_reason is not None:
129             raise SessionTerminated(self.terminal_reason)
130         if monotonic() >= self.deadline - 2.0 and path != "/exit":
131             raise TimeoutError("Insufficient real time: completeness not certified")
132         response = self.environment.act(path, position=point, channel=channel)
133         if response.get("accepted") is not True:
134             raise RuntimeError("Rejected action; cached state was not advanced")
135         self.virtual_time = float(response["virtual_time_s"])
136         if path == "/enter":
137             self.max_virtual_duration = float(response.get("max_virtual_duration_s", float("inf")))
138         if point is not None:
139             self.position = tuple(point)
140         if path == "/measure":
141             self.radio_channel = channel
142         if path in ("/measure", "/clear") and self.virtual_time >= self.max_virtual_duration:
143             self.terminal_reason = "virtual_timeout"
144         return response
145
146     def clear(self, channel, point, guaranteed=False):
147         result = self.action("/clear", point, channel)["clear_result"]
148         if result == "success":
149             self.cleared.add(channel)
150             self.channel_state[channel] = "cleared"
151             self.tracks.pop(channel, None)
152             return True
153         if result != "no_target_in_range":
154             raise RuntimeError(f"Unknown clear result: {result}")
155         if guaranteed:
156             raise InconsistentObservations("Certified radius-20 clearing failed")
157         if channel in self.tracks:
158             self.tracks[channel].failures.append(tuple(point))
159         return False
160
161     def measure(self, channel, point):
162         result = self.action("/measure", point, channel)
163         status = result["measure_result"]
164         self.observation_points[channel].append(tuple(point))
165         track = self.tracks.get(channel)
166         if track is None and status == "direction":
167             track = Track(observations=self.observation_points[channel])
168             self.tracks[channel] = track
169         if track is not None:
170             track.observations = self.observation_points[channel]
171         if status == "near":
172             self.channel_state[channel] = "detected"
173             if self.terminal_reason is None:
174                 self.clear(channel, point, guaranteed=True)
175         elif status == "direction":
176             self.channel_state[channel] = "detected"
177             track.polygon = clip_bearing(track.polygon, point, result["svd_deg"])
178             if not track.polygon:
179                 raise InconsistentObservations(
180                     f"Empty feasible region on channel {channel}"
181                 )
182             track.readings.append((tuple(point), float(result["svd_deg"])))
183         elif status != "no_signal":
184             raise RuntimeError(f"Unknown measure result: {status}")
185         # No-signal observations are not convex halfplanes. In Q4 in particular,
186         # deleting a 1000-m disk would silently exclude real sources.

```

```

187         return status
188
189     def scan_anchor(self, index):
190         point = self.waypoints[index]
191         channels = [ch for ch in range(1, 21) if ch not in self.cleared]
192         channels.sort(key=lambda ch: (ch != self.radio_channel, ch))
193         for channel in channels:
194             if channel in self.cleared:
195                 continue
196             if channel in self.tracks:
197                 if self.scan_policy == "unknown":
198                     continue
199                 if self.scan_policy == "useful":
200                     center, radius = enclosing_circle(self.tracks[channel].polygon)
201                     if radius <= 19.9 or dist(point, center) - radius > 1500:
202                         continue
203             self.measure(channel, point)
204             self.scan_records[channel].add(index)
205             if len(self.cleared) == 16:
206                 self.termination = "known_count_upper_bound_16"
207                 break
208             if self.terminal_reason is not None:
209                 break
210             if self.strategy == "baseline" and channel in self.tracks:
211                 self.localize(channel)
212             if len(self.cleared) == 16:
213                 self.termination = "known_count_upper_bound_16"
214                 break
215             if self.terminal_reason is not None:
216                 break
217             self.pending.discard(index)
218             if index not in self.visited:
219                 self.visited.append(index)
220
221     def optical_finish(self, channel):
222         self.fallbacks += 1
223         covering = certified_optical_cover if self.optical_policy == "slabs" else optical_cover
224         cover = covering(self.tracks[channel].polygon, radius=19.9)
225         if not cover:
226             raise InconsistentObservations("An empty region cannot certify clearing")
227         start = min(range(len(cover)), key=lambda k: dist(self.position, cover[k]))
228         # Rotate the certified snake to the nearest cell. At most one long
229         # wraparound jump; the remaining edges are adjacent rectangle cells.
230         order = cover[start:] + cover[:start]
231         for point in order:
232             if self.clear(channel, point):
233                 return
234             if self.terminal_reason is not None:
235                 return
236         raise InconsistentObservations("Exhausted a certified optical cover without clearing")
237
238     def localize(self, channel):
239         losses = 0
240         failed_probe = None
241         for probe_index in range(self.max_probes):
242             if channel in self.cleared or self.terminal_reason is not None:
243                 return
244             track = self.tracks[channel]
245             center, radius = enclosing_circle(track.polygon)
246             if max(dist(self.position, p) for p in track.polygon) <= 19.9:
247                 self.clear(channel, self.position, guaranteed=True)
248                 return
249             if radius <= 19.9:
250                 self.clear(channel, center, guaranteed=True)
251                 return
252             # When the remaining uncertainty is small, a 3-s failed optical
253             # attempt can cost less than travelling to take another bearing.

```

```

254     if radius <= 38 and center not in track.failures:
255         if self.clear(channel, center):
256             return
257         if self.terminal_reason is not None:
258             return
259         self.optical_finish(channel)
260         return
261     previous = track.readings[-1][0]
262     dx, dy = center[0] - previous[0], center[1] - previous[1]
263     length = hypot(dx, dy)
264     if length < 1e-8:
265         self.optical_finish(channel)
266         return
267     unit, normal = (dx / length, dy / length), (-dy / length, dx / length)
268     if failed_probe is not None and self.loss_recovery:
269         vector = (failed_probe[0] - previous[0], failed_probe[1] - previous[1])
270         axial = vector[0] * unit[0] + vector[1] * unit[1]
271         lateral = vector[0] * normal[0] + vector[1] * normal[1]
272         point = (
273             previous[0] + 0.5 * (axial * unit[0] - lateral * normal[0]),
274             previous[1] + 0.5 * (axial * unit[1] - lateral * normal[1]),
275         )
276         if any(point == p for p in track.observations) or point in track.failures:
277             self.optical_finish(channel)
278             return
279     else:
280         point = None
281         if self.probe_policy == "adaptive":
282             point = choose_probe(
283                 track.polygon,
284                 track.readings,
285                 self.position,
286                 self.problem,
287                 probes_remaining=self.max_probes - probe_index,
288                 rejected_points=track.failures + track.observations,
289                 probe_scale=self.probe_scale,
290             )
291         if point is not None:
292             self.adaptive_probes += 1
293         if point is None:
294             offset = min(120.0, max(25.0, self.probe_scale * radius))
295             candidates = [
296                 (
297                     center[0] + sign * offset * normal[0],
298                     center[1] + sign * offset * normal[1],
299                 )
300                 for sign in (-1, 1)
301             ]
302             fresh = [
303                 p
304                 for p in candidates
305                 if p not in track.observations
306                 and all(p != s for s in track.failures)
307             ]
308             if not fresh:
309                 self.optical_finish(channel)
310                 return
311             point = min(fresh, key=lambda p: dist(self.position, p))
312     status = self.measure(channel, point)
313     if channel in self.cleared or self.terminal_reason is not None:
314         return
315     if status == "no_signal":
316         losses += 1
317         if self.clear(channel, point):
318             return
319         if self.terminal_reason is not None:
320             return

```

```

321         failed_probe = point
322         if losses >= 2 and not self.loss_recovery:
323             self.optical_finish(channel)
324             return
325         else:
326             failed_probe = None
327         if channel not in self.cleared and self.terminal_reason is None:
328             self.optical_finish(channel)
329
330     def share_current_stop(self):
331         """Fuse an extra bearing for known sources at an already reached stop."""
332         if self.strategy not in ("integrated", "enhanced", "refined") or self.terminal_reason is not None or
333         len(self.cleared) == 16:
334             return
335         for channel in list(self.tracks):
336             if channel in self.cleared:
337                 continue
338             track = self.tracks[channel]
339             center, radius = enclosing_circle(track.polygon)
340             if radius <= 19.9:
341                 continue
342             if any(self.position == p for p in track.observations):
343                 continue
344             if dist(self.position, center) - radius > 1500:
345                 continue
346             self.measure(channel, self.position)
347             if self.terminal_reason is not None:
348                 return
349
350     def next_anchor(self):
351         if self.anchor_policy == "two_opt":
352             return min(self.pending, key=lambda i: self.anchor_rank[i])
353         if self.anchor_policy == "rolling":
354             indices = sorted(self.pending)
355             order = optimize_waypoints([self.waypoints[i] for i in indices], self.position)
356             return indices[order[0]]
357         return min(self.pending, key=lambda i: (dist(self.position, self.waypoints[i]), i))
358     def _result(self):
359         complete = len(self.cleared) == 16
360         if not complete:
361             for channel in range(1, 21):
362                 if channel in self.cleared:
363                     self.channel_state[channel] = "cleared"
364                 elif (self.channel_state[channel] != "detected"
365                     and len(self.scan_records[channel]) == len(self.waypoints)):
366                     self.channel_state[channel] = "absent-certified"
367                 else:
368                     complete = False
369                     break
370             else:
371                 complete = True
372         if self.terminal_reason is not None:
373             self.termination = ("complete_after_" if complete else "incomplete_") + self.terminal_reason
374         return {
375             "problem": self.problem,
376             "strategy": self.strategy,
377             "cleared_channels": sorted(self.cleared),
378             "n_cleared": len(self.cleared),
379             "virtual_time_s": self.virtual_time,
380             "average_time_s": self.virtual_time / len(self.cleared)
381             if self.cleared else None,
382             "probe_policy": self.probe_policy,
383             "probe_scale": self.probe_scale,
384             "anchor_policy": self.anchor_policy,
385             "ring_radius_m": hypot(*self.waypoints[1]) if self.problem == 3 else None,
386             "loss_recovery": self.loss_recovery,

```

```

387         "adaptive_probes": self.adaptive_probes,
388         "channel_certificates": dict(self.channel_state),
389         "certificate_complete": complete,
390         "terminal_reason": self.terminal_reason,
391         "visited_anchors": len(self.visited),
392         "total_anchors": len(self.waypoints),
393         "optical_fallbacks": self.fallbacks,
394         "termination": self.termination,
395         "coverage_policy": self.coverage_policy,
396         "optical_policy": self.optical_policy,
397         "dispatch_policy": self.dispatch_policy,
398         "scan_policy": self.scan_policy,
399     }
400
401     def run(self):
402         try:
403             return self._run_body()
404         except (TimeoutError, ConnectionError, AmbiguousActionError,
405               DeadlineExceeded, SessionTerminated) as exc:
406             self.terminal_reason = self.terminal_reason or type(exc).__name__
407             self.termination = self.termination or "incomplete_" + type(exc).__name__
408             return self._result()
409
410     def select_task(self):
411         return choose_task(
412             self.position,
413             {i: self.waypoints[i] for i in sorted(self.pending)},
414             {ch: enclosing_circle(track.polygon) for ch, track in self.tracks.items()},
415             choice=self.dispatch_policy,
416         )
417
418     def _run_body(self):
419         entry_started = monotonic()
420         entered = self.action("/enter")
421         self.deadline = entry_started + entered["remaining_real_duration_s"]
422         self.scan_anchor(0)
423         while (self.pending or self.tracks) and self.terminal_reason is None:
424             if len(self.cleared) == 16:
425                 self.termination = "known_count_upper_bound_16"
426                 break
427             if self.strategy == "baseline":
428                 if self.tracks:
429                     self.localize(next(iter(self.tracks)))
430                 elif self.pending:
431                     self.scan_anchor(self.next_anchor())
432                 continue
433             if self.dispatch_policy != "separate":
434                 kind, index = self.select_task()
435                 if kind == "target":
436                     self.localize(index)
437                     self.share_current_stop()
438                 else:
439                     self.scan_anchor(index)
440                 continue
441             targets = [
442                 (dist(self.position, enclosing_circle(track.polygon)[0]), ch)
443                 for ch, track in self.tracks.items()
444             ]
445             anchors = [
446                 (
447                     dist(self.position, self.waypoints[i])
448                     + 30.0 * (20 - len(self.cleared)),
449                     i,
450                 )
451                 for i in ([self.next_anchor()] if self.pending and self.anchor_policy == "rolling" else self.
pending)
452             ]

```

```

453         # Convert a full scan to equivalent travel metres (6 s * 5 m/s).
454         # This is a myopic time heuristic, not a global-optimality claim.
455         if targets and (not anchors or min(targets)[0] <= min(anchors)[0]):
456             self.localize(min(targets)[1])
457             self.share_current_stop()
458         else:
459             self.scan_anchor(self.next_anchor())
460     if self.termination is None:
461         if self.terminal_reason is None:
462             for channel in range(1, 21):
463                 if channel in self.cleared:
464                     self.channel_state[channel] = "cleared"
465                 elif (self.channel_state[channel] != "detected"
466                     and len(self.scan_records[channel]) == len(self.waypoints)):
467                     self.channel_state[channel] = "absent-certified"
468                 else:
469                     raise RuntimeError("Incomplete per-channel discovery coverage")
470             self.termination = "all_channels_cleared_or_coverage_certified_absent"
471         else:
472             current = self._result()
473             if current["certificate_complete"]:
474                 self.termination = "complete_after_" + self.terminal_reason
475             else:
476                 self.termination = "incomplete_" + self.terminal_reason
477         if self.terminal_reason is None:
478             self.action("/exit")
479     return self._result()

```

Listing 2: 本地环境与模拟器接口

```

1  """Synthetic, not official, CUMCM 2026 B environment (standard library only).
2
3  Generation assumptions are deliberately explicit, not claims about official data:
4  * random: N is discrete uniform on 10..16; N distinct channels sampled uniformly
5    without replacement; positions iid uniform in the radius-1800 disk (sqrt(U)
6    radial law); radio radii iid uniform [1000,1500]; orientations uniform [0,2*pi).
7    Q3 is all omni. Q4 chooses a uniform count in 1..N-1 and a uniform subset
8    of directional sources, guaranteeing both types.
9  * minimum_radius: random geometry, but every radio radius is exactly 1000.
10 * outward_boundary: 16 equally spaced sources on the arena boundary, a seeded
11   common rotation, all radii 1000. Q4 has 15 outward-facing directional sources
12   and one omni, so inward-only search is deliberately inadequate.
13 * clustered: 16 sources uniform in a 35m disk around a seeded center at radius
14   1650; all radio radii 1000, with the random mixed-type rule for Q4.
15
16 Errors are deterministic functions of (seed, channel, exact coordinate): smooth
17 is a bounded two-sinusoid field; extreme is spatial-hashed +/-1 degree;
18 iid_location is spatial-hashed uniform [-1,1], NOT independent repeat readings.
19 The latter is a synthetic hash field, not an assertion of official IID noise.
20 Sources and their parameters are private. Only post-exit summary reveals counts.
21 There are no virtual-duration sleeps; movement time is rounded to microseconds.
22 """
23
24 from __future__ import annotations
25
26 import hashlib
27 import math
28 import random
29 import struct
30 import threading
31 import time
32 from dataclasses import dataclass
33
34 Point = tuple[float, float]
35 SCENARIOS = ("random", "minimum_radius", "outward_boundary", "clustered")
36 ERROR_MODES = ("smooth", "extreme", "iid_location")
37

```

```

38
39 def _action_args(path: str, position: Point | None, channel: int | None):
40     """Validate before any state mutation; match protocol coordinate limits."""
41     if path not in ("/enter", "/exit", "/measure", "/clear"):
42         raise ValueError("Unknown action path (paths must be exact)")
43     if path in ("/enter", "/exit"):
44         if position is not None or channel is not None:
45             raise ValueError("enter/exit do not accept position or channel")
46         return None, None
47     if not isinstance(position, (tuple, list)) or len(position) != 2:
48         raise ValueError("position must contain exactly two finite coordinates")
49     if any(isinstance(v, bool) or not isinstance(v, (int, float)) for v in position):
50         raise ValueError("Coordinates must be JSON numbers")
51     try:
52         point = (float(position[0]), float(position[1]))
53     except OverflowError as exc:
54         raise ValueError("Coordinate outside protocol range") from exc
55     if any(not math.isfinite(v) or abs(v) > 2_000_000 for v in point):
56         raise ValueError("Coordinates must be finite and within +/-2000000m")
57     if (
58         isinstance(channel, bool)
59         or not isinstance(channel, (int, float))
60         or not 1 <= channel <= 20
61         or int(channel) != channel
62     ):
63         raise ValueError("channel must be an integer in 1..20")
64     return point, int(channel)
65
66
67 @dataclass(slots=True)
68 class _Source:
69     position: Point
70     radius: float
71     normal: Point | None
72     phase: float
73     cleared: bool = False
74
75
76 class LocalSimulator:
77     """One local test session; /enter initializes state, duplicate entry rejects.
78
79     log contains only observable request/response/timing data. After termination,
80     construct a new instance for another session, just as the official interface
81     requires a new GUI test. Automatic deadline termination closes this session;
82     no fabricated /exit response is generated in that case.
83     """
84
85     def __init__(
86         self,
87         seed: int,
88         problem: int = 3,
89         error_mode: str = "smooth",
90         scenario: str = "random",
91     ):
92         if isinstance(seed, bool) or not isinstance(seed, int):
93             raise TypeError("seed must be an integer")
94         if problem not in (3, 4):
95             raise ValueError("problem must be 3 or 4")
96         if error_mode not in ERROR_MODES or scenario not in SCENARIOS:
97             raise ValueError(f"Use scenarios {SCENARIOS} and modes {ERROR_MODES}")
98         self._seed, self._problem = seed, problem
99         self._error_mode, self._scenario = error_mode, scenario
100         self._lock = threading.Lock()
101         self._status = "ready"
102         self._sources: dict[int, _Source] = {}
103         self._position: Point = (0.0, 0.0)
104         self._radio_channel = 1

```

```

105     self._virtual_us = 0
106     self._distance = 0.0
107     self._measures = self._switches = self._clear_attempts = self._failed_clears = 0
108     self._started: float | None = None
109     self._finished: float | None = None
110     self._exit_reason: str | None = None
111     self.log: list[dict] = []
112
113     def _generate(self):
114         rng = random.Random(self._seed)
115         n = (
116             16
117             if self._scenario in ("outward_boundary", "clustered")
118             else rng.randint(10, 16)
119         )
120         channels = rng.sample(range(1, 21), n)
121         if self._problem == 3:
122             directional = set()
123         elif self._scenario == "outward_boundary":
124             directional = set(channels[:-1])
125         else:
126             directional = set(rng.sample(channels, rng.randint(1, n - 1)))
127         rotation = rng.uniform(0, math.tau)
128         center = (1650 * math.cos(rotation), 1650 * math.sin(rotation))
129         sources = {}
130         for i, channel in enumerate(channels):
131             theta = rng.uniform(0, math.tau)
132             if self._scenario == "outward_boundary":
133                 theta = rotation + math.tau * i / n
134                 position = (1800 * math.cos(theta), 1800 * math.sin(theta))
135             elif self._scenario == "clustered":
136                 r = 35 * math.sqrt(rng.random())
137                 position = (
138                     center[0] + r * math.cos(theta),
139                     center[1] + r * math.sin(theta),
140                 )
141             else:
142                 r = 1800 * math.sqrt(rng.random())
143                 position = (r * math.cos(theta), r * math.sin(theta))
144                 radius = rng.uniform(1000, 1500) if self._scenario == "random" else 1000.0
145                 orientation = (
146                     theta
147                     if self._scenario == "outward_boundary"
148                     else rng.uniform(0, math.tau)
149                 )
150                 normal = (
151                     math.cos(orientation), math.sin(orientation)
152                 )
153                 if channel in directional
154                     else None
155                 sources[channel] = _Source(
156                     position, radius, normal, rng.uniform(0, math.tau)
157                 )
158         self._sources = sources
159
160     def _bearing_error(self, point: Point, channel: int, source: _Source) -> float:
161         x, y = point
162         if self._error_mode == "smooth":
163             return 0.6 * math.sin(x / 173 + y / 251 + source.phase) + 0.4 * math.sin(
164                 x / 419 - y / 137 + 2 * source.phase
165             )
166         # Canonicalize signed zero: it is the same physical coordinate.
167         packed = struct.pack("!ddB", x or 0.0, y or 0.0, channel)
168         salt = str(self._seed).encode("ascii") + b":"
169         value = int.from_bytes(
170             hashlib.blake2b(salt + packed, digest_size=8).digest(), "big"
171         )

```

```

172         if self._error_mode == "extreme":
173             return 1.0 if value > 1 else -1.0
174         return 2 * (value / ((1 << 64) - 1)) - 1
175
176     def _visible(self, source: _Source, point: Point) -> bool:
177         dx, dy = point[0] - source.position[0], point[1] - source.position[1]
178         if math.hypot(dx, dy) > source.radius:
179             return False
180         if source.normal is None:
181             return True
182         a, b = dx * source.normal[0], dy * source.normal[1]
183         # Include the closed halfplane boundary despite floating-point dot error;
184         # tolerance is only a handful of machine ulps, not an angular margin.
185         return a + b >= -8 * math.ulp(max(abs(a), abs(b), 1.0))
186
187     def _response(self, accepted: bool = True, **fields) -> dict:
188         return {
189             "accepted": accepted,
190             "real_timestamp_ms": int(time.time() * 1000),
191             "virtual_time_s": self._virtual_us / 1_000_000 if accepted else 0,
192             **fields,
193         }
194
195     def _finish(self, reason: str):
196         self._status = "exited"
197         self._finished = time.monotonic()
198         self._exit_reason = reason
199
200     def act(
201         self, path: str, position: Point | None = None, channel: int | None = None
202     ) -> dict:
203         position, channel = _action_args(path, position, channel)
204         with self._lock:
205             if self._status == "exited":
206                 raise ConnectionError(
207                     "Local session interface closed after exit/timeout"
208                 )
209             if self._started is not None and time.monotonic() >= self._started + 1200:
210                 self._finish("real_timeout")
211                 raise ConnectionError("Local session reached its real deadline")
212             request = {"path": path}
213             if position is not None:
214                 request.update(
215                     position={"x": position[0], "y": position[1]}, channel=channel
216                 )
217             before = self._virtual_us
218             radio_before = self._radio_channel
219             distance = 0.0
220             switched = False
221             action_seconds = 0
222             if path == "/enter":
223                 if self._status != "ready":
224                     response = self._response(False)
225                 else:
226                     self._generate()
227                     self._position, self._radio_channel = (0.0, 0.0), 1
228                     self._virtual_us = 0
229                     self._started = time.monotonic()
230                     self._status = "active"
231                     response = self._response(
232                         max_virtual_duration_s=360000,
233                         max_real_duration_s=1200,
234                         remaining_real_duration_s=1200,
235                     )
236             elif self._status != "active":
237                 response = self._response(False)
238             elif path == "/exit":

```

```

239         self._finish("user_exit")
240         response = self._response(exit_reason="user_exit")
241     else:
242         distance = math.dist(self._position, position)
243         source = self._sources.get(channel)
244         live = source is not None and not source.cleared
245         if path == "/measure":
246             switched = channel != self._radio_channel
247             self._radio_channel = channel
248             self._switches += int(switched)
249             self._measures += 1
250             action_seconds = 5
251             fields = {"measure_result": "no_signal"}
252             if live and self._visible(source, position):
253                 if math.dist(position, source.position) <= 5:
254                     fields = {"measure_result": "near"}
255                 else:
256                     angle = math.degrees(
257                         math.atan2(
258                             source.position[1] - position[1],
259                             source.position[0] - position[0],
260                         )
261                     )
262                     bearing = (
263                         round(
264                             (
265                                 angle
266                                 + self._bearing_error(position, channel, source)
267                             )
268                             % 360,
269                             2,
270                         )
271                         % 360
272                     )
273                     fields = {"measure_result": "direction", "svd_deg": bearing}
274             else:
275                 self._clear_attempts += 1
276                 success = live and math.dist(position, source.position) <= 20
277                 if success:
278                     source.cleared = True
279                 else:
280                     self._failed_clears += 1
281                 action_seconds = 5 if success else 3
282                 fields = {
283                     "clear_result": "success" if success else "no_target_in_range"
284                 }
285             self._position = position
286             self._distance += distance
287             self._virtual_us += (
288                 round(distance / 5 * 1_000_000)
289                 + (action_seconds + int(switched)) * 1_000_000
290             )
291             response = self._response(**fields)
292             if self._virtual_us >= 360000 * 1_000_000:
293                 # An action registered before the deadline is allowed to finish.
294                 self._finish("virtual_timeout")
295         self.log.append(
296             {
297                 "request": request,
298                 "response": dict(response),
299                 "distance_m": distance,
300                 "move_time_s": round(distance / 5, 6),
301                 "switch_time_s": int(switched),
302                 "action_time_s": action_seconds,
303                 "radio_channel_before": radio_before,
304                 "radio_channel_after": self._radio_channel,
305                 "action_virtual_time_s": (self._virtual_us - before) / 1_000_000,

```

```

306         }
307     )
308     return response
309
310 def summary(self) -> dict:
311     """Evaluation only: inaccessible during a run; never consumed by planner."""
312     with self._lock:
313         if self._status != "exited":
314             raise RuntimeError("summary is available only after session exit")
315         n = len(self._sources)
316         cleared = sum(source.cleared for source in self._sources.values())
317         virtual_time = self._virtual_us / 1_000_000
318         directional = sum(
319             source.normal is not None for source in self._sources.values()
320         )
321         return {
322             "evidence": "local_synthetic_not_official",
323             "seed": self._seed,
324             "problem": self._problem,
325             "scenario": self._scenario,
326             "error_mode": self._error_mode,
327             "n_sources": n,
328             "n_cleared": cleared,
329             "n_omnidirectional": n - directional,
330             "n_directional": directional,
331             "fraction_cleared": cleared / n,
332             "virtual_time_s": virtual_time,
333             "average_time_s": virtual_time / cleared if cleared else None,
334             "distance_m": self._distance,
335             "measures": self._measures,
336             "switches": self._switches,
337             "clear_attempts": self._clear_attempts,
338             "failed_clears": self._failed_clears,
339             "real_duration_s": self._finished - self._started,
340             "exit_reason": self._exit_reason,
341         }

```

Listing 3: 几何计算与覆盖工具

```

1  """Conservative bearing geometry and whole-region optical clearing certificates.
2
3  Polygons are ordered convex vertex lists (either winding); [] means infeasible,
4  never unbounded. All distances are metres and bearings are counterclockwise degrees.
5  """
6
7  from __future__ import annotations
8
9  from decimal import Decimal, localcontext
10 from math import ceil, cos, hypot, isfinite, nextafter, pi, radians, sin, sqrt, ulp
11 from random import Random
12
13 Point = tuple[float, float]
14 Circle = tuple[Point, float]
15 ARENA_RADIUS = 1800.0
16 MAX_RADIO_RADIUS = 1500.0
17 BEARING_ERROR_DEG = 1.005
18
19
20 def _check_points(points: list[Point]) -> None:
21     if any(not (isfinite(x) and isfinite(y)) for x, y in points):
22         raise ValueError("Geometry requires finite coordinates")
23
24
25 def _padding(scale: float) -> float:
26     return max(1e-9, 64.0 * ulp(max(1.0, scale)))
27
28

```

```

29 def initial_polygon(sides: int = 180) -> list[Point]:
30     """Return a CCW circumscribed polygon, never an inscribed approximation."""
31     if isinstance(sides, bool) or not isinstance(sides, int) or sides < 3:
32         raise ValueError("sides must be an integer at least three")
33     radius = (ARENA_RADIUS + _padding(ARENA_RADIUS)) / cos(pi / sides)
34     return [
35         (radius * cos(2 * pi * i / sides), radius * sin(2 * pi * i / sides))
36         for i in range(sides)
37     ]
38
39
40 def _clip_halfplane(
41     poly: list[Point], origin: Point, normal: Point, bound: float, padding: float
42 ) -> list[Point]:
43     """Clip n.(x-origin) <= bound+padding using the same shifted boundary."""
44     if not poly:
45         return []
46     nx, ny = normal
47     ox, oy = origin
48     limit = bound + padding
49     result: list[Point] = []
50     previous = poly[-1]
51     dp = nx * (previous[0] - ox) + ny * (previous[1] - oy) - limit
52     for current in poly:
53         dc = nx * (current[0] - ox) + ny * (current[1] - oy) - limit
54         if (dp <= 0.0) != (dc <= 0.0):
55             # Opposite signs prevent cancellation in dp-dc. Interpolate from
56             # the nearer endpoint to reduce cancellation near a polygon vertex.
57             if abs(dp) <= abs(dc):
58                 t = dp / (dp - dc)
59                 crossing = (
60                     previous[0] + t * (current[0] - previous[0]),
61                     previous[1] + t * (current[1] - previous[1]),
62                 )
63             else:
64                 t = dc / (dc - dp)
65                 crossing = (
66                     current[0] + t * (previous[0] - current[0]),
67                     current[1] + t * (previous[1] - current[1]),
68                 )
69             if not result or crossing != result[-1]:
70                 result.append(crossing)
71             if dc <= 0.0 and (not result or current != result[-1]):
72                 result.append(current)
73             previous, dp = current, dc
74     if len(result) > 1 and result[0] == result[-1]:
75         result.pop()
76     return result
77
78
79 def clip_bearing(
80     poly: list[Point],
81     point: Point,
82     bearing_deg: float,
83     error_deg: float = BEARING_ERROR_DEG,
84 ) -> list[Point]:
85     """Intersect a bounded region with a forward wedge and a safe range cap.
86
87     The default includes the 0.005-degree two-decimal API rounding allowance.
88     The cap is a relaxation of distance <= 1500, not an assumed known range.
89     Contradictions remain []; no fallback region or constraint reset is applied.
90     """
91     _check_points(poly)
92     _check_points([point])
93     if not isfinite(bearing_deg) or not isfinite(error_deg) or not 0 <= error_deg < 90:
94         raise ValueError("Use finite bearing and 0 <= error_deg < 90")
95     if not poly:

```

```

96         return []
97     theta = radians(bearing_deg % 360.0)
98     error = radians(error_deg)
99     low, high = theta - error, theta + error
100     scale = max(
101         MAX_RADIO_RADIUS,
102         abs(point[0]),
103         abs(point[1]),
104         *(max(abs(x), abs(y)) for x, y in poly),
105     )
106     padding = _padding(scale)
107     # cross(u_low, x-point)>=0 and cross(u_high, x-point)<=0.
108     result = _clip_halfplane(poly, point, (sin(low), -cos(low)), 0.0, padding)
109     result = _clip_halfplane(result, point, (-sin(high), cos(high)), 0.0, padding)
110     result = _clip_halfplane(result, point, (-cos(theta), -sin(theta)), 0.0, padding)
111     return _clip_halfplane(
112         result, point, (cos(theta), sin(theta)), MAX_RADIO_RADIUS, padding
113     )
114
115
116 def _farthest_pair(poly: list[Point]) -> tuple[Point, Point, float]:
117     first = second = poly[0]
118     distance = 0.0
119     for i, a in enumerate(poly):
120         for j in range(i):
121             b = poly[j]
122             candidate = hypot(a[0] - b[0], a[1] - b[1])
123             if candidate > distance:
124                 first, second, distance = a, b, candidate
125     return first, second, distance
126
127
128 def diameter(poly: list[Point]) -> float:
129     """Exact vertex-pair characterization,  $O(n^2)$ ; empty input is an error."""
130     _check_points(poly)
131     if not poly:
132         raise ValueError("An infeasible region has no localization diameter")
133     return _farthest_pair(poly)[2]
134
135
136 def _pair_circle(a: Point, b: Point) -> Circle:
137     center = (a[0] + (b[0] - a[0]) / 2, a[1] + (b[1] - a[1]) / 2)
138     return center, max(hypot(center[0] - p[0], center[1] - p[1]) for p in (a, b))
139
140
141 def _cross(a: Point, b: Point, c: Point) -> float:
142     bx, by = b[0] - a[0], b[1] - a[1]
143     cx, cy = c[0] - a[0], c[1] - a[1]
144     determinant = bx * cy - by * cx
145     scale = max(abs(bx), abs(by), abs(cx), abs(cy))
146     if abs(determinant) > 64 * ulp(max(1.0, scale * scale)):
147         return determinant
148     with localcontext() as context:
149         context.prec = 80
150         ax, ay = Decimal(a[0]), Decimal(a[1])
151         return float(
152             (Decimal(b[0]) - ax) * (Decimal(c[1]) - ay)
153             - (Decimal(b[1]) - ay) * (Decimal(c[0]) - ax)
154         )
155
156
157 def _circumcircle(a: Point, b: Point, c: Point) -> Circle | None:
158     bx, by = b[0] - a[0], b[1] - a[1]
159     cx, cy = c[0] - a[0], c[1] - a[1]
160     determinant = bx * cy - by * cx
161     scale = max(abs(bx), abs(by), abs(cx), abs(cy))
162     if abs(determinant) <= 64 * ulp(max(1.0, scale * scale)):

```

```

163     # Decimal fallback resolves almost-collinear triples from the actual
164     # binary input values, without pretending a small determinant is zero.
165     with localcontext() as context:
166         context.prec = 80
167         axd, ayd = Decimal(a[0]), Decimal(a[1])
168         bxd, byd = Decimal(b[0]) - axd, Decimal(b[1]) - ayd
169         cxd, cyd = Decimal(c[0]) - axd, Decimal(c[1]) - ayd
170         det = 2 * (bxd * cyd - byd * cxd)
171         if not det:
172             return None
173         bb, cc = bxd * bxd + byd * byd, cxd * cxd + cyd * cyd
174         center = (
175             float(axd + (cyd * bb - byd * cc) / det),
176             float(ayd + (bxd * cc - cxd * bb) / det),
177         )
178     else:
179         bb, cc = bx * bx + by * by, cx * cx + cy * cy
180         center = (
181             a[0] + (cy * bb - by * cc) / (2 * determinant),
182             a[1] + (bx * cc - cx * bb) / (2 * determinant),
183         )
184     return center, max(hypot(center[0] - p[0], center[1] - p[1]) for p in (a, b, c))
185
186
187 def _contains(circle: Circle, point: Point) -> bool:
188     center, radius = circle
189     return hypot(point[0] - center[0], point[1] - center[1]) <= nextafter(
190         radius, float("inf")
191     )
192
193
194 def _two_boundary_circle(points: list[Point], end: int, a: Point, b: Point) -> Circle:
195     base = _pair_circle(a, b)
196     left: Circle | None = None
197     right: Circle | None = None
198     left_cross = right_cross = 0.0
199     for i in range(end):
200         p = points[i]
201         if _contains(base, p):
202             continue
203         side = _cross(a, b, p)
204         candidate = _circumcircle(a, b, p)
205         if candidate is None:
206             continue
207         center_side = _cross(a, b, candidate[0])
208         if side > 0 and (left is None or center_side > left_cross):
209             left, left_cross = candidate, center_side
210         elif side < 0 and (right is None or center_side < right_cross):
211             right, right_cross = candidate, center_side
212     if left is None:
213         return base if right is None else right
214     if right is None:
215         return left
216     return left if left[1] <= right[1] else right
217
218
219 def enclosing_circle(poly: list[Point]) -> Circle:
220     """Minimum enclosing circle, with a tiny outward floating-point guard.
221
222     Randomized incremental 1/2/3-support construction, not a diameter-disk or
223     approximate iterative fit. The local fixed shuffle never alters global RNG.
224     Empty input raises instead of manufacturing a spurious clearing location.
225     """
226     _check_points(poly)
227     if not poly:
228         raise ValueError("An infeasible region has no enclosing-circle center")
229     points = list(dict.fromkeys(poly))

```

```

230 Random(0).shuffle(points)
231 circle: Circle | None = None
232 for i, p in enumerate(points):
233     if circle is not None and _contains(circle, p):
234         continue
235     circle = (p, 0.0)
236     for j in range(i):
237         q = points[j]
238         if _contains(circle, q):
239             continue
240         if circle[1] == 0.0:
241             circle = _pair_circle(p, q)
242         else:
243             circle = _two_boundary_circle(points, j + 1, p, q)
244 assert circle is not None
245 center = circle[0]
246 radius = max(hypot(p[0] - center[0], p[1] - center[1]) for p in points)
247 if radius == 0.0:
248     return center, 0.0
249 scale = max(radius, abs(center[0]), abs(center[1]))
250 return center, nextafter(radius + _padding(scale), float("inf"))
251
252
253 def optical_cover(poly: list[Point], radius: float = 19.9) -> list[Point]:
254     """Cover the ENTIRE convex polygon, not merely its vertices or boundary.
255
256     Use its MEC when possible; otherwise tile an enclosing rectangle aligned
257     with a diameter. Every grid cell fits in its own radius-radius disk. Some
258     centers may lie outside the polygon or arena, which the API permits.
259     [] stays []: callers must treat it as infeasibility, not successful clearing.
260     """
261     _check_points(poly)
262     if not isfinite(radius) or radius <= 0:
263         raise ValueError("Optical covering radius must be finite and positive")
264     if not poly:
265         return []
266     center, enclosing_radius = enclosing_circle(poly)
267     if enclosing_radius <= radius:
268         return [center]
269     a, b, distance = _farthest_pair(poly)
270     if distance == 0.0:
271         return [a]
272     ux, uy = (b[0] - a[0]) / distance, (b[1] - a[1]) / distance
273     # Re-normalize before forming the orthogonal frame.
274     norm = hypot(ux, uy)
275     ux, uy = ux / norm, uy / norm
276     projected = [
277         ((x - a[0]) * ux + (y - a[1]) * uy, -(x - a[0]) * uy + (y - a[1]) * ux)
278         for x, y in poly
279     ]
280     scale = max(distance, *(max(abs(x), abs(y)) for x, y in poly))
281     guard = _padding(scale)
282     safe_radius = radius - 8 * guard
283     if safe_radius <= 0:
284         raise ValueError(
285             "Radius is below floating-point resolution at these coordinates"
286         )
287     xmin, xmax = (
288         min(p[0] for p in projected) - guard,
289         max(p[0] for p in projected) + guard,
290     )
291     ymin, ymax = (
292         min(p[1] for p in projected) - guard,
293         max(p[1] for p in projected) + guard,
294     )
295     side = sqrt(2.0) * safe_radius
296     nx = max(1, ceil((xmax - xmin) / side))

```

```

297     ny = max(1, ceil((ymax - ymin) / side))
298     dx, dy = (xmax - xmin) / nx, (ymax - ymin) / ny
299     centers: list[Point] = []
300     for j in range(ny):
301         y = ymin + (j + 0.5) * dy
302         indices = range(nx) if j % 2 == 0 else range(nx - 1, -1, -1)
303         for i in indices:
304             x = xmin + (i + 0.5) * dx
305             centers.append((a[0] + x * ux - y * uy, a[1] + x * uy + y * ux))
306     return centers

```

Listing 4: 官方接口客户端

```

1  """Serial, append-only-logged HTTP adapter for an already-open simulator test.
2
3  This module does NOT authenticate, launch a GUI, or start a formal test. The user
4  must first start the intended test in the official GUI and wait for readiness.
5  Only explicit act calls cause simulator actions; close never sends /exit.
6
7  A new logical action receives a UUID. Transient retries preserve exactly the
8  same bytes, path, and request ID. Exhausted ambiguous actions fail closed: no
9  new action can be submitted on this client because the previous action may have
10 executed. Logs are ordinary plaintext observations, NOT official encrypted logs.
11 Real sleeps are used solely for short transport-retry backoff, never virtual time.
12 """
13
14 from __future__ import annotations
15
16 import http.client
17 import json
18 import math
19 import queue
20 import threading
21 import time
22 import unicodedata
23 import uuid
24 from pathlib import Path
25 from urllib.error import HTTPError, URLError
26 from urllib.parse import urlsplit
27 from urllib.request import HTTPRedirectHandler, ProxyHandler, Request, build_opener
28
29 from environment import Point, _action_args
30
31
32 class ClientError(RuntimeError):
33     """An action did not produce a verified accepted response."""
34
35
36 class RejectedActionError(ClientError):
37     def __init__(self, status: int, response: dict):
38         self.status, self.response = status, response
39         super().__init__(
40             f"Simulator rejected action: HTTP {status}, accepted={response.get('accepted')!r}"
41         )
42
43
44 class AmbiguousActionError(ClientError):
45     """The action may have executed; consult the log and simulator GUI."""
46
47
48 class ClientProtocolError(ClientError):
49     pass
50
51
52 class DeadlineExceeded(ClientError):
53     pass
54

```

```

55
56 class _NoRedirect(HTTPRedirectHandler):
57     def redirect_request(self, req, fp, code, msg, headers, newurl):
58         # Never let urllib turn a POST into a GET or send identifiers elsewhere.
59         return None
60
61
62 def _number(value) -> bool:
63     return (
64         not isinstance(value, bool)
65         and isinstance(value, (int, float))
66         and math.isfinite(value)
67     )
68
69
70 def _unique_object(pairs):
71     result = {}
72     for key, value in pairs:
73         if key in result:
74             raise ValueError(f"Duplicate response key: {key}")
75         result[key] = value
76     return result
77
78
79 class HTTPClient:
80     """Official wire schema, no truth API, one in-flight logical action at a time.
81
82     Cached public position/radio_channel/virtual_time_s describe only confirmed
83     accepted actions. The real deadline uses /enter.remaining_real_duration_s,
84     conservatively subtracting all elapsed time since its first send, including
85     transport retries. Before entry, a bounded 20-second connection budget applies
86     because the actual GUI window deadline is not available through the API.
87     """
88
89     _MAX_ATTEMPTS = 4
90     _REQUEST_TIMEOUT_S = 5.0
91     _ACTION_RETRY_BUDGET_S = 20.0
92     _MAX_RESPONSE_BYTES = 65536
93     _TRANSIENT_HTTP = frozenset((429, 500, 502, 503, 504))
94
95     def __init__(self, base_url: str, robot_id: str, log_path: str):
96         parsed = urlsplit(base_url)
97         if (
98             parsed.scheme not in ("http", "https")
99             or not parsed.hostname
100             or parsed.path not in ("", "/")
101             or parsed.query
102             or parsed.fragment
103             or parsed.username is not None
104             or parsed.password is not None
105         ):
106             raise ValueError(
107                 "base_url must be an HTTP(S) origin, without credentials/path/query"
108             )
109         # Force port validation before sending anything.
110         _ = parsed.port
111         if (
112             not isinstance(robot_id, str)
113             or not 1 <= len(robot_id.encode("utf-8")) <= 64
114         ):
115             raise ValueError("robot_id must contain 1..64 UTF-8 bytes")
116         if any(unicodedata.category(c) in ("Cc", "Cf") for c in robot_id):
117             raise ValueError("robot_id cannot contain control or format characters")
118         self._base_url, self._robot_id = base_url.rstrip("/"), robot_id
119         self._lock = threading.Lock()
120         self._opener = build_opener(ProxyHandler({}), _NoRedirect())
121         self._closed = False

```

```

122     self._uncertain = False
123     self._entered = self._exited = False
124     self._deadline: float | None = None
125     self._max_virtual_duration_s: float | None = None
126     self.position: Point = (0.0, 0.0)
127     self.radio_channel = 1
128     self.virtual_time_s = 0.0
129     self._log = Path(log_path).open("a", encoding="utf-8", buffering=1) # noqa: SIM115 - lifecycle closed
    by close()

130
131 @property
132 def remaining_real_duration_s(self) -> float | None:
133     """Locally conservative seconds left; unknown until accepted /enter."""
134     return (
135         None
136         if self._deadline is None
137         else max(0.0, self._deadline - time.monotonic())
138     )
139
140 def _record(self, event: dict):
141     self._log.write(
142         json.dumps(
143             {"logged_at_ms": int(time.time() * 1000), **event},
144             ensure_ascii=False,
145             allow_nan=False,
146             separators=(",", ":"),
147         )
148         + "\n"
149     )
150     self._log.flush()
151
152 def _decode(self, raw: bytes) -> dict:
153     if len(raw) > self._MAX_RESPONSE_BYTES:
154         raise ClientProtocolError("Response exceeds bounded JSON body size")
155     try:
156         response = json.loads(raw.decode("utf-8"), object_pairs_hook=_unique_object)
157     except (UnicodeDecodeError, ValueError) as exc:
158         raise ClientProtocolError(
159             "Response is not valid unambiguous UTF-8 JSON"
160         ) from exc
161     if not isinstance(response, dict) or type(response.get("accepted")) is not bool:
162         raise ClientProtocolError("Response lacks a boolean accepted field")
163     if (
164         not _number(response.get("real_timestamp_ms"))
165         or not _number(response.get("virtual_time_s"))
166         or response["virtual_time_s"] < 0
167     ):
168         raise ClientProtocolError("Response lacks valid numeric timestamps")
169     return response
170
171 def _validate_accepted(self, path: str, response: dict):
172     virtual_time = response["virtual_time_s"]
173     if path == "/enter":
174         remaining = response.get("remaining_real_duration_s")
175         if (
176             not _number(remaining)
177             or not 0 <= remaining <= 1200
178             or int(remaining) != remaining
179         ):
180             raise ClientProtocolError("Invalid actual remaining real duration")
181         if (
182             not _number(response.get("max_virtual_duration_s"))
183             or response["max_virtual_duration_s"] <= 0
184             or not _number(response.get("max_real_duration_s"))
185             or response["max_real_duration_s"] <= 0
186             or virtual_time != 0
187         ):

```

```

188         raise ClientProtocolError(
189             "Invalid accepted entry limits or initial time"
190         )
191     elif virtual_time < self.virtual_time_s:
192         raise ClientProtocolError("Accepted action moved virtual time backwards")
193     if path == "/measure":
194         result = response.get("measure_result")
195         if result not in ("near", "no_signal", "direction"):
196             raise ClientProtocolError("Unknown measure result")
197         if result == "direction":
198             angle = response.get("svd_deg")
199             if not _number(angle) or not 0 <= angle < 360:
200                 raise ClientProtocolError("Invalid direction angle")
201             elif "svd_deg" in response:
202                 raise ClientProtocolError("Bearing supplied without direction result")
203     elif path == "/clear" and response.get("clear_result") not in (
204         "success",
205         "no_target_in_range",
206     ):
207         raise ClientProtocolError("Unknown clear result")
208     elif path == "/exit" and response.get("exit_reason") != "user_exit":
209         raise ClientProtocolError("Unexpected successful exit reason")
210
211     def _commit(
212         self,
213         path: str,
214         position: Point | None,
215         channel: int | None,
216         response: dict,
217         first_send: float,
218     ):
219         if path == "/enter":
220             self._entered = True
221             self.position, self.radio_channel = (0.0, 0.0), 1
222             self._deadline = first_send + response["remaining_real_duration_s"]
223             self._max_virtual_duration_s = response["max_virtual_duration_s"]
224         elif path == "/exit":
225             self._exited = True
226         else:
227             self.position = position
228             if path == "/measure":
229                 self.radio_channel = channel
230             self.virtual_time_s = response["virtual_time_s"]
231             self._uncertain = False
232
233     def _exchange(self, request: Request, deadline: float) -> tuple[int, bytes]:
234         """Bound connection, headers and body together, not just idle sockets.
235
236         On expiry the daemon may finish the same in-flight request, but cannot
237         mutate client state or send another action. act remains fail-closed.
238         """
239         completed = queue.Queue(maxsize=1)
240
241     def receive():
242         try:
243             available = deadline - time.monotonic()
244             if available <= 0:
245                 raise TimeoutError("Absolute HTTP deadline elapsed")
246             try:
247                 stream = self._opener.open(
248                     request, timeout=min(self._REQUEST_TIMEOUT_S, available)
249                 )
250             except HTTPError as exc:
251                 stream = exc
252             with stream:
253                 status = stream.code
254                 expected_length = stream.headers.get("Content-Length")

```

```

255         if expected_length is not None:
256             try:
257                 expected_length = int(expected_length)
258             except ValueError as exc:
259                 raise ClientProtocolError("Invalid Content-Length") from exc
260             if not 0 <= expected_length <= self._MAX_RESPONSE_BYTES:
261                 raise ClientProtocolError(
262                     "Response exceeds bounded JSON body size"
263                 )
264             raw = stream.read(self._MAX_RESPONSE_BYTES + 1)
265             if expected_length is not None and len(raw) < expected_length:
266                 raise http.client.IncompleteRead(
267                     raw, expected_length - len(raw)
268                 )
269             completed.put((True, (status, raw)))
270         except (
271             ClientError,
272             OSError,
273             ValueError,
274             TypeError,
275             TimeoutError,
276             http.client.HTTPException,
277         ) as exc:
278             completed.put((False, exc))
279
280     threading.Thread(target=receive, daemon=True, name="simulator-response").start()
281     try:
282         succeeded, value = completed.get(
283             timeout=max(0.0, deadline - time.monotonic())
284         )
285     except queue.Empty as exc:
286         raise TimeoutError(
287             "Absolute HTTP deadline elapsed; outcome remains unknown"
288         ) from exc
289     if not succeeded:
290         raise value
291     return value
292
293     def act(
294         self, path: str, position: Point | None = None, channel: int | None = None
295     ) -> dict:
296         position, channel = _action_args(path, position, channel)
297         with self._lock:
298             if self._closed:
299                 raise ClientError("Client is closed")
300             if self._uncertain:
301                 raise AmbiguousActionError(
302                     "Previous outcome is unresolved; no different action may be sent"
303                 )
304             if self._exited:
305                 raise ClientError("Session already exited; do not query /exit again")
306             if self._deadline is not None and time.monotonic() >= self._deadline:
307                 raise DeadlineExceeded(
308                     "Actual remaining real duration has elapsed; interface may be closed"
309                 )
310             if (
311                 self._max_virtual_duration_s is not None
312                 and self.virtual_time_s >= self._max_virtual_duration_s
313             ):
314                 raise DeadlineExceeded(
315                     "Virtual deadline reached; no post-timeout /exit query is legal"
316                 )
317             if path == "/enter" and self._entered:
318                 raise ClientError("Cannot enter the same test twice")
319             if path != "/enter" and not self._entered:
320                 raise ClientError(
321                     "Explicit successful /enter is required before other actions"

```

```

322         )
323     request_id = uuid.uuid4().hex
324     payload = {
325         "arena_id": "default",
326         "robot_id": self._robot_id,
327         "request_id": request_id,
328     }
329     if position is not None:
330         payload.update(
331             position={"x": position[0], "y": position[1]}, channel=channel
332         )
333     body = json.dumps(
334         payload, ensure_ascii=False, allow_nan=False, separators=(",", ":")
335     ).encode("utf-8")
336     first_send = time.monotonic()
337     stop = first_send + self._ACTION_RETRY_BUDGET_S
338     if self._deadline is not None:
339         stop = min(stop, self._deadline)
340     last_error = "No response"
341     for attempt in range(1, self._MAX_ATTEMPTS + 1):
342         available = stop - time.monotonic()
343         if available <= 0:
344             break
345         self._record(
346             {
347                 "event": "request",
348                 "path": path,
349                 "request_id": request_id,
350                 "attempt": attempt,
351                 "body": payload,
352             }
353         )
354         request = Request(
355             self._base_url + path,
356             data=body,
357             headers={"Content-Type": "application/json"},
358             method="POST",
359         )
360         # From this point until a valid outcome, execution is uncertain.
361         self._uncertain = True
362         try:
363             status, raw = self._exchange(request, stop)
364             self._record(
365                 {
366                     "event": "response",
367                     "path": path,
368                     "request_id": request_id,
369                     "attempt": attempt,
370                     "http_status": status,
371                     "body_utf8": raw.decode("utf-8", errors="replace"),
372                 }
373             )
374         except (URLError, OSError, http.client.HTTPException) as exc:
375             # Includes connection refusal, broken connections, timeouts,
376             # truncated HTTP bodies. No state is inferred from any of these.
377             last_error = f"{type(exc).__name__}: {exc}"
378             self._record(
379                 {
380                     "event": "transport_error",
381                     "path": path,
382                     "request_id": request_id,
383                     "attempt": attempt,
384                     "error": last_error,
385                 }
386             )
387         else:
388             if status in self._TRANSIENT_HTTP:

```

```

389         last_error = f"Transient HTTP {status}"
390     else:
391         response = self._decode(raw)
392         if status != 200 or response["accepted"] is not True:
393             if response["accepted"] is True:
394                 raise ClientProtocolError(
395                     "accepted=true with non-200 HTTP status"
396                 )
397             self._uncertain = False
398             raise RejectedActionError(status, response)
399         self._validate_accepted(path, response)
400         radio_before = self.radio_channel
401         distance = (
402             math.dist(self.position, position)
403             if position is not None
404             else 0.0
405         )
406         switch = int(path == "/measure" and channel != radio_before)
407         self._record(
408             {
409                 "event": "accepted",
410                 "path": path,
411                 "request_id": request_id,
412                 "distance_m": distance,
413                 "radio_channel_before": radio_before,
414                 "radio_channel_after": channel
415                 if path == "/measure"
416                 else radio_before,
417                 "switch_time_s": switch,
418                 "action_virtual_time_s": response["virtual_time_s"]
419                 - self.virtual_time_s,
420             }
421         )
422         self._commit(path, position, channel, response, first_send)
423         return response
424     if attempt < self._MAX_ATTEMPTS:
425         delay = min(
426             0.2 * 2 ** (attempt - 1), max(0.0, stop - time.monotonic())
427         )
428         if delay > 0:
429             time.sleep(delay)
430     raise AmbiguousActionError(
431         f"Action {request_id} has no confirmed outcome after bounded retries: {last_error}"
432     )
433
434     def close(self):
435         """Close only this adapter's log. Never start/end a simulator session."""
436         with self._lock:
437             if not self._closed:
438                 self._log.close()
439                 self._closed = True

```

Listing 5: 问题三、四策略实现

```

1  """Field-informed candidates; consumes public responses, never practice labels.
2
3  Historical Planner(strategy='refined') remains unchanged. Candidate options
4  are explicit for paired ablation, and only verified configurations are exposed
5  by the final runner.
6  """
7
8  from __future__ import annotations
9
10 from math import dist, sqrt
11
12 from geometry import enclosing_circle
13 from joint_dispatch_candidate import choose_task

```

```

14 from negative_geometry import exclude_omni_disk
15 from planner import InconsistentObservations, Planner
16 from route_portfolio import choose_portfolio_task
17
18
19 class FieldPlanner(Planner):
20     def __init__(
21         self,
22         environment,
23         problem=3,
24         *,
25         premeasure=True,
26         negative_updates=True,
27         portfolio=True,
28         risk_weight=None,
29         probe_scale=0.22,
30         approach_clear=None,
31     ):
32         super().__init__(
33             environment, problem=problem, strategy="refined", probe_scale=probe_scale
34         )
35         self.premeasure = premeasure
36         self.negative_updates = negative_updates and problem == 3
37         self.portfolio = portfolio
38         self.approach_clear = problem == 3 if approach_clear is None else approach_clear
39         self.risk_weight = (
40             (0.0 if problem == 3 else 1.0) if risk_weight is None else risk_weight
41         )
42         self.negative_stations = {channel: [] for channel in range(1, 21)}
43         self.premeasured_channels = set()
44         self.zero_travel_probes = 0
45         self.negative_cuts = 0
46         self.clear_access_saved_m = 0.0
47
48     def measure(self, channel, point):
49         status = super().measure(channel, point)
50         if not self.negative_updates or channel in self.cleared:
51             return status
52         if status == "no_signal":
53             self.negative_stations[channel].append(tuple(point))
54             track = self.tracks.get(channel)
55             if track is not None and status in ("direction", "no_signal"):
56                 # A convex outer approximation remains safe after each disk
57                 # exclusion, including negatives collected before first detection.
58                 for station in self.negative_stations[channel]:
59                     before = track.polygon
60                     track.polygon = exclude_omni_disk(before, station, sides=16)
61                     self.negative_cuts += track.polygon != before
62                     if not track.polygon:
63                         raise InconsistentObservations(
64                             "Omnidirectional negative readings exclude the entire track"
65                         )
66             return status
67
68     def localize(self, channel):
69         track = self.tracks.get(channel)
70         if (
71             self.premeasure
72             and channel not in self.premeasured_channels
73             and track is not None
74             and self.terminal_reason is None
75         ):
76             center, radius = enclosing_circle(track.polygon)
77             if (
78                 radius > 38
79                 and tuple(self.position) not in track.observations
80                 and dist(self.position, center) - radius <= 1500

```

```

81         ):
82             self.zero_travel_probes += 1
83             self.measure(channel, self.position)
84             self.premeasured_channels.add(channel)
85             if channel in self.cleared or self.terminal_reason is not None:
86                 return
87         return super().localize(channel)
88
89     def clear(self, channel, point, guaranteed=False):
90         track = self.tracks.get(channel)
91         length = dist(self.position, point)
92         if (
93             self.approach_clear
94             and guaranteed
95             and track is not None
96             and length > 1e-6
97             and max(dist(point, p) for p in track.polygon) <= 19.9
98         ):
99             # Slide toward the robot inside the intersection of all vertex
100             # radius-19.9 disks. Each vertex supplies an exact quadratic bound.
101             u = (
102                 (self.position[0] - point[0]) / length,
103                 (self.position[1] - point[1]) / length,
104             )
105             shift = length
106             for p in track.polygon:
107                 dx, dy = point[0] - p[0], point[1] - p[1]
108                 dot = dx * u[0] + dy * u[1]
109                 discriminant = dot * dot + 19.9**2 - dx * dx - dy * dy
110                 if discriminant < 0.0:
111                     shift = 0.0
112                     break
113                 shift = min(shift, -dot + sqrt(discriminant))
114             shift = max(0.0, shift - 1e-6)
115             candidate = (point[0] + shift * u[0], point[1] + shift * u[1])
116             if max(dist(candidate, p) for p in track.polygon) <= 19.9:
117                 self.clear_access_saved_m += length - dist(self.position, candidate)
118                 point = candidate
119             return super().clear(channel, point, guaranteed=guaranteed)
120
121     def select_task(self):
122         anchors = {i: self.waypoints[i] for i in sorted(self.pending)}
123         targets = {
124             ch: enclosing_circle(track.polygon) for ch, track in self.tracks.items()
125         }
126         if self.portfolio:
127             return choose_portfolio_task(
128                 self.position, anchors, targets, risk_weight=self.risk_weight
129             )
130         return choose_task(
131             self.position,
132             anchors,
133             targets,
134             choice="guarded",
135             risk_weight=self.risk_weight,
136         )
137
138     def _result(self):
139         result = super()._result()
140         result.update(
141             strategy="field",
142             premeasure=self.premeasure,
143             negative_updates=self.negative_updates,
144             portfolio=self.portfolio,
145             risk_weight=self.risk_weight,
146             zero_travel_probes=self.zero_travel_probes,
147             negative_cuts=self.negative_cuts,

```

```

148         approach_clear=self.approach_clear,
149         clear_access_saved_m=self.clear_access_saved_m,
150     )
151     return result

```

Listing 6: 路线组合与负观测几何

```

1  """Bounded deterministic open-route portfolio using public task summaries only.
2
3  This module chooses an order, not an execution path or a clearance certificate.
4  It never executes actions, drops obligations, or reads simulator/source state.
5  """
6
7  from __future__ import annotations
8
9  from math import dist, fsum, isfinite
10
11  Point = tuple[float, float]
12  Task = tuple[str, int]
13
14
15  def _cost(
16      route: list[int],
17      origin: list[float],
18      edges: list[list[float]],
19      penalty: list[float],
20  ) -> float:
21      try:
22          value = fsum(
23              (
24                  origin[route[0]],
25                  penalty[route[0]],
26                  *(edges[a][b] for a, b in zip(route, route[1:])),
27              )
28          )
29      except OverflowError as error:
30          raise ValueError("route proxy must be finite") from error
31      if not isfinite(value):
32          raise ValueError("route proxy must be finite")
33      return value
34
35
36  def _insertion_delta(
37      route: list[int],
38      node: int,
39      gap: int,
40      origin: list[float],
41      edges: list[list[float]],
42      penalty: list[float],
43  ) -> float:
44      if gap == 0:
45          return (
46              origin[node]
47              + edges[node][route[0]]
48              - origin[route[0]]
49              + penalty[node]
50              - penalty[route[0]]
51          )
52      before = route[gap - 1]
53      delta = edges[before][node]
54      if gap < len(route):
55          after = route[gap]
56          delta += edges[node][after] - edges[before][after]
57      return delta
58
59
60  def _control_route(

```

```

61     origin: list[float], edges: list[list[float]], penalty: list[float]
62 ) -> list[int]:
63     """Exact route construction/move order of joint_dispatch_candidate.choose_task.
64
65     Keep this control independent of portfolio refinements: first task, nearest
66     updates, insertion arithmetic, scan order, tolerance and accepted-move cap
67     deliberately match the existing guarded implementation (center at weight 0).
68     """
69     n = len(origin)
70     first = min(range(n), key=lambda i: (origin[i] + penalty[i], i))
71     route = [first]
72     remaining = set(range(n))
73     remaining.remove(first)
74     nearest = [min(origin[i], edges[first][i]) for i in range(n)]
75     while remaining:
76         node = min(remaining, key=lambda i: (nearest[i], i))
77         best_gap = 0
78         best_delta = _insertion_delta(route, node, 0, origin, edges, penalty)
79         for gap in range(1, len(route) + 1):
80             delta = _insertion_delta(route, node, gap, origin, edges, penalty)
81             if delta < best_delta:
82                 best_gap, best_delta = gap, delta
83         route.insert(best_gap, node)
84         remaining.remove(node)
85         for other in remaining:
86             nearest[other] = min(nearest[other], edges[node][other])
87
88     for _ in range(n):
89         changed = False
90         for left in range(n - 1):
91             for right in range(left + 1, n):
92                 old = (
93                     origin[route[left]]
94                     if left == 0
95                     else edges[route[left - 1]][route[left]]
96                 )
97                 new = (
98                     origin[route[right]]
99                     if left == 0
100                    else edges[route[left - 1]][route[right]]
101                )
102                 if right + 1 < n:
103                     old += edges[route[right]][route[right + 1]]
104                     new += edges[route[left]][route[right + 1]]
105                 if left == 0:
106                     old += penalty[route[left]]
107                     new += penalty[route[right]]
108                 if new + 1e-12 * max(1.0, old, new) < old:
109                     route[left : right + 1] = reversed(route[left : right + 1])
110                     changed = True
111                     break
112             if changed:
113                 break
114         if not changed:
115             break
116     return route
117
118
119 def _cheapest_route(
120     origin: list[float], edges: list[list[float]], penalty: list[float]
121 ) -> list[int]:
122     """Choose the globally cheapest (remaining task, gap), not nearest task."""
123     n = len(origin)
124     first = min(range(n), key=lambda i: (origin[i] + penalty[i], i))
125     route = [first]
126     remaining = set(range(n))
127     remaining.remove(first)

```

```

128     while remaining:
129         # Tuple comparison makes node/gap ties independent of set iteration.
130         _, node, gap = min(
131             (_insertion_delta(route, node, gap, origin, edges, penalty), node, gap)
132             for node in remaining
133             for gap in range(len(route) + 1)
134         )
135         route.insert(gap, node)
136         remaining.remove(node)
137     return route
138
139
140 def _nearest_route(first: int, edges: list[list[float]]) -> list[int]:
141     remaining = set(range(len(edges)))
142     remaining.remove(first)
143     route = [first]
144     while remaining:
145         node = min(remaining, key=lambda i: (edges[route[-1]][i], i))
146         route.append(node)
147         remaining.remove(node)
148     return route
149
150
151 def _improve(
152     route: list[int],
153     origin: list[float],
154     edges: list[list[float]],
155     penalty: list[float],
156 ) -> tuple[list[int], float]:
157     """Best improvement across reversals and one-node relocations, bounded passes."""
158     n = len(route)
159     cost = _cost(route, origin, edges, penalty)
160     for _ in range(min(n, 12)):
161         # Ignore changes below the scale-dependent numerical margin. Every
162         # accepted move is also checked against the complete recomputed proxy.
163         best_delta = -1e-12 * max(1.0, cost)
164         move: tuple[str, int, int] | None = None
165         for left in range(n - 1):
166             for right in range(left + 1, n):
167                 old = (
168                     origin[route[left]]
169                     if left == 0
170                     else edges[route[left - 1]][route[left]]
171                 )
172                 new = (
173                     origin[route[right]]
174                     if left == 0
175                     else edges[route[left - 1]][route[right]]
176                 )
177                 if right + 1 < n:
178                     old += edges[route[right]][route[right + 1]]
179                     new += edges[route[left]][route[right + 1]]
180                 if left == 0:
181                     old += penalty[route[left]]
182                     new += penalty[route[right]]
183                 delta = new - old
184                 if delta < best_delta:
185                     best_delta, move = delta, ("reverse", left, right)
186
187     for index, node in enumerate(route):
188         if n == 1:
189             break
190         if index == 0:
191             following = route[1]
192             removal = (
193                 origin[following]
194                 + penalty[following]

```

```

195         - origin[node]
196         - penalty[node]
197         - edges[node][following]
198     )
199     else:
200         before = route[index - 1]
201         removal = -edges[before][node]
202         if index + 1 < n:
203             following = route[index + 1]
204             removal += edges[before][following] - edges[node][following]
205         # Gaps are in the ORIGINAL route. The two adjacent gaps are no-ops;
206         # every other gap retains its endpoints when node is removed.
207         for gap in range(n + 1):
208             if gap == index or gap == index + 1:
209                 continue
210             delta = removal + _insertion_delta(
211                 route, node, gap, origin, edges, penalty
212             )
213             if delta < best_delta:
214                 best_delta, move = delta, ("relocate", index, gap)
215
216         if move is None:
217             break
218         candidate = route.copy()
219         kind, left, right = move
220         if kind == "reverse":
221             candidate[left : right + 1] = reversed(candidate[left : right + 1])
222         else:
223             node = candidate.pop(left)
224             candidate.insert(right if right < left else right - 1, node)
225             candidate_cost = _cost(candidate, origin, edges, penalty)
226             if not candidate_cost < cost:
227                 # Cancellation in an edge delta must never worsen the true proxy.
228                 break
229             route, cost = candidate, candidate_cost
230     return route, cost
231
232
233 def choose_portfolio_task(
234     position: Point,
235     anchors: dict[int, Point],
236     targets: dict[int, tuple[Point, float]],
237     risk_weight: float = 1.0,
238 ) -> Task:
239     """Return the first task of the best complete deterministic open permutation.
240
241     Proxy = origin-to-first distance + all consecutive center/anchor distances
242     + risk_weight * first target radius (zero for a first anchor). Weight zero
243     reproduces the old center objective. No return-to-origin edge is charged.
244     The retained control is exactly the old nearest-insertion/capped-2-opt
245     route, so the selected complete route's computed proxy is never worse.
246     This is NOT a guarantee about the first hop, executed distance or total T.
247
248     At most four seeds (control, global cheapest insertion, two nearest-neighbor
249     starts) receive min(n, 12) best-improvement passes each. Tasks sort target
250     before anchor, then integer key; equal costs retain the earlier candidate.
251     All tasks survive, including identical coordinates/keys across task kinds.
252     Inputs are never mutated. Empty tasks, nonfinite geometry/proxy or negative
253     radii/risk weight raise ValueError. Time  $O(n^3)$ , memory  $O(n^2)$ .
254     """
255     target_keys = sorted(targets)
256     anchor_keys = sorted(anchors)
257     tasks: list[Task] = [("target", key) for key in target_keys]
258     tasks.extend(("anchor", key) for key in anchor_keys)
259     n = len(tasks)
260     if not n:
261         raise ValueError("cannot dispatch an empty task set")

```

```

262 coords = [targets[key][0] for key in target_keys]
263 coords.extend(anchors[key] for key in anchor_keys)
264 radii = [targets[key][1] for key in target_keys]
265 radii.extend(0.0 for _ in anchor_keys)
266 if any(
267     len(point) != 2 or not all(isfinite(x) for x in point)
268     for point in [position, *coords]
269 ):
270     raise ValueError("positions must be finite two-dimensional points")
271 if any(not isfinite(radius) or radius < 0 for radius in radii):
272     raise ValueError("MEC radii must be finite and nonnegative")
273 if not isfinite(risk_weight) or risk_weight < 0:
274     raise ValueError("risk_weight must be finite and nonnegative")
275 penalty = [risk_weight * radius for radius in radii]
276 if any(not isfinite(value) for value in penalty):
277     raise ValueError("weighted radii must be finite")
278 origin = [dist(position, point) for point in coords]
279 edges = [[0.0] * n for _ in range(n)]
280 for i in range(n):
281     for j in range(i):
282         edges[i][j] = edges[j][i] = dist(coords[i], coords[j])
283 if any(not isfinite(value) for value in origin) or any(
284     not isfinite(value) for row in edges for value in row
285 ):
286     raise ValueError("pairwise distances must be finite")
287 # This conservative finite upper bound also excludes overflow in local
288 # delta arithmetic; physical simulator distances are far below this limit.
289 try:
290     bound = fsum((max(origin), max(penalty), *(max(row) for row in edges)))
291 except OverflowError as error:
292     raise ValueError("route proxy bound must be finite") from error
293 if not isfinite(bound):
294     raise ValueError("route proxy bound must be finite")
295
296 control = _control_route(origin, edges, penalty)
297 control_cost = _cost(control, origin, edges, penalty)
298 best_route, best_cost = control, control_cost
299 # The second start is the next distinct task in origin+penalty order, not
300 # another random restart or a second copy of the same first task.
301 starts = sorted(range(n), key=lambda i: (origin[i] + penalty[i], i))[:2]
302 seeds = [control, _cheapest_route(origin, edges, penalty)]
303 seeds.extend(_nearest_route(first, edges) for first in starts)
304 seen: set[tuple[int, ...]] = set()
305 for seed in seeds:
306     identity = tuple(seed)
307     if identity in seen:
308         continue
309     seen.add(identity)
310     route, cost = _improve(seed, origin, edges, penalty)
311     if cost < best_cost:
312         best_route, best_cost = route, cost
313 # Explicit control fallback guards the proxy guarantee, not actual travel.
314 if best_cost > control_cost:
315     return tasks[control[0]]
316 return tasks[best_route[0]]

```

Listing 7: 本地实验与官方日志分析入口

```

1 """Frozen field-policy confirmation; disjoint from official and development cases."""
2
3 from __future__ import annotations
4 import json
5 from pathlib import Path
6 from field_study import run_field, pair_stats, summarize
7
8 CONFIGS = {
9     3: {

```

```

10     "control": None,
11     "route_only": {"premeasure": False, "portfolio": True},
12     "route_info": {"premeasure": True, "negative_updates": True, "portfolio": True},
13     "field": {
14         "premeasure": True,
15         "negative_updates": True,
16         "portfolio": True,
17         "approach_clear": True,
18     },
19 },
20 4: {"control": None, "field": {"premeasure": True, "portfolio": True}},
21 }
22
23
24 def group(problem, name, seeds, scenario="random", mode="smooth"):
25     rows = [
26         run_field(seed, problem, CONFIGS[problem][name], scenario, mode)
27         for seed in seeds
28     ]
29     for row in rows:
30         row["configuration"] = name
31     return rows
32
33
34 def main():
35     records = []
36     summary = {}
37     comparisons = {}
38     stress = {}
39     for p in (3, 4):
40         groups = {}
41         for name in CONFIGS[p]:
42             groups[name] = group(p, name, range(7100000, 7101000))
43             records.extend(groups[name])
44             summary[f"Q{p}_{name}"] = summarize(groups[name])
45             comparisons[f"Q{p}"] = pair_stats(groups["control"], groups["field"])
46             comparisons[f"Q{p}_ablation"] = {
47                 f"{a}_to_{b}": pair_stats(groups[a], groups[b])
48                 for a, b in zip(groups, list(groups)[1:])
49             }
50             print(f"Q{p}", json.dumps(comparisons[f"Q{p}"]), flush=True)
51             cases = [
52                 (s, m, range(7200000, 7200050))
53                 for s in ("minimum_radius", "outward_boundary", "clustered")
54                 for m in ("smooth", "extreme", "iid_location")
55             ]
56             cases += [
57                 ("random", m, range(7210000, 7210200)) for m in ("extreme", "iid_location")
58             ]
59             for s, m, seeds in cases:
60                 a = group(p, "control", seeds, s, m)
61                 b = group(p, "field", seeds, s, m)
62                 records.extend(a)
63                 records.extend(b)
64                 key = f"Q{p}_{s}_{m}"
65                 stress[key] = {
66                     "control": summarize(a),
67                     "field": summarize(b),
68                     "paired": pair_stats(a, b),
69                 }
70                 print(key, json.dumps(stress[key]["paired"]), flush=True)
71     result = {
72         "evidence": "fresh_local_paired_confirmation_not_official_counterfactual",
73         "selection": "Selected using official-observation bottlenecks and development seeds6100000..6100149  
only. Refined probe=.22 control is unchanged. No retuning after this study.",
74         "main_seed_range": [7100000, 7100999],
75         "stress_seed_range": [7200000, 7200049],

```

```

76     "alternate_error_seed_range": [7210000, 7210199],
77     "development_seed_range": [6100000, 6100149],
78     "main_runs": 6000,
79     "stress_runs": 1800,
80     "alternate_error_runs": 1600,
81     "total_runs": len(records),
82     "configs": CONFIGS,
83     "summary": summary,
84     "comparison": comparisons,
85     "stress": stress,
86     "runs": records,
87 }
88 assert len(records) == 9400
89 Path("results/field_confirmation.json").write_text(
90     json.dumps(result, ensure_ascii=False, indent=2) + "\n"
91 )
92 print(json.dumps({"total_runs": len(records), "summary": summary}, indent=2))
93
94
95 if __name__ == "__main__":
96     main()

```

Listing 8: 统一打包与复现验证

```

1  """Build anonymous research materials; replay local studies and supplied practice."""
2
3  from __future__ import annotations
4
5  import argparse
6  import csv
7  import hashlib
8  import importlib.metadata
9  import json
10 import os
11 import re
12 import shutil
13 import subprocess
14 import sys
15 import tempfile
16 import time
17 import zipfile
18 from pathlib import Path, PurePosixPath
19
20 import pymupdf
21
22 ROOT = Path(__file__).resolve().parent
23 SOURCES = (
24     "geometry.py",
25     "coverage.py",
26     "planner.py",
27     "environment.py",
28     "client.py",
29     "run.py",
30     "official_run.py",
31     "intersection.py",
32     "second_point.py",
33     "experiments.py",
34     "compare_recovery.py",
35     "compare_routes.py",
36     "compare_probes.py",
37     "verify_geometry.py",
38     "verify_coverage_limits.py",
39     "verify_refinements.py",
40     "verify_terminal.py",
41     "verify_http.py",
42     "make_enhancement_assets.py",
43     "make_assets.py",

```

```

44     "audit_pdf.py",
45     "package.py",
46     "active_policy_candidate.py",
47     "route_policy_candidate.py",
48     "active_policy_notes.txt",
49     "route_policy_notes.txt",
50     "polar_coverage_candidate.py",
51     "polar_coverage_notes.txt",
52     "joint_dispatch_candidate.py",
53     "joint_dispatch_notes.txt",
54     "optical_policy_candidate.py",
55     "optical_policy_notes.txt",
56     "refinement_study.py",
57     "research_experiments.py",
58     "verify_research.py",
59     "make_research_assets.py",
60     "make_official_assets.py",
61     "fast_vs_compact_410.py",
62     "field_policy.py",
63     "negative_geometry.py",
64     "route_portfolio.py",
65     "field_study.py",
66     "field_confirmation.py",
67     "verify_field.py",
68     "official_analysis.py",
69     "make_field_assets.py",
70     "make_competition_assets.py",
71     "negative_geometry_notes.txt",
72     "route_portfolio_notes.txt",
73     "contracts.json",
74     "report_contracts.json",
75     "enhancements.py",
76     "ring_sweep.py",
77     "develop_anchors.py",
78     "theory_additions.txt",
79     "paper_expansion.txt",
80     "coverage_notes.txt",
81     "coverage_limits.tex",
82     "requirements.txt",
83     "reproduce.txt",
84     "official_status.json",
85     "audit/route_exact_audit.py",
86     "audit/q3_ablation_audit.py",
87     "audit/distribution_audit.py",
88     "audit/correctness_audit.py",
89     "audit/alternative_screen.py",
90     "audit/freeze_compare.py",
91 )
92 RESULTS = (
93     "final_experiments.json",
94     "geometry_verification.json",
95     "intersection_verification.json",
96     "second_point_example.json",
97     "recovery_development.json",
98     "route_candidate_comparison.json",
99     "adaptive_candidate_comparison.json",
100    "terminal_verification.json",
101    "ring_sweep.json",
102    "enhancement_experiments.json",
103    "anchor_development.json",
104    "enhancement_review.json",
105    "coverage_limits.json",
106    "refinement_verification.json",
107    "refinement_development.json",
108    "research_experiments.json",
109    "research_verification.json",
110    "research_review.json",

```

```

111     "field_development.json",
112     "field_confirmation.json",
113     "field_verification.json",
114     "field_review.json",
115     "official_practice_analysis.json",
116     "fast_vs_compact_410.json",
117     "practice_optimization_screen.json",
118     "http_verification.json",
119     "http_deadline_verification.json",
120     "review_resolutions.json",
121     "operation_bound.json",
122     "paper_review.json",
123     "visual_review.json",
124     "final_review.json",
125     "user_supplied_field_practice_summary.json",
126     "environment_manifest.json",
127     "audit_route_exact.json",
128     "audit_q3_ablation.json",
129     "audit_distribution.json",
130     "audit_correctness.json",
131     "audit_alternatives.json",
132     "audit_freeze_compare.json",
133 )
134
135 def verify_asset_inputs(root: Path):
136     for metadata_name in ("metadata.json", "enhancement_metadata.json", "research_metadata.json", "
137         official_practice_metadata.json", "field_metadata.json"):
138         metadata = json.loads((root / "report/generated" / metadata_name).read_text())
139         for name, expected in metadata["input_sha256"].items():
140             actual = hashlib.sha256((root / name).read_bytes()).hexdigest()
141             if actual != expected:
142                 raise ValueError(f"Stale generated asset input: {name}")
143
144 def deterministic_content(value):
145     """Exclude measured wall clocks and their maximum, not behavioral statistics."""
146     if isinstance(value, dict):
147         return {key: deterministic_content(item) for key, item in value.items()
148             if key not in ("real_duration_s", "max_real_duration_s", "max_real_s")}
149     if isinstance(value, list):
150         return [deterministic_content(item) for item in value]
151     return value
152
153
154 def practice_files(root: Path):
155     """Allow only the manifest's 60 sanitized practice files, never private raw logs."""
156     root = root.resolve()
157     manifest = root / "manifest.csv"
158     if manifest.is_symlink():
159         raise ValueError("Practice manifest must not be a symbolic link")
160     with manifest.open(encoding="utf-8", newline="") as stream:
161         rows = list(csv.DictReader(stream))
162         if len(rows) != 20 or {(int(r["problem"]), int(r["round"])) for r in rows} != {
163             (p, n) for p in (3, 4) for n in range(1, 11)
164         }:
165             raise ValueError("Expected exactly ten supplied practice rounds per problem")
166     files = {"manifest.csv": manifest}
167
168 def check_identifiers(value):
169     if isinstance(value, dict):
170         for key, item in value.items():
171             if key == "robot_id" and item != "TEAM_REDACTED":
172                 raise ValueError("Unredacted robot_id in practice data")
173             check_identifiers(item)
174     elif isinstance(value, list):
175         for item in value:
176             check_identifiers(item)

```

```

177
178 for row in rows:
179     controller = f"data/controller/q{int(row['problem'])}/round_{int(row['round']):02d}"
180     for key in ("private_requests", "observed_result", "simulator_result"):
181         name = row[key]
182         relative = PurePosixPath(name)
183         if relative.is_absolute() or "." in relative.parts or "\\" in name:
184             raise ValueError(f"Unsafe practice path: {name}")
185         if key == "simulator_result":
186             allowed = (relative.parent == PurePosixPath("data/simulator_results")
187                       and re.fullmatch(r"practice-p[34]-[0-9]+-[A-Z0-9-]+\..result\.json", relative.name))
188         else:
189             filename = "private_requests.jsonl" if key == "private_requests" else "observed_result.json"
190             allowed = name == f"{controller}/{filename}"
191         path = root / relative
192         if not allowed or path.is_symlink() or not path.resolve().is_relative_to(root):
193             raise ValueError(f"Practice path outside permitted source scope: {name}")
194         if name in files:
195             raise ValueError(f"Repeated practice source: {name}")
196         text = path.read_text(encoding="utf-8")
197         values = [json.loads(line) for line in text.splitlines()] if key == "private_requests" else [json.
loads(text)]
198         for value in values:
199             check_identifiers(value)
200             if key == "private_requests" and value.get("event") == "request":
201                 if value["body"].get("robot_id") != "TEAM_REDACTED":
202                     raise ValueError(f"Missing sanitized request identity: {name}")
203                 if isinstance(value, dict) and "body_utf8" in value:
204                     check_identifiers(json.loads(value["body_utf8"]))
205             files[name] = path
206 if len(files) != 61:
207     raise ValueError("Expected 60 distinct practice files plus manifest")
208 return files
209
210
211 def verify_practice_inputs(root: Path):
212     files = practice_files(root / "practice_data")
213     analysis = json.loads((root / "results/official_practice_analysis.json").read_text())
214     actual = {name: hashlib.sha256(path.read_bytes()).hexdigest() for name, path in files.items()}
215     if actual != analysis["input_sha256"]:
216         raise ValueError("Official practice source hashes do not match analysis")
217     return actual
218
219 def verify_archive(archive: Path):
220     records = []
221     with tempfile.TemporaryDirectory(prefix="b-reproduce-") as directory:
222         extracted = Path(directory)
223         with zipfile.ZipFile(archive) as bundle:
224             for item in bundle.infolist():
225                 relative = PurePosixPath(item.filename)
226                 if relative.is_absolute() or "." in relative.parts:
227                     raise ValueError("Unsafe member in generated archive")
228                 bundle.extractall(extracted)
229             manifest = json.loads((extracted / "MANIFEST.json").read_text())
230             for name, information in manifest["files"].items():
231                 raw = (extracted / name).read_bytes()
232                 assert hashlib.sha256(raw).hexdigest() == information["sha256"], name
233             verify_asset_inputs(extracted)
234             practice_hashes = verify_practice_inputs(extracted)
235             immutable_names = [name for name in manifest["files"]
236                               if name in SOURCES or name.startswith("practice_data/")
237                               or (name.startswith("report/") and name.endswith(".tex")
238                                   and not name.startswith("report/generated/"))]
239             expected_runs = {
240                 "final_experiments.json": 4430,
241                 "enhancement_experiments.json": 3080,
242                 "ring_sweep.json": 2200,

```

```

243         "refinement_development.json": 2550,
244         "research_experiments.json": 10700,
245         "field_development.json": 2250,
246         "field_confirmation.json": 9400,
247     }
248     baselines = {
249         name: json.loads((extracted / "results" / name).read_text())
250         for name in (*expected_runs, "research_verification.json", "field_verification.json", "
official_practice_analysis.json", "fast_vs_compact_410.json", "practice_optimization_screen.json")
251     }
252     commands = [
253         [sys.executable, "verify_geometry.py"],
254         [sys.executable, "intersection.py"],
255         [sys.executable, "second_point.py"],
256         [
257             sys.executable,
258             "experiments.py",
259             "--cases",
260             "500",
261             "--output",
262             "results/final_experiments.json",
263         ],
264         [sys.executable, "enhancements.py"],
265         [sys.executable, "ring_sweep.py"],
266         [sys.executable, "verify_terminal.py"],
267         [sys.executable, "verify_refinements.py"],
268         [sys.executable, "verify_coverage_limits.py"],
269         [sys.executable, "compare_routes.py"],
270         [sys.executable, "compare_probes.py"],
271         [sys.executable, "verify_research.py"],
272         [sys.executable, "official_analysis.py", "--handoff", "practice_data"],
273         [sys.executable, "make_official_assets.py"],
274         [sys.executable, "fast_vs_compact_410.py"],
275         [sys.executable, "refinement_study.py", "--cases", "150", "--seed", "3100000"],
276         [sys.executable, "research_experiments.py"],
277         [sys.executable, "verify_field.py"],
278         [sys.executable, "field_study.py", "--cases", "150", "--seed", "6100000"],
279         [sys.executable, "field_confirmation.py"],
280         [sys.executable, "make_assets.py"],
281         [sys.executable, "make_enhancement_assets.py"],
282         [sys.executable, "make_research_assets.py"],
283         [sys.executable, "make_field_assets.py"],
284         [
285             "latexmk",
286             "-xelatex",
287             "-interaction=nonstopmode",
288             "-halt-on-error",
289             "paper.tex",
290         ],
291         [
292             "latexmk",
293             "-xelatex",
294             "-interaction=nonstopmode",
295             "-halt-on-error",
296             "ai_usage.tex",
297         ],
298     ]
299     environment = {key: value for key, value in os.environ.items() if key != "PYTHONPATH"}
300     environment["MPLBACKEND"] = "Agg"
301     for command in commands:
302         started = time.monotonic()
303         cwd = extracted / "report" if command[0] == "latexmk" else extracted
304         completed = subprocess.run(
305             command,
306             cwd=cwd,
307             text=True,
308             stdout=subprocess.PIPE,

```

```

309         stderr=subprocess.STDOUT,
310         timeout=1800,
311         check=False,
312         env=environment,
313     )
314     records.append(
315         {
316             "command": [Path(command[0]).name, *command[1:]],
317             "returncode": completed.returncode,
318             "seconds": time.monotonic() - started,
319         }
320     )
321     if completed.returncode:
322         raise RuntimeError(
323             f"Extracted archive command failed: {command}\n{completed.stdout[-12000:]}"
324         )
325     reproduced_runs = {}
326     auxiliary_runs = {}
327     for name, baseline in baselines.items():
328         replay = json.loads((extracted / "results" / name).read_text())
329         if name in expected_runs:
330             expected = expected_runs[name]
331             assert baseline["total_runs"] == replay["total_runs"] == expected, name
332             if name == "ring_sweep.json":
333                 count = sum(len(group["runs"]) for phase in ("development", "held_out")
334                             for group in replay[phase].values())
335             else:
336                 count = len(replay["runs"])
337                 for old, new in zip(baseline["runs"], replay["runs"], strict=True):
338                     assert deterministic_content(old) == deterministic_content(new), (name, old["seed"])
339             assert count == expected, (name, count)
340             reproduced_runs[name] = count
341         if name == "fast_vs_compact_410.json":
342             assert baseline["total_runs"] == replay["total_runs"] == 2000
343             for label in ("compact25", "fast25"):
344                 old_rows = [r for r in baseline["runs"] if r["configuration"] == label]
345                 new_rows = [r for r in replay["runs"] if r["configuration"] == label]
346                 assert [r["seed"] for r in old_rows] == list(range(4100000, 4101000))
347                 assert [r["seed"] for r in new_rows] == list(range(4100000, 4101000))
348                 for old, new in zip(old_rows, new_rows, strict=True):
349                     assert deterministic_content(old) == deterministic_content(new)
350             auxiliary_runs[name] = len(replay["runs"])
351         assert deterministic_content(baseline) == deterministic_content(replay), name
352     verify_asset_inputs(extracted)
353     assert verify_practice_inputs(extracted) == practice_hashes
354     for name in immutable_names:
355         assert hashlib.sha256((extracted / name).read_bytes()).hexdigest() == manifest["files"][name][
            "sha256"], name
356     assert sum(reproduced_runs.values()) == 34610
357     for filename in ("paper", "ai_usage"):
358         log = (extracted / "report" / f"{filename}.log").read_text(errors="replace")
359         assert "Missing character:" not in log and "Overfull" not in log
360         assert (
361             "undefined references" not in log and "undefined citations" not in log
362         )
363     assert len(pymupdf.open(extracted / "report" / f"{filename}.pdf")) > 0
364     return {
365         "status": "passed",
366         "archive_sha256": hashlib.sha256(archive.read_bytes()).hexdigest(),
367         "manifest_hashes_verified": True,
368         "all_4430_behavioral_runs_reproduced_exactly": True,
369         "all_3080_enhancement_runs_reproduced_exactly": True,
370         "ring_sweep_reproduced_exactly": True,
371         "all_2200_ring_runs_reproduced_exactly": True,
372         "all_10700_research_runs_reproduced_exactly": True,
373         "all_2550_development_runs_reproduced_exactly": True,
374         "research_verification_reproduced_exactly": True,

```

```

375     "all_2250_field_development_runs_reproduced_exactly": True,
376     "all_9400_field_confirmation_runs_reproduced_exactly": True,
377     "field_verification_reproduced_exactly": True,
378     "practice_source_hashes_verified_before_and_after": practice_hashes,
379     "immutable_source_hashes_verified_after_replay": True,
380     "pythonpath_removed": True,
381     "official_practice_analysis_reproduced_exactly": True,
382     "fast_vs_compact_screen_reproduced_exactly": True,
383     "generated_asset_input_hashes_verified": True,
384     "experiment_runs_reproduced_by_dataset": reproduced_runs,
385     "experiment_runs_reproduced_total": sum(reproduced_runs.values()),
386     "auxiliary_runs_excluded_from_core_total": auxiliary_runs,
387     "excluded_nondeterminism": "local measured wall-clock real_duration_s, max_real_duration_s and
max_real_s only; official timestamp/runtime observations remain exact",
388     "commands": records,
389 }
390
391
392 def main():
393     parser = argparse.ArgumentParser()
394     parser.add_argument("--verify", action="store_true")
395     parser.add_argument("--handoff", type=Path, default=(ROOT / "practice_data" if (ROOT / "practice_data").
is_dir()
396                                     else ROOT.parent / "ai_optimization_handoff_20260912"))
397     args = parser.parse_args()
398     verify_asset_inputs(ROOT)
399     for prefix, name in (("paper", "paper"), ("ai", "ai_usage")):
400         audit = json.loads((ROOT / "report/visual" / f"{prefix}_audit.json").read_text())
401         actual = hashlib.sha256((ROOT / "report" / f"{name}.pdf").read_bytes()).hexdigest()
402         if audit["sha256"] != actual or audit["out_of_page_spans"] or audit["replacement_characters"]:
403             raise ValueError(f"Missing or stale PDF audit: {name}")
404         if prefix == "paper" and not (audit["abstract_keywords_on_first_page"]
405                                     and 0 < audit["body_pages_excluding_abstract"] <= 30):
406             raise ValueError("Paper abstract/body page rule violated")
407     environment = {
408         "python": sys.version,
409         "environment": "conda py314",
410         "packages": {
411             name: importlib.metadata.version(name)
412             for name in ("numpy", "scipy", "matplotlib", "pymupdf")
413         },
414     }
415     (ROOT / "results/environment_manifest.json").write_text(
416         json.dumps(environment, indent=2) + "\n"
417     )
418     files = {name: ROOT / name for name in SOURCES}
419     files.update({"results/" + name: ROOT / "results" / name for name in RESULTS})
420     supplied_practice = practice_files(args.handoff)
421     analysis = json.loads((ROOT / "results/official_practice_analysis.json").read_text())
422     practice_hashes = {name: hashlib.sha256(path.read_bytes()).hexdigest()
423                       for name, path in supplied_practice.items()}
424     if practice_hashes != analysis["input_sha256"]:
425         raise ValueError("Supplied practice files do not match official analysis input hashes")
426     files.update({"practice_data/" + name: path for name, path in supplied_practice.items()})
427     files["AI工具使用详情.pdf"] = ROOT / "report/ai_usage.pdf"
428     for path in (ROOT / "report").iterdir():
429         if path.suffix in (".tex", ".pdf"):
430             files[path.relative_to(ROOT).as_posix()] = path
431     for folder in ("generated", "figures"):
432         for path in (ROOT / "report" / folder).iterdir():
433             if path.is_file() and not path.name.startswith("."):
434                 files[path.relative_to(ROOT).as_posix()] = path
435     for filename in ("paper_audit.json", "ai_audit.json"):
436         path = ROOT / "report" / "visual" / filename
437         files[path.relative_to(ROOT).as_posix()] = path
438     missing = [name for name, path in files.items() if not path.is_file()]
439     if missing:

```

```

440         raise RuntimeError(f"Deliverable proof is missing: {missing}")
441     anonymous_pattern = re.compile(r"/Users/[A-Za-z0-9._-]+/")
442     for name, path in files.items():
443         if path.suffix in (".py", ".txt", ".tex", ".json", ".jsonl", ".csv") and anonymous_pattern.search(
444             path.read_text(encoding="utf-8")
445         ):
446             raise ValueError(f"Personal absolute path in anonymous material: {name}")
447         if path.suffix == ".pdf":
448             document = pymupdf.open(path)
449             if anonymous_pattern.search(
450                 "\n".join(page.get_text() for page in document)
451             ):
452                 raise ValueError(f"Personal absolute path in PDF: {name}")
453     manifest = {
454         "status": "RESEARCH_VERSION_OFFICIAL_PRACTICE_COMPLETE_FORMAL_TESTS_PENDING",
455         "not_ready_for_contest_submission": True,
456         "official_practice": "Twenty user-supplied Q3/Q4 practice observations were analyzed offline; formal
457         tests and encrypted exports remain pending",
458         "official_practice_replay_boundary": "All 20 supplied traces are independently costed; strict baseline
459         policy replay matches only 8/20. Divergence stops replay, with no fabricated responses.",
460         "practice_source_files": 60,
461         "default_strategy": "Q3 refined, Q4 field; explicit --strategy overrides",
462         "official_practice_control": "refined, probe_scale=0.22 for Q3 and Q4",
463         "human_review": "No actual contestant review record; AI review is not human review",
464         "python_environment": "conda py314",
465         "python_version": sys.version.split()[0],
466         "files": {
467             name: {
468                 "sha256": hashlib.sha256(path.read_bytes()).hexdigest(),
469                 "bytes": path.stat().st_size,
470             }
471             for name, path in sorted(files.items())
472         },
473     }
474     destination = ROOT / "delivery"
475     destination.mkdir(exist_ok=True)
476     archive = destination / "B题研究支撑材料.zip"
477     with zipfile.ZipFile(
478         archive, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9
479     ) as bundle:
480         for name, path in sorted(files.items()):
481             bundle.write(path, name)
482             bundle.writestr(
483                 "MANIFEST.json", json.dumps(manifest, ensure_ascii=False, indent=2) + "\n"
484             )
485     with zipfile.ZipFile(archive) as bundle:
486         assert bundle.testzip() is None
487         for name, information in manifest["files"].items():
488             assert hashlib.sha256(bundle.read(name)).hexdigest() == information["sha256"], name
489     verify_asset_inputs(ROOT)
490     for name, path in supplied_practice.items():
491         assert hashlib.sha256(path.read_bytes()).hexdigest() == practice_hashes[name], name
492     shutil.copy2(ROOT / "report/paper.pdf", destination / "B题研究论文.pdf")
493     shutil.copy2(ROOT / "report/ai_usage.pdf", destination / "AI工具使用详情.pdf")
494     for path in (
495         archive,
496         destination / "B题研究论文.pdf",
497         destination / "AI工具使用详情.pdf",
498     ):
499         assert path.stat().st_size <= 20 * 2**20, path.name
500     status = {
501         "status": manifest["status"],
502         "official_tests": "PRACTICE_COMPLETE_FORMAL_PENDING",
503         "human_review": "OUTSTANDING",
504         "source_files": len(files),
505         "archive_bytes": archive.stat().st_size,
506         "archive_sha256": hashlib.sha256(archive.read_bytes()).hexdigest(),

```

```

505     }
506     if args.verify:
507         proof = verify_archive(archive)
508         (destination / "复现验证.json").write_text(
509             json.dumps(proof, ensure_ascii=False, indent=2) + "\n"
510         )
511         status["extracted_archive_reproduction"] = "passed"
512         (destination / "交付状态.json").write_text(
513             json.dumps(status, ensure_ascii=False, indent=2) + "\n"
514         )
515         print(json.dumps(status, ensure_ascii=False, indent=2))
516
517
518 if __name__ == "__main__":
519     main()

```

Listing 9: 关键几何和状态核验

```

1  """Boundary/invariant checks for field policy; synthetic and analytic evidence."""
2
3  from __future__ import annotations
4  import json
5  import math
6  from pathlib import Path
7  import random
8
9  from field_policy import FieldPlanner
10 from geometry import _cross, clip_bearing, enclosing_circle, initial_polygon
11 from negative_geometry import exclude_omni_disk
12 from planner import Track
13 from route_portfolio import _control_route, _cost, _improve, choose_portfolio_task
14 from joint_dispatch_candidate import choose_task
15 from verify_terminal import DeadlineAdapter
16
17
18 def contains(poly, p, tolerance=2e-6):
19     if not poly:
20         return False
21     if len(poly) == 1:
22         return math.dist(poly[0], p) <= tolerance
23     if len(poly) == 2:
24         a, b = poly
25         return abs(math.dist(a, p) + math.dist(b, p) - math.dist(a, b)) <= tolerance
26     return all(
27         _cross(poly[i], poly[(i + 1) % len(poly)], p)
28         >= -tolerance * max(1, math.dist(poly[i], poly[(i + 1) % len(poly)]))
29         for i in range(len(poly))
30     )
31
32
33 def negative_checks():
34     rng = random.Random(6311000)
35     count = 0
36     largest_cut = 0.0
37     for sides in (16, 32):
38         for index in range(500):
39             phi = rng.uniform(0, math.tau)
40             r = 1800 * math.sqrt(rng.random())
41             source = (r * math.cos(phi), r * math.sin(phi))
42             theta = rng.uniform(0, math.tau)
43             distance = rng.uniform(6, 1500)
44             station = (
45                 source[0] - distance * math.cos(theta),
46                 source[1] - distance * math.sin(theta),
47             )
48             poly = clip_bearing(
49                 initial_polygon(),

```

```

50         station,
51         (math.degrees(theta) + rng.uniform(-1, 1)) % 360,
52     )
53     for j in range(3):
54         a = rng.uniform(0, math.tau)
55         gap = 1000 + 1e-8 if index % 3 == 0 else rng.uniform(1000, 2500)
56         negative = (
57             source[0] + gap * math.cos(a),
58             source[1] + gap * math.sin(a),
59         )
60         before = poly.copy()
61         poly = exclude_omni_disk(poly, negative, sides=sides)
62         assert before and poly and contains(poly, source), (
63             sides,
64             index,
65             source,
66             negative,
67         )
68         assert all(contains(before, v) for v in poly)
69         largest_cut = max(
70             largest_cut, enclosing_circle(before)[1] - enclosing_circle(poly)[1]
71         )
72         count += 1
73     assert exclude_omni_disk([(1000.0, 0.0)], (0.0, 0.0), sides=sides) == [
74         (1000.0, 0.0)
75     ]
76     assert exclude_omni_disk([(0.0, 0.0)], (0.0, 0.0), sides=sides) == []
77     segment = [(-1200.0, 0.0), (1200.0, 0.0)]
78     assert exclude_omni_disk(segment, (0.0, 0.0), sides=sides) == segment
79     for shift in ((2e6, -2e6), (0.0, 0.0)):
80         p = [
81             (shift[0] + 1200, shift[1] - 5),
82             (shift[0] + 1400, shift[1] - 5),
83             (shift[0] + 1400, shift[1] + 5),
84         ]
85         assert exclude_omni_disk(p, shift, sides=sides) == p
86     try:
87         exclude_omni_disk([(0.0, 0.0)], (1e308, 1e308), radius=1e308)
88     except ValueError:
89         pass
90     else:
91         raise AssertionError("Unrepresentable half-plane arithmetic must be rejected")
92     return {
93         "true_source_containment_cases": count,
94         "boundary_and_degenerate_cases": 10,
95         "maximum_observed_radius_cut_m": largest_cut,
96         "scope": "sampling complements finite inscribed-polygon exclusion proof",
97     }
98
99
100 def route_checks():
101     rng = random.Random(6312000)
102     cases = 0
103     for n in (1, 2, 3, 7, 25, 41, 51):
104         for iteration in range(10):
105             pts = [
106                 (rng.uniform(-2500, 2500), rng.uniform(-2500, 2500)) for _ in range(n)
107             ]
108             pos = (rng.uniform(-1000, 1000), rng.uniform(-1000, 1000))
109             penalty = [rng.uniform(0, 750) for _ in pts]
110             origin = [math.dist(pos, p) for p in pts]
111             edges = [[math.dist(a, b) for b in pts] for a in pts]
112             control = _control_route(origin, edges, penalty)
113             improved, cost = _improve(control, origin, edges, penalty)
114             assert sorted(improved) == list(range(n)) and cost <= _cost(
115                 control, origin, edges, penalty
116             )

```

```

117         targets = {i: (p, penalty[i]) for i, p in enumerate(pts)}
118         assert choose_task(pos, {}, targets, choice="guarded") == (
119             "target",
120             control[0],
121         )
122         assert choose_portfolio_task(pos, {}, targets)[1] in targets
123         cases += 1
124     # Exact two-task optimum is useful because an open tail has no return edge.
125     for positions in (
126         ((1.0, 0.0), (2.0, 0.0)),
127         ((-1.0, 0.0), (2.0, 0.0)),
128         ((0.0, 0.0), (0.0, 0.0)),
129     ):
130         for risk in (0.0, 1.0, 2.0):
131             targets = {i: (point, float(i * 4)) for i, point in enumerate(positions)}
132             selected = choose_portfolio_task((0.0, 0.0), {}, targets, risk)
133             costs = {
134                 i: math.dist((0, 0), positions[i])
135                 + math.dist(*positions)
136                 + risk * targets[i][1]
137                 for i in range(2)
138             }
139             assert costs[selected[1]] == min(costs.values())
140     return {
141         "random_route_cases": cases,
142         "exact_two_task_cases": 9,
143         "control_never_worsened": True,
144     }
145
146
147 def clearing_checks():
148     rng = random.Random(6313000)
149     count = 0
150     saved = 0.0
151     for _ in range(200):
152         env = DeadlineAdapter(max_virtual=360000)
153         policy = FieldPlanner(env, 3, approach_clear=True)
154         policy.action("/enter")
155         c = (rng.uniform(-1500, 1500), rng.uniform(-1500, 1500))
156         radius = rng.uniform(0.01, 19.8)
157         poly = [
158             (
159                 c[0] + radius * math.cos(k * math.pi / 3),
160                 c[1] + radius * math.sin(k * math.pi / 3),
161             )
162             for k in range(6)
163         ]
164         policy.tracks[1] = Track(polygon=poly)
165         policy.channel_state[1] = "detected"
166         original = list(poly)
167         before = math.dist(policy.position, c)
168         assert policy.clear(1, c, guaranteed=True)
169         assert all(math.dist(v, policy.position) <= 19.9 for v in original)
170         assert math.dist((0, 0), policy.position) <= before + 1e-7
171         saved += policy.clear_access_saved_m
172         count += 1
173     for p in (3, 4):
174         env = DeadlineAdapter(remaining=1, max_virtual=360000)
175         result = FieldPlanner(env, p).run()
176         assert not result["certificate_complete"] and "/exit" not in env.paths
177     return {
178         "clearing_regions": count,
179         "shortened_approach_total_m": saved,
180         "real_deadline_cases": 2,
181     }
182
183

```

```

184 def main():
185     result = {
186         "evidence": "local_field_geometry_and_behavior_verification_not_official",
187         "negative_geometry": negative_checks(),
188         "routes": route_checks(),
189         "clearing": clearing_checks(),
190     }
191     Path("results/field_verification.json").write_text(
192         json.dumps(result, indent=2) + "\n"
193     )
194     print(json.dumps(result, indent=2))
195
196
197 if __name__ == "__main__":
198     main()

```

Listing 10: 竞赛版图表生成

```

1  """Generate compact competition-paper vector figures.
2
3  This module is intentionally data-light: ring/direction/localization panels are
4  schematics, while ``competition_costs.pdf`` uses only the frozen field log.
5  Run from ``b_solution`` with the project conda environment.
6  """
7  from __future__ import annotations
8
9  import hashlib
10 import json
11 import math
12 from pathlib import Path
13
14 import matplotlib
15 matplotlib.use("Agg")
16 import matplotlib.pyplot as plt
17 from matplotlib.patches import Circle, Polygon, Rectangle, FancyArrowPatch
18 import numpy as np
19
20 ROOT = Path(__file__).resolve().parent
21 FIGURES = ROOT / "report" / "figures"
22 GENERATED = ROOT / "report" / "generated"
23 BLUE, ORANGE, GREEN, GREY = "#0072B2", "#E69F00", "#009E73", "#666666"
24
25
26 def save(fig, name):
27     FIGURES.mkdir(parents=True, exist_ok=True)
28     fig.savefig(FIGURES / name, format="pdf", metadata={"Creator": "CUMCM competition assets"})
29     plt.close(fig)
30
31
32 def base(title):
33     fig, ax = plt.subplots(figsize=(16 / 2.54, 7 / 2.54), layout="constrained")
34     ax.set_title(title, fontsize=11, pad=5)
35     ax.axis("off")
36     return fig, ax
37
38
39 def taskmap():
40     fig, ax = base("四问：从几何覆盖到定向定位")
41     xs = [0.08, .31, .54, .77]; labels = ["Q1\n几何边界", "Q2\n接收盘与覆盖", "Q3\n全向清除", "Q4\n定向定位"]
42     for i, (x, lab) in enumerate(zip(xs, labels)):
43         ax.add_patch(Rectangle((x, .38), .15, .25, facecolor="#EAF2F8" if i < 2 else "#FFF2DF", edgecolor=BLUE
44             if i < 2 else ORANGE, lw=1.5))
45         ax.text(x + .075, .505, lab, ha="center", va="center", fontsize=9)
46         if i < 3: ax.add_patch(FancyArrowPatch((x+.15,.505),(xs[i+1],.505),arrowstyle="->",lw=1.4,color=GREY))
47     ax.text(.5, .18, "可行域 → 最小包围盘 → 扫描停止 → 局部测向与光学覆盖", ha="center", fontsize=8, color=GREY)
48     save(fig, "competition_taskmap.pdf")

```

```

49
50 def ring():
51     fig, ax = base("Q3 环形布点")
52     ax.set_aspect("equal"); ax.set_xlim(-2050,2050); ax.set_ylim(-2050,2050)
53     ax.add_patch(Circle((0,0),1800,fill=False,ls="--",color=GREY,lw=1))
54     u=np.array([np.cos(np.pi/6),np.sin(np.pi/6)]); g=1800*u; p=np.array([1200.,0.]);
55     angles=np.arange(6)*np.pi/3; pts=np.c_[1200*np.cos(angles),1200*np.sin(angles)]
56     ax.plot(*np.vstack([pts,pts[0]]).T,color=ORANGE,lw=1.5); ax.scatter(pts[:,0],pts[:,1],s=20,color=ORANGE,
57                                zorder=3)
58     g=np.array([1800*np.cos(np.pi/6),1800*np.sin(np.pi/6)]); nearest=1200*np.array([1.,0.]); ax.scatter(*g,
59                                color="#D55E00",marker="x",s=45,zorder=4); ax.plot([g[0],nearest[0]],[g[1],nearest[1]],color=GREEN,lw=2)
60     ax.annotate("最坏源点",xy=g,xytext=(g[0]+120,g[1]+120),fontSize=8); ax.annotate("968.90 m, 余量31.10 m",xy
61     =((g[0]+nearest[0])*0.5,(g[1]+nearest[1])*0.5),fontSize=8,color=GREEN)
62     for c in pts: ax.add_patch(Circle(c,1000,fill=False,color=BLUE,lw=.8,alpha=.65))
63     ax.text(-1750,-1930,"7 个接收盘 (半径 1000 m); 6 环点 + 原点",fontSize=8,color=BLUE)
64     save(fig,"competition_ring.pdf")
65
66 def direction():
67     fig, axes = plt.subplots(1,2,figsize=(16/2.54,7/2.54),layout="constrained")
68     for ax in axes: ax.set_aspect("equal"); ax.axis("off")
69     ax=axes[0]; ax.add_patch(Circle((0,0),1,fill=False,color=BLUE)); g=np.array([.7,.7]); ax.scatter(*g,color=
70     ORANGE); ax.arrow(*g,.35,.35,width=.015,color=ORANGE); ax.fill_between([-0.9,.9],[-0.9,.9],color=ORANGE,alpha
71     =.08); ax.text(-.9,-1.15,"g 在源圆周; 前向半平面朝外",fontSize=8)
72     ax=axes[1]; tri=np.array([[-.8,-.6],[.75,-.5],[.05,.65]]); ax.add_patch(Polygon(tri,fill=False,color=BLUE,
73     lw=1.5)); g=np.array([-0.05,.25]); n=np.array([.45,.25]); h=g+5*n; ax.scatter(*g,color=ORANGE); ax.scatter
74     (*h,color=GREEN); c=tri.mean(0); ax.scatter(*c,color=GREY); ax.plot([c[0],0],[c[1],-.95],"--",color=GREEN)
75     ; ax.text(-.9,-1.15,"含 h=g+5n 的三角形重心投影",fontSize=8); ax.text(*g," g",fontSize=8); ax.text(*h," h",
76     fontSize=8)
77     fig.text(.5,.98,"Q4 方向性与双环 (原理示意, 非观测轨迹)",ha="center",va="top",fontSize=10); save(fig,"
78     competition_direction.pdf")
79
80 def localize():
81     fig, axes = plt.subplots(1,2,figsize=(16/2.54,7/2.54),layout="constrained")
82     for ax in axes: ax.set_aspect("equal"); ax.axis("off")
83     ax=axes[0]; n=np.array([0.,1.]); z=np.array([-0.6,0.]); delta=.18; ax.plot([-1,1],[0,0],color=BLUE); ax.plot
84     ([-1,1],[delta,delta],"--",color=ORANGE); ax.plot([-1,1],[-delta,-delta],"--",color=ORANGE); ax.scatter(np
85     .linspace(-.8,.8,5),np.zeros(5),color=BLUE,s=16); ax.arrow(*z,*n,width=.01,color=GREEN); ax.text
86     (-.95,-.95,"横向测点: z plus/minus delta n",fontSize=8)
87     ax=axes[1]; ax.add_patch(Rectangle((-0.9,-.65),1.8,1.3,fill=False,color=BLUE)); xx=np.linspace(-.75,.75,5);
88     yy=np.linspace(-.5,.5,4); X,Y=np.meshgrid(xx,yy)
89     for x,y in zip(X.flat,Y.flat): ax.add_patch(Circle((x,y),.12,fill=False,color=ORANGE,lw=.7))
90     ax.scatter(X,Y,s=10,color=ORANGE); ax.text(-.9,-.95,"半径 19.9 m 圆覆盖矩形网格 (2 面板)",fontSize=8)
91     fig.text(.5,.98,"局部测向与光学覆盖 (原理示意)",ha="center",va="top",fontSize=10); save(fig,"
92     competition_localize.pdf")
93
94 def dispatch():
95     fig, ax = base("Q3 扫描闭环")
96     ax.text(.5,.78,"公开观测",ha="center",fontSize=9); ax.add_patch(FancyArrowPatch((.5,.72),(.5,.58),
97     arrowstyle="->",color=GREY))
98     ax.text(.5,.52,"选择扫描 / 定位",ha="center",fontSize=9); ax.add_patch(FancyArrowPatch((.5,.46),(.5,.31),
99     arrowstyle="->",color=GREY))
100     ax.text(.5,.24,"清除频道并更新骨架",ha="center",fontSize=9); ax.add_patch(FancyArrowPatch((.5,.18),(.3,.08),
101     connectionstyle="arc3,rad=.3",arrowstyle="->",color=GREY))
102     ax.text(.3,.03,"继续循环",ha="center",fontSize=8,color=BLUE); ax.text(.76,.08,"停止: 各频道已清除 / 全骨架无
103     信号\n或 16 个不同频道已清除",ha="center",fontSize=8,color=GREEN)
104     save(fig,"competition_dispatch.pdf")
105
106 def costs():
107     d=json.loads(ROOT/"results/field_confirmation.json").read_text(encoding="utf-8"))
108     rows=[r for r in d["runs"] if 7100000<int(r["seed"])<=7100999]
109     assert len(rows)==6000
110     groups=[]
111     for p in (3,4):

```

```

97     for cfg in (("control","控制"),("field","field")):
98         rr=[r for r in rows if r["problem"]==p and r["configuration"]==cfg[0]]; assert len(rr)==1000
99         vals=[]
100        for r in rr:
101            op=5*r["measures"]+r["switches"]+3*r["failed_clears"]+5*r["n_cleared"]
102            move=r["virtual_time_s"]-op
103            assert math.isclose(move,r["distance_m"]/5,abs_tol=1e-5)
104            vals.append((move/r["n_cleared"],op/r["n_cleared"]))
105            groups.append((f"Q{p} {cfg[1]}",np.mean(vals,axis=0)))
106    fig,ax=plt.subplots(figsize=(16/2.54,7/2.54),layout="constrained"); x=np.arange(4); mv=np.array([g[1][0]
    for g in groups]); op=np.array([g[1][1] for g in groups]); ax.bar(x,mv,color=BLUE,label="移动"); ax.bar(x,
    op,bottom=mv,color=ORANGE,label="操作"); ax.set_xticks(x,[g[0] for g in groups],fontsize=8); ax.set_ylabel
    ("T/K (s)"); ax.set_title("主组成本分解",fontsize=10); ax.legend(fontsize=8); ax.grid(axis="y",alpha=.2);
    save(fig,"competition_costs.pdf")
107 def main():
108     plt.rcParams.update({"font.size":9,"font.family":"sans-serif","font.sans-serif":["FandolSong","Arial
    Unicode MS","PingFang SC","DejaVu Sans"],"pdf.fonttype":42})
109     for f in (taskmap,ring,direction,localize,dispatch, costs): f()
110     inputs=[ROOT/"results/field_confirmation.json",ROOT/"geometry.py",ROOT/"coverage.py",Path(__file__)]
111     meta={"generator":str(Path(__file__).relative_to(ROOT)),"figures":[f"figures/competition_{n}.pdf" for n in
    ("taskmap","ring","direction","localize","dispatch","costs")],"input_sha256":{str(p.relative_to(ROOT)):
    hashlib.sha256(p.read_bytes()).hexdigest() for p in inputs}}
112     GENERATED.mkdir(parents=True,exist_ok=True); (GENERATED/"competition_metadata.json").write_text(json.dumps(
    meta,ensure_ascii=False,indent=2)+"\n",encoding="utf-8")
113
114 if __name__ == "__main__": main()

```

Listing 11: 任务调度与候选策略

```

1  """Joint, obligation-preserving open-route dispatch from public track summaries.
2
3  This module only chooses a task. It does not certify a target position, execute
4  an action, mark an anchor visited, or remove any discovery obligation.
5  """
6
7  from __future__ import annotations
8
9  from math import dist, isfinite
10
11  Point = tuple[float, float]
12  Task = tuple[str, int]
13
14
15  def choose_task(
16      position: Point,
17      anchors: dict[int, Point],
18      targets: dict[int, tuple[Point, float]],
19      *,
20      choice: str = "guarded",
21      risk_weight: float = 1.0,
22  ) -> Task:
23      """Choose the first task of a joint nearest-insertion/open-2-opt route.
24
25      ``center`` minimizes a point-center open-route surrogate. ``guarded``
26      adds the first task's MEC radius (zero for anchors), discouraging an
27      immediate commitment to an uncertain center. Neither surrogate predicts
28      localization actions or total simulator time. Rebuild inputs after every
29      completed task; a returned target must use the existing certified localizer,
30      not clear at a surrogate point.
31
32      All tasks remain in the internal permutation, even at identical positions.
33      Ties use target before anchor, then ascending integer ID; insertion gaps
34      and 2-opt segments are scanned left to right. Inputs are not modified.
35      Both mappings empty, invalid choice, nonfinite geometry, or an invalid risk
36      weight raises ValueError.
37      """
38      if choice not in ("center", "guarded"):

```

```

39     raise ValueError("choice must be 'center' or 'guarded'")
40 tasks: list[Task] = [("target", key) for key in sorted(targets)]
41 tasks.extend(("anchor", key) for key in sorted(anchors))
42 n = len(tasks)
43 if not n:
44     raise ValueError("cannot dispatch an empty task set")
45
46 coords = [targets[key][0] for key in sorted(targets)]
47 coords.extend(anchors[key] for key in sorted(anchors))
48 radii = [targets[key][1] for key in sorted(targets)]
49 radii.extend(0.0 for _ in anchors)
50 if any(len(point) != 2 or not all(isfinite(x) for x in point)
51       for point in [position, *coords]):
52     raise ValueError("positions must be finite two-dimensional points")
53 if any(not isfinite(radius) or radius < 0 for radius in radii):
54     raise ValueError("MEC radii must be finite and nonnegative")
55 if not isfinite(risk_weight) or risk_weight < 0:
56     raise ValueError("risk_weight must be finite and nonnegative")
57 penalty = [risk_weight * radius for radius in radii] if choice == "guarded" else [0.0] * n
58 origin = [dist(position, point) for point in coords]
59 edges = [[0.0] * n for _ in range(n)]
60 for i in range(n):
61     for j in range(i):
62         edges[i][j] = edges[j][i] = dist(coords[i], coords[j])
63 if (any(not isfinite(value) for value in origin)
64     or any(not isfinite(value) for row in edges for value in row)):
65     raise ValueError("pairwise distances must be finite")
66
67 # Nearest insertion: select the task nearest the current route (including
68 # the fixed origin), then insert it in the least-cost open-route gap.
69 first = min(range(n), key=lambda i: (origin[i] + penalty[i], i))
70 route = [first]
71 remaining = set(range(n))
72 remaining.remove(first)
73 nearest = [min(origin[i], edges[first][i]) for i in range(n)]
74 while remaining:
75     node = min(remaining, key=lambda i: (nearest[i], i))
76     best_gap = 0
77     best_delta = (origin[node] + edges[node][route[0]] - origin[route[0]]
78                  + penalty[node] - penalty[route[0]])
79     for gap in range(1, len(route) + 1):
80         before = route[gap - 1]
81         delta = edges[before][node]
82         if gap < len(route):
83             after = route[gap]
84             delta += edges[node][after] - edges[before][after]
85         if delta < best_delta:
86             best_gap, best_delta = gap, delta
87     route.insert(best_gap, node)
88     remaining.remove(node)
89     for other in remaining:
90         nearest[other] = min(nearest[other], edges[node][other])
91
92 # Symmetric internal edges cancel on reversal; an open tail has no return
93 # edge. A reversal starting at zero also changes the first-radius penalty.
94 # Cap accepted moves to make online CPU cost predictable, not a claim of
95 # reaching a local or global optimum.
96 for _ in range(n):
97     changed = False
98     for left in range(n - 1):
99         for right in range(left + 1, n):
100             old = (origin[route[left]] if left == 0
101                   else edges[route[left - 1]][route[left]])
102             new = (origin[route[right]] if left == 0
103                   else edges[route[left - 1]][route[right]])
104             if right + 1 < n:
105                 old += edges[route[right]][route[right + 1]]

```

```

106         new += edges[route[left]][route[right + 1]]
107     if left == 0:
108         old += penalty[route[left]]
109         new += penalty[route[right]]
110     if new + 1e-12 * max(1.0, old, new) < old:
111         route[left:right + 1] = reversed(route[left:right + 1])
112         changed = True
113         break
114     if changed:
115         break
116     if not changed:
117         break
118     return tasks[route[0]]

```

Listing 12: Q3 负观测几何

```

1  """Conservative exclusion of a station-centred open disk (Q3 only).
2
3  The routine here is deliberately independent of planner state: it only removes
4  information justified by a Q3 ``no_signal`` observation.
5  """
6
7  from __future__ import annotations
8
9  from math import cos, isfinite, pi, sin
10
11  from geometry import Point, _clip_halfplane, _cross, _padding
12
13
14  def _hull(points: list[Point]) -> list[Point]:
15      """Monotone-chain hull, retaining no duplicate vertices."""
16      pts = sorted(set(points))
17      if len(pts) <= 2:
18          return pts
19
20      lower: list[Point] = []
21      for p in pts:
22          while len(lower) >= 2 and _cross(lower[-2], lower[-1], p) <= 0.0:
23              lower.pop()
24          lower.append(p)
25      upper: list[Point] = []
26      for p in reversed(pts):
27          while len(upper) >= 2 and _cross(upper[-2], upper[-1], p) <= 0.0:
28              upper.pop()
29          upper.append(p)
30      return lower[:-1] + upper[::-1]
31
32
33  def exclude_omni_disk(
34      poly: list[Point], station: Point, radius: float = 1000.0, *, sides: int = 16
35  ) -> list[Point]:
36      """Return a conservative convex outer approximation of ``poly`` minus the open disk.
37
38      ``no_signal`` in Q3 proves that a source is strictly farther than
39      ``radius`` from ``station``. An inscribed regular polygon is used as a
40      finite certificate for the disk. Each clipped outside piece is safe, and
41      their convex hull is returned. The input list and its point objects are
42      never modified.
43      """
44      if sides not in (16, 32):
45          raise ValueError("sides must be 16 or 32")
46      if len(station) != 2 or not all(isfinite(v) for v in station):
47          raise ValueError("station must have finite coordinates")
48      if not isfinite(radius) or radius <= 0.0:
49          raise ValueError("radius must be a positive finite number")
50      if not poly:
51          return []

```

```

52 if any(len(p) != 2 or not all(isfinite(v) for v in p) for p in poly):
53     raise ValueError("polygon requires finite coordinates")
54 scale = max(1.0, radius, *(abs(v) for p in poly for v in p), *map(abs, station))
55 if not isfinite(16.0 * scale * scale):
56     raise ValueError("Geometry scale exceeds finite half-plane arithmetic")
57
58 # Q is strictly inscribed: its vertices are at radius*cos(pi/sides).
59 # This makes every Q point, including edges, safely inside the disk.
60 qradius = radius * cos(pi / sides)
61 cx, cy = station
62 q = [
63     (
64         cx + qradius * cos(2.0 * pi * i / sides),
65         cy + qradius * sin(2.0 * pi * i / sides),
66     )
67     for i in range(sides)
68 ]
69 pad = _padding(scale)
70 pieces: list[Point] = []
71
72 # Q is CCW. For edge a->b, Q satisfies cross(b-a,x-a)>=0;
73 # outside is the opposite half-plane, expressed in _clip_halfplane form.
74 for i, a in enumerate(q):
75     b = q[(i + 1) % sides]
76     ex, ey = b[0] - a[0], b[1] - a[1]
77     normal = (ey, -ex) # normal with n.(x-a) <= 0 inside Q
78     bound = 0.0
79     # If every input vertex is on this outside side, P is already safe.
80     if all(
81         normal[0] * (x - a[0]) + normal[1] * (y - a[1]) >= -pad for x, y in poly
82     ):
83         return list(poly)
84     part = _clip_halfplane(poly, a, (-normal[0], -normal[1]), -bound, pad)
85     pieces.extend(part)
86 if not pieces:
87     return []
88 return _hull(pieces)

```

Listing 13: Q3Q4 实验运行

```

1 """Finite field-informed development screen; no official counterfactual claims."""
2
3 from __future__ import annotations
4 import argparse
5 import json
6 import math
7 import statistics
8 from pathlib import Path
9 from environment import LocalSimulator
10 from field_policy import FieldPlanner
11 from planner import Planner
12
13 CONFIGS = {
14     "control": None,
15     "premeasure": {"premeasure": True},
16     "negative": {"premeasure": False, "negative_updates": True},
17     "pre_negative": {"premeasure": True, "negative_updates": True},
18     "risk2": {"premeasure": True, "risk_weight": 2.0},
19     "risk4": {"premeasure": True, "risk_weight": 4.0},
20     "risk8": {"premeasure": True, "risk_weight": 8.0},
21     "portfolio": {"premeasure": True, "portfolio": True},
22     "portfolio_negative": {
23         "premeasure": True,
24         "portfolio": True,
25         "negative_updates": True,
26     },
27     "portfolio_risk2": {"premeasure": True, "portfolio": True, "risk_weight": 2.0},

```

```

28     "portfolio_clear": {"premeasure": True, "portfolio": True, "approach_clear": True},
29     "all": {
30         "premeasure": True,
31         "portfolio": True,
32         "negative_updates": True,
33         "approach_clear": True,
34     },
35 }
36
37
38 def run_field(seed, problem, config, scenario="random", mode="smooth"):
39     env = LocalSimulator(seed, problem, mode, scenario)
40     settings = {
41         "premeasure": False,
42         "negative_updates": False,
43         "portfolio": False,
44         "approach_clear": False,
45     }
46     planner = (
47         Planner(env, problem, probe_scale=0.22)
48         if config is None
49         else FieldPlanner(env, problem, **(settings | config))
50     )
51     result = planner.run()
52     actual = env.summary()
53     assert (
54         result["certificate_complete"] and actual["n_cleared"] == actual["n_sources"]
55     ), (seed, problem, result, actual)
56     pos = (0.0, 0.0)
57     radio = 1
58     us = 0
59     successes = set()
60     for event in env.log:
61         request, response = event["request"], event["response"]
62         assert response["accepted"]
63         if request["path"] in ("/measure", "/clear"):
64             p = request["position"]["x"], request["position"]["y"]
65             us += round(math.dist(pos, p) / 5 * 1000000)
66             pos = p
67             if request["path"] == "/measure":
68                 us += (5 + int(radio != request["channel"])) * 1000000
69                 radio = request["channel"]
70             else:
71                 success = response["clear_result"] == "success"
72                 us += (5 if success else 3) * 1000000
73                 if success:
74                     assert request["channel"] not in successes
75                     successes.add(request["channel"])
76         assert abs(us / 1000000 - response["virtual_time_s"]) < 2e-6
77     assert len(successes) == actual["n_sources"]
78     return {
79         **actual,
80         "certificate_complete": True,
81         "timing_independently_verified": True,
82         "zero_travel_probes": result.get("zero_travel_probes", 0),
83         "negative_cuts": result.get("negative_cuts", 0),
84     }
85
86
87 def pair_stats(control, candidate):
88     assert [r["seed"] for r in control] == [r["seed"] for r in candidate]
89     diffs = [
90         a["average_time_s"] - b["average_time_s"]
91         for a, b in zip(control, candidate, strict=True)
92     ]
93     mean = statistics.mean(diffs)
94     se = statistics.stdev(diffs) / math.sqrt(len(diffs))

```

```

95     return {
96         "mean_saved_s": mean,
97         "saving_pct": 100
98         * mean
99         / statistics.mean(r["average_time_s"] for r in control),
100        "approx_95pct_ci": [mean - 1.96 * se, mean + 1.96 * se],
101        "faster_cases": sum(d > 0 for d in diffs),
102    }
103
104
105 def summarize(group):
106     values = sorted(r["average_time_s"] for r in group)
107     return {
108         "cases": len(group),
109         "all_clear": sum(r["fraction_cleared"] == 1 for r in group),
110         "mean": statistics.mean(values),
111         "p90": values[math.ceil(0.9 * len(values)) - 1],
112         "worst": max(values),
113         "distance": statistics.mean(r["distance_m"] for r in group),
114         "measures": statistics.mean(r["measures"] for r in group),
115         "mean_failed_clear_attempts_per_case": statistics.mean(
116             r["failed_clears"] for r in group
117         ),
118         "max_real_s": max(r["real_duration_s"] for r in group),
119     }
120
121
122 def main():
123     parser = argparse.ArgumentParser()
124     parser.add_argument("--cases", type=int, default=150)
125     parser.add_argument("--seed", type=int, default=6100000)
126     parser.add_argument(
127         "--output", type=Path, default=Path("results/field_development.json")
128     )
129     args = parser.parse_args()
130     summaries = {}
131     rows = []
132     for p in (3, 4):
133         names = (
134             (
135                 "control",
136                 "premeasure",
137                 "negative",
138                 "pre_negative",
139                 "portfolio",
140                 "portfolio_negative",
141                 "all",
142             )
143             if p == 3
144             else (
145                 "control",
146                 "premeasure",
147                 "risk2",
148                 "risk4",
149                 "risk8",
150                 "portfolio",
151                 "portfolio_risk2",
152                 "portfolio_clear",
153             )
154         )
155         control = None
156         for name in names:
157             group = [
158                 run_field(seed, p, CONFIGS[name])
159                 for seed in range(args.seed, args.seed + args.cases)
160             ]
161             for row in group:

```

```

162         row["configuration"] = name
163     rows.extend(group)
164     if control is None:
165         control = group
166     summaries[f"Q{p}_{name}"] = {
167         **summarize(group),
168         **pair_stats(control, group),
169     }
170     print(f"Q{p}_{name}", json.dumps(summaries[f"Q{p}_{name}"]), flush=True)
171 args.output.write_text(
172     json.dumps(
173         {
174             "evidence": "field_informed_local_development_not_official",
175             "seed_range": [args.seed, args.seed + args.cases - 1],
176             "configs": CONFIGS,
177             "summary": summaries,
178             "total_runs": len(rows),
179             "runs": rows,
180         },
181         ensure_ascii=False,
182         indent=2,
183     )
184     + "\n"
185 )
186
187
188 if __name__ == "__main__":
189     main()

```

Listing 14: 问题一交会计算

```

1  """Q1: actual bearing-halfplane intersection, without an artificial bounding box.
2
3  Feasibility and four coordinate LPs distinguish empty/unbounded/bounded sets.
4  For a bounded set all feasible pairwise boundary intersections are enumerated.
5  This optional Q1 analysis uses SciPy; the online search policy uses stdlib only.
6  """
7
8  from __future__ import annotations
9
10 import itertools
11 import json
12 import math
13 from pathlib import Path
14
15 import numpy as np
16 from scipy.optimize import linprog
17
18 from geometry import diameter, enclosing_circle
19
20
21 def solve_intersection(stations, bearings_deg, error_deg=1.0):
22     if len(stations) != len(bearings_deg) or not 0 <= error_deg < 90:
23         raise ValueError("Match stations/readings and use an error in [0,90) degrees")
24     if not stations:
25         return {"status": "unbounded", "vertices": [], "diameter_m": None}
26     rows, rhs = [], []
27     alpha = math.radians(error_deg)
28     for (x, y), angle in zip(stations, bearings_deg):
29         if not all(math.isfinite(value) for value in (x, y, angle)):
30             raise ValueError("Finite coordinates and bearings are required")
31         theta = math.radians(angle % 360)
32         for normal in (
33             (math.sin(theta - alpha), -math.cos(theta - alpha)),
34             (-math.sin(theta + alpha), math.cos(theta + alpha)),
35             (-math.cos(theta), -math.sin(theta)),
36         ):

```

```

37         rows.append(normal)
38         rhs.append(normal[0] * x + normal[1] * y)
39     a, b = np.asarray(rows), np.asarray(rhs)
40     feasible = linprog(
41         np.zeros(2), A_ub=a, b_ub=b, bounds=[(None, None)] * 2, method="highs"
42     )
43     if feasible.status == 2:
44         return {"status": "infeasible", "vertices": [], "diameter_m": None}
45     if not feasible.success:
46         raise RuntimeError(f"Halfplane feasibility LP failed: {feasible.message}")
47     for objective in ((1.0, 0.0), (-1.0, 0.0), (0.0, 1.0), (0.0, -1.0)):
48         bound = linprog(
49             objective, A_ub=a, b_ub=b, bounds=[(None, None)] * 2, method="highs"
50         )
51         if bound.status == 3:
52             return {"status": "unbounded", "vertices": [], "diameter_m": None}
53         if not bound.success:
54             raise RuntimeError(f"Halfplane boundedness LP failed: {bound.message}")
55     vertices = []
56     for i, j in itertools.combinations(range(len(a)), 2):
57         determinant = a[i, 0] * a[j, 1] - a[i, 1] * a[j, 0]
58         if determinant == 0:
59             continue
60         point = (
61             (b[i] * a[j, 1] - a[i, 1] * b[j]) / determinant,
62             (a[i, 0] * b[j] - b[i] * a[j, 0]) / determinant,
63         )
64         if np.max(a @ point - b) <= 1e-7 and all(
65             math.dist(point, old) > 1e-7 for old in vertices
66         ):
67             vertices.append(point)
68     if not vertices:
69         raise ArithmeticError(
70             "Bounded nonempty intersection had no numerical vertex; inspect conditioning"
71         )
72     center = tuple(np.mean(vertices, axis=0))
73     vertices.sort(
74         key=lambda point: math.atan2(point[1] - center[1], point[0] - center[0])
75     )
76     circle = enclosing_circle(vertices)
77     return {
78         "status": "bounded",
79         "vertices": vertices,
80         "diameter_m": diameter(vertices),
81         "mec_center": circle[0],
82         "mec_radius_m": circle[1],
83         "max_halfplane_residual_m": max(float(np.max(a @ p - b)) for p in vertices),
84     }
85
86
87 def main():
88     triangle = [(0.0, 0.0), (36.0, 0.0), (18.0, 18 * math.sqrt(3))]
89     length = 18 * math.sqrt(3) / math.tan(math.radians(2)) - 18
90     stations = []
91     for first, second in zip(triangle, triangle[1:] + triangle[:1]):
92         stations.append(
93             (
94                 first[0] - length * (second[0] - first[0]) / 36,
95                 first[1] - length * (second[1] - first[1]) / 36,
96             )
97         )
98     bounded = solve_intersection(stations, [1, 121, 241])
99     unbounded = solve_intersection([(0.0, 0.0)], [0.0])
100     infeasible = solve_intersection([(0.0, 0.0), (-10.0, 0.0)], [0.0, 180.0])
101     assert bounded["status"] == "bounded" and abs(bounded["diameter_m"] - 36) < 1e-6
102     assert unbounded["status"] == "unbounded" and infeasible["status"] == "infeasible"
103     result = {

```

```

104     "evidence": "Q1_direct_halfplane_algorithm_examples",
105     "bounded": bounded,
106     "unbounded": unbounded,
107     "infeasible": infeasible,
108 }
109 Path("results").mkdir(exist_ok=True)
110 Path("results/intersection_verification.json").write_text(
111     json.dumps(result, indent=2) + "\n"
112 )
113 print(json.dumps(result, indent=2))
114
115
116 if __name__ == "__main__":
117     main()

```

Listing 15: 问题二测点示例

```

1  """Q2 robust reception lens and explicitly sampled min-max geometry scoring."""
2
3  from __future__ import annotations
4
5  import json
6  import math
7  from pathlib import Path
8
9  from geometry import clip_bearing, enclosing_circle, initial_polygon
10
11
12 def safe_candidates(station=(0.0, 0.0), bearing_deg=0.0, alpha_deg=1.005):
13     alpha, angle = math.radians(alpha_deg), math.radians(bearing_deg)
14     u, n = (math.cos(angle), math.sin(angle)), (-math.sin(angle), math.cos(angle))
15     choices = []
16     for t in (125.0, 250.0, 375.0, 500.0, 625.0, 750.0, 875.0):
17         bound = min(
18             math.sqrt(1000**2 - t * t),
19             math.sqrt(1000**2 - (t - 1000 * math.cos(alpha)) ** 2)
20             - 1000 * math.sin(alpha),
21         )
22         for proportion in (0.25, 0.5, 0.75):
23             for sign in (-1, 1):
24                 b = sign * proportion * bound
25                 point = (
26                     station[0] + t * u[0] + b * n[0],
27                     station[1] + t * u[1] + b * n[1],
28                 )
29                 choices.append({"point": point, "t_m": t, "b_m": b})
30     return choices
31
32
33 def example():
34     """Normalized illustrative station/readout, not a supplied official case.
35
36     Safe reception follows the analytic lens theorem; sampled ranking does not
37     certify continuous worst-case optimality. No prior distribution is assumed.
38     """
39     first = clip_bearing(initial_polygon(), (0.0, 0.0), 0.0)
40     source_samples = [
41         (r * math.cos(math.radians(phi)), r * math.sin(math.radians(phi)))
42         for r in range(25, 1501, 25)
43         for phi in (-1.0, 0.0, 1.0)
44     ]
45     output = []
46     choices = safe_candidates() + [{"point": (750.0, 0.0), "t_m": 750.0, "b_m": 0.0}]
47     for candidate in choices:
48         q = candidate["point"]
49         worst_radius = 0.0
50         for source in source_samples:

```

```

51         assert math.dist(q, source) <= max(1000, math.dist((0, 0), source)) + 1e-7
52         if math.dist(q, source) <= 5:
53             radius = 5.0
54         else:
55             truth = math.degrees(math.atan2(source[1] - q[1], source[0] - q[0]))
56             radius = 0.0
57             for error in (-1.0, 0.0, 1.0):
58                 reading = round((truth + error) % 360, 2) % 360
59                 poly = clip_bearing(first, q, reading)
60                 radius = max(radius, enclosing_circle(poly)[1])
61             worst_radius = max(worst_radius, radius)
62         output.append(
63             {
64                 **candidate,
65                 "sampled_worst_mec_radius_m": worst_radius,
66                 "first_move_and_measure_s": math.hypot(*q) / 5 + 5,
67             }
68         )
69         best = min(
70             output,
71             key=lambda row: (
72                 row["sampled_worst_mec_radius_m"],
73                 row["first_move_and_measure_s"],
74             ),
75         )
76         result = {
77             "evidence": "illustrative_Q2_grid_score_not_global_optimum",
78             "first_station": [0, 0],
79             "first_reading_deg": 0,
80             "source_samples": len(source_samples),
81             "second_error_values_deg": [-1, 0, 1],
82             "best_sampled_geometry": best,
83             "collinear_control": output[-1],
84             "candidates": output,
85         }
86         Path("results").mkdir(exist_ok=True)
87         Path("results/second_point_example.json").write_text(
88             json.dumps(result, indent=2) + "\n"
89         )
90         print(json.dumps({k: v for k, v in result.items() if k != "candidates"}, indent=2))
91
92
93 if __name__ == "__main__":
94     example()

```

Listing 16: 覆盖点生成

```

1  """Finite, analytically justified discovery waypoint sets for CUMCM B Q3/Q4.
2
3  These functions prescribe measurement locations, not a route or a stopping rule.
4  Every unresolved channel needs its own completed coverage record. In particular,
5  Q4 waypoints outside the source arena must not be projected back into the arena.
6  """
7
8  from __future__ import annotations
9
10 import math
11
12 Point = tuple[float, float]
13
14 ARENA_RADIUS = 1800.0
15 MIN_RADIO_RADIUS = 1000.0
16 OMNI_COVER_RADIUS = 900.0
17 DEFAULT_SPACING = 990.0
18
19
20 def omni_waypoints(ring_radius: float | None = None) -> list[Point]:

```

```

21     """Return origin plus six ring points; the legacy radius certifies 900 m.
22
23     The 1200-m contracted ring instead has exact cover radius given by
24     ring_cover_radius, still below the guaranteed 1000-m reception distance.
25     No unspecified source-position prior is used by this geometric guarantee.
26     """
27     ring_radius = OMNI_COVER_RADIUS * math.sqrt(3.0) if ring_radius is None else float(ring_radius)
28     if not math.isfinite(ring_radius) or not 1200.0 <= ring_radius <= OMNI_COVER_RADIUS * math.sqrt(3.0):
29         raise ValueError("Supported ring radius is [1200, 900*sqrt(3)] m")
30     return [(0.0, 0.0)] + [
31         (
32             ring_radius * math.cos(k * math.pi / 3.0),
33             ring_radius * math.sin(k * math.pi / 3.0),
34         )
35         for k in range(6)
36     ]
37
38
39 def ring_cover_radius(ring_radius: float) -> float:
40     """Exact covering radius of origin plus six equally spaced ring points."""
41     omni_waypoints(ring_radius)
42     return max(ring_radius / math.sqrt(3.0),
43               math.sqrt(ARENA_RADIUS**2 + ring_radius**2
44                       - math.sqrt(3.0) * ARENA_RADIUS * ring_radius))
45
46
47 def directional_waypoints(spacing: float = DEFAULT_SPACING) -> list[Point]:
48     """Return a finite triangular lattice with a full source-domain guard.
49
50     For a source g inside the arena, the elementary triangle containing g has
51     vertices within spacing of g and norm <= ARENA_RADIUS + spacing.
52     If g is a lattice vertex, all six nearest neighbours satisfy that same
53     norm bound, so every emission direction has a nonzero forward neighbour.
54     No such six-neighbour claim is made for vertices outside the source arena.
55     Support 900 <= spacing <= 999; ordering is origin then row and column.
56     """
57     spacing = float(spacing)
58     if not math.isfinite(spacing) or not 900.0 <= spacing <= MIN_RADIO_RADIUS - 1.0:
59         raise ValueError("spacing must be finite and in [900, 999] m")
60
61     # With a=(s,0), b=(s/2,s*sqrt(3)/2), the squared norm of i*a+j*b
62     # is s*s*(i*i+i*j+j*j). Compare the radial cutoff using integers, so
63     # floating-point rounding cannot accidentally remove a boundary vertex.
64     numerator, denominator = spacing.as_integer_ratio()
65     guard_numerator = 1800 * denominator + numerator
66     numerator_squared = numerator * numerator
67     guard_squared = guard_numerator * guard_numerator
68     # q=i*i+i*j+j*j >= 3*i*i/4 and >= 3*j*j/4. The simpler bound
69     # |i|,|j| <= ceil(2*(1800+s)/s) safely contains the radial selection.
70     bound = (2 * guard_numerator + numerator - 1) // numerator
71     row_height = spacing * math.sqrt(3.0) / 2.0
72     points: list[Point] = [(0.0, 0.0)]
73     for j in range(-bound, bound + 1):
74         for i in range(-bound, bound + 1):
75             if i == 0 and j == 0:
76                 continue
77             norm_squared_units = i * i + i * j + j * j
78             if norm_squared_units * numerator_squared <= guard_squared:
79                 points.append((spacing * (i + j / 2.0), row_height * j))
80     return points
81
82
83 def coverage_certificate() -> dict:
84     """Return analytic constants/counts, not a sampled or experimental result.
85
86     Counts describe the default 990 m lattice. Norm shells are exact integer
87     values of i*i+i*j+j*j; their multiplicities can be checked independently.

```

```

88     No simulator actions, numerical coverage sampling, or routing occur here.
89     """
90     return {
91         "certificate_kind": "analytic_geometric_cover",
92         "arena_radius_m": ARENA_RADIUS,
93         "minimum_radio_radius_m": MIN_RADIO_RADIUS,
94         "channels": 20,
95         "omni_legacy": {
96             "waypoint_count": 7,
97             "ring_radius_m": OMNI_COVER_RADIUS * math.sqrt(3.0),
98             "cover_radius_m": OMNI_COVER_RADIUS,
99             "radio_slack_m": MIN_RADIO_RADIUS - OMNI_COVER_RADIUS,
100            "proof": (
101                "r<=900 is covered by the origin. For 900<=r<=1800, "
102                "choose a ring center with angular gap <=pi/6. Then "
103                "distance^2<=r^2-2700*r+2430000<=810000 by convexity "
104                "and equality of the two endpoint values."
105            ),
106        },
107        "omni_enhanced": {
108            "waypoint_count": 7,
109            "ring_radius_m": 1200.0,
110            "cover_radius_m": ring_cover_radius(1200.0),
111            "radio_slack_m": MIN_RADIO_RADIUS - ring_cover_radius(1200.0),
112            "proof": "Exact envelope formula ring_cover_radius(1200); no point or channel obligation is removed
113            .",
114        },
115        "omni": {
116            "waypoint_count": 7,
117            "ring_radius_m": OMNI_COVER_RADIUS * math.sqrt(3.0),
118            "cover_radius_m": OMNI_COVER_RADIUS,
119            "radio_slack_m": MIN_RADIO_RADIUS - OMNI_COVER_RADIUS,
120            "proof": "Legacy 900m certificate; enhanced 1200m certificate is in omni_enhanced.",
121        },
122        "directional": {
123            "spacing_m": DEFAULT_SPACING,
124            "basis_m": [
125                [DEFAULT_SPACING, 0.0],
126                [DEFAULT_SPACING / 2.0, DEFAULT_SPACING * math.sqrt(3.0) / 2.0],
127            ],
128            "selection_radius_m": ARENA_RADIUS + DEFAULT_SPACING,
129            "maximum_source_vertex_distance_m": DEFAULT_SPACING,
130            "radio_slack_m": MIN_RADIO_RADIUS - DEFAULT_SPACING,
131            "waypoint_count": 31,
132            "inside_or_on_arena_count": 13,
133            "outside_arena_count": 18,
134            "norm_squared_units_counts": {0: 1, 1: 6, 3: 6, 4: 6, 7: 12},
135            "maximum_waypoint_radius_m": DEFAULT_SPACING * math.sqrt(7.0),
136            "closed_halfplane_required": True,
137            "boundary_guard_required": True,
138            "proof": (
139                "An elementary triangular-lattice triangle containing g has "
140                "three vertices within spacing of g. Their norms are at most "
141                "1800+spacing, hence all are selected. Write "
142                "g=sum(lambda_i*v_i), lambda_i>=0, sum(lambda_i)=1. "
143                "For any emission normal n, sum(lambda_i*n dot (v_i-g))=0, "
144                "so at least one vertex is in the closed emission halfplane. "
145                "If g is a lattice vertex inside the source arena, all six "
146                "nearest neighbors have norm <=1800+spacing and are selected; "
147                "one makes angle <=pi/6 with any emission normal."
148            ),
149            "count_proof": (
150                "At spacing 990 the cutoff is i^2+i*j+j^2<=961/121. "
151                "The possible shells are 0,1,3,4,7 with multiplicities "
152                "1,6,6,6,12. The arena cutoff is 400/121, leaving shells "
153                "0,1,3 inside and shells 4,7 outside."
154            ),
155        },
156    }

```

```

154     },
155     "termination": (
156         "For each of 20 channels, retain either a confirmed successful "
157         "clear or a complete all-no_signal covering scan for that channel. "
158         "A detected but uncleared channel remains unresolved. With at "
159         "most one source per channel, all 20 certificates imply no "
160         "remaining source. Independently, 16 distinct confirmed clears "
161         "reach the stated global upper bound. Fewer clears alone do not."
162     ),
163     "no_signal": {
164         "q3": "Excludes every source at distance <=1000 from that detector.",
165         "q4": (
166             "Does not exclude that disk: a source may emit away from the "
167             "detector. Only the completed per-channel halfplane cover "
168             "excludes every source position and orientation."
169         ),
170     },
171     "assumptions": [
172         "Sources are stationary, persist until cleared, and use distinct channels.",
173         "Directional emission is a closed 180-degree halfplane with fixed orientation.",
174         "Reception reaches at least 1000 m throughout the emission halfplane.",
175         "All designated measurements actually complete on the indicated channel.",
176         "Near-source responses count as detections, not as no_signal.",
177         "Robot coordinates outside the radius-1800 source arena are permitted.",
178         "Geometric proofs use exact real coordinates; default float coordinates have 10 m range slack.",
179     ],
180 }

```

A.2 结果文件与复现入口

以下文件用于复现实验、图表和论文中的数值结果。

文件或目录	用途
results/research_verification.json	新覆盖、光学与行为证书核验
results/research_review.json	本轮最终独立自动审查
results/field_development.json	2250 次新策略开发运行（非样本外）
results/field_confirmation.json	9400 次新策略冻结确认运行
results/field_verification.json	新策略几何与行为有限核验
results/field_review.json	新策略独立自动审查
results/official_practice_analysis.json	用户提供的 20 轮官方演练 离线动作分解
results/fast_vs_compact_410.json	fast25 与 compact25 的本地 配对筛选负结果
results/practice_optimization_screen.json	官方演练轨迹驱动的参数筛选 及否决记录
results/ring_sweep.json	Q3 环半径开发筛选及样本 外配对证据
results/coverage_limits.json	全部 31 点不可删构造

results/refinement_verification.json	收缩环与角区间上界验证
results/geometry_verification.json	几何交叉核验与回归
results/intersection_verification.json	原始交会区域三种状态
results/http_verification.json	幂等重试与计时验证
results/http_deadline_verification.json	慢速响应绝对期限验证
results/second_point_example.json	第二测点候选评分
results/recovery_development.json	恢复策略开发对照
results/visual_review.json	PDF 逐页视觉检查记录
results/enhancement_review.json	增强策略独立审阅记录
results/paper_review.json	非官方自动审阅记录
results/final_review.json	历史增强结论与新世界证据的独立审查
report/generated/	自动数字、表格与轨迹证据
report/figures/	基础、历史增强及新研究矢量图
report/ai_usage.tex	AI 使用详情源文件
reproduce.txt、requirements.txt	命令、环境及依赖
MANIFEST.json	支撑包内文件校验值
results/route_candidate_comparison.json	路由候选负结果
results/adaptive_candidate_comparison.json	自适应测点候选负结果与子指标
results/terminal_verification.json	虚拟/现实期限终端回归
theory_additions.txt、paper_expansion.txt	独立理论草稿
report/generated/coverage_limits_table.tex	不可删性汇总表
polar_coverage_notes.txt	极坐标覆盖推导草稿
joint_dispatch_notes.txt	联合调度推导草稿
optical_policy_notes.txt	光学候选推导草稿
report/research_theory.tex	正文直接引用的新理论片段
report/generated/research_metadata.json	新图表输入哈希与分母记录
negative_geometry_notes.txt、route_portfolio_notes.txt	负反馈几何及路线组合推导
report/generated/field_metadata.json	field 图表输入哈希与分母
practice_data/manifest.csv	20 轮演练脱敏原始文件对应表
practice_data/data/	manifest 指定的 60 个脱敏原始文件
contracts.json、report_contracts.json	接口与证据口径的描述性快照

图表由make_assets.py、make_enhancement_assets.py、make_research_assets.py、make_official_assets.py 和make_field_assets.py 读取对应 JSON 生成，完整可运行代码如上。理论片段已在正文引用，不重复以代码字体排印中文。notes 文件

只记录推导，实际结果以结果 JSON 为准。入口按问题默认 Q3 refined、Q4 field, --strategy 可显式覆盖；两题探测比例均为 0.22；历史 enhanced 另列。完整复现所需审查脚本及结果也保留在支撑包中。audit_pdf.py 渲染全部页面并检查文字边界；package.py--verify 在移除 PYTHONPATH 的临时解压目录中完整重建七组核心实验，共 34610 次执行，逐项比较完整 JSON，仅排除本地墙钟 real_duration_s 及派生 max_real_duration_s、max_real_s。五份 metadata 输入哈希在初末校验。脱敏演练文件不重生，逐个核对 SHA-256，运行official_analysis.py--handoffpractice_data重新分析并比较；随后重建全部图表与编译，不以原目录存在文件代替复现通过。

未经脱敏的私有 HTTP 日志、正式加密日志及模拟器程序不进入本匿名研究包。仅收录 manifest 指定、经 robot_id=TEAM_REDACTED 及路径范围检查的 60 个演练原始文件；退出后源数、类型和案例标签只能用于离线评价。20 轮成本均由原始请求与反馈独立复算；严格旧策略动作回放仅 8 轮完整匹配，其余在首次未观测动作处停止，不造反馈。随包原始材料支持离线分析复现，不代表新策略官方成绩或全部旧策略回放通过。正式六份加密日志和真实人工审核仍未完成。